Solidum FEM

Plataforma de Análisis por Elementos Finitos en Python

Sigla: `FF-MU`

Manual de Usuario

Sintaxis del archivo YAML, catálogos de componentes,
ejemplos completos y workflow de ejecución

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

11 de noviembre de 2026

Índice general

Sobre este manual v

1. Introducción y Configuración del Entorno 1
- 1.1. Filosofía del Proyecto . . . 1
  - 1.1.1. Modelo Data-Driven . . . 1
- 1.2. Requisitos del Sistema . . . 1
  - 1.2.1. Dependencias . . . 1
  - 1.2.2. Dependencias opcionales . . . 2
- 1.3. Herramientas Externas Recomendadas . . . 2
- 1.4. Ejecución de un Análisis . . . 2

2. Arquitectura del Programa 3
- 2.1. Separación Elemento ↔ Material . . . 3
- 2.2. Auto-Registro de Componentes . . . 3
- 2.3. Validación Temprana del Input . . . 4

3. Archivo de Entrada YAML 5
- 3.1. Geometría: `nodes` y `elements` . . . 6
- 3.2. Importación de Malla Gmsh: `mesh` . . . 6
  - 3.2.1. Mapeo Avanzado: `mesh_physical_groups` . . . 7
- 3.3. Materiales . . . 7
- 3.4. Secciones de marco: `sections` . . . 8
- 3.5. Módulos de usuario: `user_modules` . . . 11
- 3.6. Condiciones de Frontera . . . 12
  - 3.6.1. Por nodo: `boundary_conditions_by_node` . . . 12
  - 3.6.2. Por coordenada: `boundary_conditions_by_coord` . . . 12
  - 3.6.3. Por grupo físico: `boundary_conditions_by_group` . . . 12
  - 3.6.4. Restricciones multipunto (MPC): `linear_constraints` . . . 12
  - 3.6.5. Refuerzo embebido: `constraints` . . . 14
- 3.7. Cargas Puntuales . . . 15
- 3.8. Masas Concentradas en Nudos: `nodal_masses` . . . 15
- 3.9. Fuerzas de Cuerpo . . . 16
  - 3.9.1. Forma general: `body_force` . . . 17
  - 3.9.2. Caso particular — peso propio: `gravity` + `density` . . . 17
  - 3.9.3. Detalles transversales . . . 17

3.10. Cargas Distribuidas de Contorno: `tractions` y `tractions_by_group` 18
3.11. Valores Variables: Expresiones 18
3.12. Solver 19
3.12.1. Bloque `convergence` 19
3.13. Etapas: `stages` 20
3.14. Salida: `output` 21

4. Catálogo de Elementos Finitos **23**
4.1. Elementos 1D 23
4.1.1. Armaduras: `Truss2D` / `Truss2DCorot` / `Truss3D` / `Truss3DCorot` / `Truss2D3` / `Truss3D3` 23
4.1.2. Cables: `Cable2DCorot` / `Cable3DCorot` 24
4.1.3. Marcos 2D: `Frame2DEuler` / `Frame2DTimoshenko` / `Frame2DEulerCorot` 25
4.1.4. Marco 3D: `Frame3D` 26
4.2. Elementos 2D 27
4.2.1. Cuadrilátero Bilineal: `Quad4` 27
4.2.2. Triángulo de Deformación Constante: `Tri3` 27
4.2.3. Cuadrilátero Serendípito de Orden 2: `Quad8` 27
4.2.4. Cuadrilátero Lagrangiano de Orden 2: `Quad9` 28
4.2.5. Triángulo Cuadrático Completo P_2: `Tri6` 28
4.3. Elementos 3D 28
4.3.1. Hexaedro Trilineal: `Hex8` 29
4.3.2. Tetraedro Lineal: `Tet4` 29
4.3.3. Hexaedro Serendípito de Orden 2: `Hex20` 29
4.3.4. Hexaedro Lagrangiano Triquadrático: `Hex27` 30
4.3.5. Tetraedro Cuadrático: `Tet10` 30
4.4. Módulo de usuario `discontinuities`: elemento con discontinuidad embebida . . . 31
4.4.1. Triángulo CST con Discontinuidad Interior: `CST_Embedded2D` 31

5. Catálogo de Modelos Constitutivos **33**
5.1. Elasticidad Lineal 33
5.1.1. `Elastic1D` — elasticidad lineal axial 33
5.1.2. `Elastic2D` — elasticidad lineal isótropa 2D 33
5.2. Plasticidad 33
5.2.1. `Elastoplastic1D` — plasticidad J2 axial con endurecimiento isótropo . 33
5.2.2. `VonMises2D` — plasticidad J2 plana con endurecimiento isótropo 34
5.2.3. `DruckerPrager2D` — plasticidad friccional cohesivo-friccional 2D 35
5.3. Daño Mecánico 36
5.3.1. `IsotropicDamage1D` — daño isótropo 1D con softening exponencial . . . 36
5.3.2. `IsotropicDamage2D` — daño isótropo 2D 36
5.3.3. `IsotropicDamageCrackBand2D` — daño isótropo con banda de grieta (fisuración distribuida) 38
5.4. Elasticidad Unilateral (Cable) 39
5.4.1. `CableMaterial1D` — cable lineal en tracción, nulo en compresión 39
5.5. Concreto 39
5.5.1. `ManderConcrete1D` — concreto uniaxial con la envolvente de Mander . . 39

5.6. Materiales 3D . . . 40
5.6.1. `Elastic3D` — elástico lineal isótropo 3D . . . 41
5.6.2. `VonMises3D` — plasticidad J2 3D con endurecimiento isótropo lineal . . . 41
5.6.3. `DruckerPrager3D` — plasticidad friccional 3D . . . 41
5.6.4. `IsotropicDamage3D` — daño isótropo 3D con softening exponencial . . . 41
5.7. Materiales de interfaz (tracción frente a desplazamiento relativo) . . . 42
5.7.1. `CohesiveDamageIsotropic` — daño cohesivo Modo-I . . . 42
5.7.2. `CohesiveDamageMixedMode` — daño cohesivo de modo mixto I–II . . . 43
5.7.3. `BondSlipModelCode` y `BondSlipMultilinear` — adherencia-deslizamiento barra–concreto . . . 43

6. Esquemas de Solución **45**
6.1. `LinearSolver` — solución directa lineal . . . 45
6.2. `NonlinearSolver` — Newton-Raphson incremental con paso adaptativo . . . 46
6.3. `ArcLengthSolver` — longitud de arco cilíndrico (Crisfield) . . . 46
6.4. Análisis por etapas: gravedad primero, empuje después . . . 48
6.4.1. Casos lineales sobre una etapa (ADR 0031) . . . 49
6.5. `IndirectDisplacementSolver` — control indirecto de desplazamiento . . . 49
6.6. Módulo de usuario `dissipation_control`: arc-length por disipación de energía . . . 51
6.6.1. `DissipationArcLengthSolver` — arc-length por disipación de energía . . . 51

7. Análisis Dinámico **53**
7.1. `ModalSolver` — análisis modal por autovalores generalizados . . . 54
7.1.1. Modal sobre el estado de una etapa (ADR 0031) . . . 55
7.1.2. Revisar las masas: masa total y centro de masa . . . 55
7.1.3. Masas de nudo y grados de libertad sin masa (ADR 0024) . . . 56
7.2. `NewmarkSolver` — transitorio lineal Newmark-β . . . 57
7.3. `NewtonNewmarkSolver` — transitorio no lineal Newton-Newmark . . . 58
7.4. `HHTSolver` y `NewtonHHTSolver` — variantes Hilber-Hughes-Taylor . . . 59
7.5. `CentralDifferenceSolver` — explícito por diferencias centradas (ADR 0009 fase 5) 60
7.6. Cómo termina un análisis en el tiempo (ADR 0025) . . . 60
7.7. Reacciones y cortante basal (ADR 0030) . . . 61
7.8. Paso adaptativo en los transitorios no lineales . . . 62
7.9. Progreso paso a paso . . . 62
7.10. Continuar un análisis (ADR 0029) . . . 62
7.11. `HarmonicSolver` — respuesta forzada armónica en frecuencia (ADR 0009 fase 6) . 63
7.12. `ResponseSpectrumSolver` — análisis sísmico por combinación modal (ADR 0009 fase 7) . . . 64
7.13. Mass lumping (ADR 0009 fase 2) . . . 65

8. Análisis Térmico **67**
8.1. Formulación . . . 67
8.2. Material: `ThermalConduction` . . . 67
8.3. Elementos: `Quad4Thermal` y `Hex8Thermal` . . . 68
8.4. Condiciones de frontera . . . 69
8.4.1. Dirichlet — temperatura impuesta . . . 69

8.4.2. Restricciones multipunto sobre `T` . . . 69
8.4.3. Neumann — flujo prescrito . . . 69
8.4.4. Fuente volumétrica . . . 70
8.5. Régimen estacionario . . . 70
8.6. Régimen transitorio: `ThetaMethodSolver` . . . 71
8.6.1. Continuar un análisis . . . 71
8.6.2. Elección de θ . . . 71
8.6.3. Elección de Δt . . . 72
8.6.4. Capacidad `lumped` por defecto . . . 72
8.6.5. Resultado . . . 72
8.7. Ejemplo completo: pared plana en régimen transitorio . . . 73
8.8. Limitaciones actuales . . . 74

9. Post-Procesamiento y Visualización **75**
9.1. Formato VTU (VTK XML) . . . 75
9.1.1. Datos Nodales (Point Data) . . . 75
9.1.2. Datos de Elemento (Cell Data) . . . 76
9.2. Animación con Archivo PVD . . . 76
9.3. Salida en Texto Plano (Estilo FEAP) . . . 76
9.4. Workflow ParaView . . . 77

10. Ejemplos Completos **78**
10.1. Marco 2D — Viga en Voladizo . . . 78
10.2. Placa Continua con Plasticidad J2 (Gmsh) . . . 79
10.3. Cable Pretensado en Régimen Corotacional . . . 80

11. Uso Avanzado: API Programática **82**
11.1. Patrón Fachada . . . 82
11.2. Un modelo YAML desde Python: `load_yaml` . . . 83
11.3. Resultado de un análisis estático: `SolveResult` . . . 84
11.4. Cargas distribuidas y sus cargas equivalentes . . . 85
11.5. Secciones por fibras . . . 85
11.6. Módulos de usuario . . . 86
11.7. Masas concentradas en nudos . . . 86
11.8. Refuerzo embebido . . . 87
11.9. Errores tipados . . . 87

12. Diagnóstico de Problemas **89**

Sobre este manual

Este manual está dirigido al **usuario final** de Solidum FEM: cubre la sintaxis del archivo de entrada `.yaml`, el catálogo completo de elementos, materiales y solvers, ejemplos de uso y el workflow de post-procesamiento. No entra al detalle de la implementación interna ni a la formulación matemática completa de cada componente.

Para la *referencia formal* de cada componente (ecuaciones, matrices $\mathbf{B}$, criterios de aceptación), consultar `manuals/Reference_manual.pdf`, generado automáticamente desde `docs/specs/` (programa principal) y `docs/user/*/specs/` (apéndice *Módulos de usuario*).

Para la *visión arquitectural* del programa (capas, bloques funcionales, mecanismos transversales y dirección de evolución), consultar `manuals/Architecture_manual.pdf`.

Este manual se regenera con:

```
python manuals/build_user_manual.py
```

No editar este PDF manualmente; cualquier corrección debe hacerse sobre los archivos fuente en `manuals/sources/user/`.

Capítulo 1

Introducción y Configuración del Entorno

1.1 Filosofía del Proyecto

Solidum FEM nace como herramienta de investigación en mecánica del medio continuo y análisis no lineal de estructuras. Su diseño persigue dos objetivos simultáneos:

1. **Rigor numérico y físico.** Cada formulación incorporada está validada contra solución analítica o benchmark establecido. La separación estricta `Elemento` ↔ `Material` permite combinar cualquier cinemática con cualquier ley constitutiva siempre que sus protocolos sean compatibles.

2. **Crecimiento sin fricción.** La arquitectura está optimizada para que añadir un elemento, material o solver nuevo sea barato. Auto-registro vía decoradores, contratos declarativos (`STRAIN_DIM`, `DOF_NAMES`) y validación temprana del input son los mecanismos centrales.

1.1.1 Modelo Data-Driven

Toda la orquestación de un análisis se concentra en un archivo `.yaml` legible por humanos. El usuario no necesita escribir código Python para definir un modelo; la API programática queda reservada para casos avanzados (optimización paramétrica, mallas generadas proceduralmente, acoplamiento con otros sistemas).

1.2 Requisitos del Sistema

Solidum FEM se ejecuta sobre Python 3.10 o superior en Windows, Linux y macOS sin compilación nativa.

1.2.1 Dependencias

```
pip install -e .            # desde la carpeta del proyecto: instala Solidum y sus
    dependencias
```

- `numpy / scipy`: álgebra lineal densa y dispersa; SuperLU como solver directo disponible siempre.
- `pyyaml`: análisis sintáctico del archivo de entrada.
- `meshio`: importación de mallas Gmsh y exportación a VTK.
- `numba`: compilación JIT de los kernels críticos en elementos sólidos 2D y 3D y en los modelos de plasticidad.

1.2.2 Dependencias opcionales

Solidum funciona completo sin ellas; con ellas, los modelos grandes se resuelven mucho más rápido o caben en memoria. Se instalan como *extras*:

```
pip install -e .[fast]         # Intel MKL Pardiso: solver directo multihilo (
    recomendado)
pip install -e .[iterative]    # pyamg: precondicionador del solver iterativo
```

- `pypardiso` (`[fast]`): con él, el solver del sistema lineal —que en un análisis grande se lleva la mayor parte del tiempo— pasa a ser multihilo; entre 5 y 56 veces más rápido según el tamaño del modelo. Se usa automáticamente, sin tocar el YAML.
- `pyamg` (`[iterative]`): sólo lo usa el solver iterativo, que se pide expresamente (`linear_algebra: iterative`) para modelos demasiado grandes para la memoria. En Python 3.14 sobre Windows no trae binarios y hay que compilarlo; ver el anexo de la capa algebraica del manual de referencia.
- `scikit-sparse` (`[cholmod]`): factorización de Cholesky; en Windows se instala con `conda`, no con `pip`.

1.3 Herramientas Externas Recomendadas

- **Gmsh**: generador de mallas; produce archivos `.msh` consumibles directamente desde el YAML.
- **ParaView**: visualización interactiva de los archivos `.vtu` generados.

1.4 Ejecución de un Análisis

El runner principal vive en `examples/ejecutar_yaml.py`. Para ejecutar un modelo:

```
python examples/ejecutar_yaml.py ruta/al/modelo.yaml
```

El script carga el YAML, valida su contenido (acumulando *todos* los errores en una sola pasada antes de abortar), construye el dominio, ensambla el solver pedido, ejecuta el análisis y exporta los resultados según el bloque `output`.

Capítulo 2

Arquitectura del Programa

2.1 Separación Elemento ↔ Material

Cada elemento finito declara dos contratos sobre el material que acepta:

- `STRAIN_DIM`: dimensión Voigt de la deformación que entrega al material. `1` para elementos axiales (truss, cable, marcos — la fibra centroidal); `3` para 2D (`[εxx, εyy, γxy]`); `6` para 3D continuo (`[εxx, εyy, εzz, γxy, γyz, γxz]`, ADR 0012).
- `DOF_NAMES`: lista de grados de libertad por nodo. Por ejemplo, `['ux', 'uy', 'rz']` para un marco 2D.

El protocolo del material es uniforme:

```
sigma, E_tangent, new_state = material.compute_state(epsilon, state_vars)
```

donde `epsilon` tiene dimensión `STRAIN_DIM`, `sigma` es el esfuerzo en la misma representación, `E_tangent` es la matriz tangente consistente y `state_vars` encapsula las variables internas (deformación plástica, daño acumulado, etc.).

Un material declara igualmente su `STRAIN_DIM`; el dominio rechaza al construir cualquier emparejamiento incompatible.

2.2 Auto-Registro de Componentes

Elementos, materiales (mecánicos, térmicos y de interfaz), secciones de marco, solvers y generadores de restricciones se registran automáticamente al arrancar el programa mediante decoradores:

```
@ElementRegistry.register
class Truss2D(Element):
    DOF_NAMES = ['ux', 'uy']
    STRAIN_DIM = 1
    ...
```

Esto permite que el archivo YAML los referencie por nombre (`type: Truss2D`) sin que el usuario tenga que importar nada manualmente. El módulo `solidum.autodiscover` recorre los paquetes `elements/`, `materials/` (incluidas las leyes de interfaz de `materials/interface/`), `sections/`,

`math/` y `bc/` (generadores de restricciones, ADR 0032) en la primera importación y puebla los registries. Los módulos de usuario (`solidum/user/<nombre>/`, formulaciones no estándar) quedan fuera de ese recorrido: se cargan sólo cuando un modelo los pide con `user_modules` (capítulo *Archivo de Entrada YAML*).

2.3 Validación Temprana del Input

El parser YAML acumula todos los errores detectados en una sola pasada (no aborta al primero) y los presenta juntos. Comprueba:

- Ausencia de bloques obligatorios: la geometría (`nodes` o `mesh`) y, si hay `elements`, un bloque de materiales. El `solver` del nivel raíz es `LinearSolver` si se omite; con `stages`, cada etapa declara el suyo y el del nivel raíz no se admite.
- IDs duplicados de nodo, material o elemento.
- Referencias a materiales o nodos inexistentes desde `elements` y `boundary_conditions`.
- Tipos de elemento, material y solver no registrados.
- **Parámetros no aceptados por el constructor del elemento.** Si un elemento define `A` e `I` como parámetros, escribir `large_strains: true` en su entrada YAML produce un error explícito que indica los parámetros válidos para ese tipo.

Capítulo 3

Archivo de Entrada YAML

La interacción exclusiva con el motor de Solidum FEM se realiza mediante el archivo de entrada `.yaml`. Un archivo válido contiene los bloques siguientes (algunos opcionales según el caso de uso):

- `nodes` *o* `mesh` — geometría.
- `materials` — catálogo de leyes constitutivas instanciadas.
- `interface_materials` — leyes tracción–desplazamiento relativo de una interfaz (leyes cohesivas de grieta, adherencia de barras), sólo si algún elemento o restricción las usa.
- `sections` — secciones de marco (sección por fibras), sólo si algún marco usa `section`; sus fibras referencian materiales de `materials`.
- `elements` — conectividad (omisible si se importa malla Gmsh).
- `boundary_conditions_by_node` / `_by_coord` / `_by_group` — restricciones de Dirichlet.
- `linear_constraints` — restricciones multipunto (MPC) escritas a mano.
- `constraints` — generadores de restricciones (refuerzo embebido, ADR 0032).
- `point_loads_by_node` / `_by_coord` / `_by_group` — cargas de Neumann puntuales.
- `nodal_masses` / `nodal_masses_by_group` / `nodal_masses_by_coord` — masas concentradas en nudos (ADR 0024).
- `gravity`, `body_force`, `tractions` / `tractions_by_group`, `thermal_loads` — cargas distribuidas y térmicas.
- `solver` — tipo de solver y sus parámetros.
- `stages` — análisis por etapas (ADR 0026, 0029 y 0031); con etapas, el `solver` y las cargas van dentro de cada etapa y no en el nivel raíz.
- `output` — frecuencia y formato de exportación de resultados.

- `user_modules` — módulos de usuario que el modelo necesita (sólo si usa una formulación no estándar). Un módulo puede añadir su propia sección (`crack_trackers` del módulo `discontinuities`).

Una sección de primer nivel que el lector no conoce es un error, no se ignora: una errata en el nombre de un bloque detiene la lectura en vez de dejar el modelo sin ese bloque.

3.1 Geometría: `nodes` y `elements`

Para modelos pequeños o estructuras reticulares (armaduras, marcos, cables), la geometría se escribe directamente en el YAML:

```yaml
nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [2.0, 0.0]}
  - {id: 3, coords: [4.0, 0.0]}

elements:
  - id: 1
    type: Frame2DEuler
    material: 1
    nodes: [1, 2]
    A: 0.01
    I: 8.33e-6
```

Las coordenadas pueden ser de 2 o 3 componentes; los elementos 1D 3D (`Truss3D`, `Cable3DCorot`, `Frame3D`) requieren las tres.

3.2 Importación de Malla Gmsh: `mesh`

Para problemas continuos 2D, se sustituye el bloque `nodes`/`elements` por una referencia al archivo `.msh`:

```yaml
mesh: "geometria_placa.msh"
mesh_material: 1
mesh_thickness: 0.1
mesh_quadrature: "2x2"
```

Parámetro	Descripción
`mesh`	Ruta al archivo `.msh` de Gmsh, relativa al YAML.
`mesh_material`	ID del material por defecto que se asocia a toda la malla.
`mesh_thickness`	Espesor por defecto (estado plano 2D).
`mesh_quadrature`	Regla de Gauss: `"2x2"` (completa, recomendada) o `"1x1"` (reducida; produce *hourglassing*). La masa consistente y la capacidad térmica se integran siempre con la regla completa, con independencia de la de `K`. Los elementos sin cuadratura (`Tri3`) la ignoran.
`mesh_element`	Opcional. Tipo de elemento de toda la malla, p. ej. `Quad4Thermal` para un modelo térmico. Sin él, cada celda gmsh da el elemento estándar: `quad` → `Quad4`, `triangle` → `Tri3`. El número de nodos del elemento debe coincidir con el de la celda.

`mesh_material` se busca en la familia de material del elemento: en `materials` para los mecánicos, en `thermal_materials` para los térmicos.

3.2.1 Mapeo Avanzado: `mesh_physical_groups`

Si la geometría tiene zonas con materiales, espesores o elementos distintos definidos como *Physical Groups* en Gmsh, se mapean explícitamente:

```yaml
mesh_physical_groups:
  "Zona_Hormigon":
    material: 1
    thickness: 0.20
    quadrature: "2x2"
  "Zona_Acero":
    material: 2
    thickness: 0.05
```

Un grupo también puede elegir su propio `element`, con cualquier parámetro que acepte ese elemento. Sin `element`, un grupo sólo admite `material`, `thickness` y `quadrature`: cualquier otra clave es un error, no se ignora.

3.3 Materiales

Cada material se declara con un `id` único y un `type` registrado en el catálogo:

```yaml
materials:
  - id: 1
    type: Elastic1D
    E: 200.0e9
    density: 7850.0
  - id: 2
    type: VonMises2D
    E: 210.0e9
    nu: 0.3
    sigma_y: 250.0e6
    H: 2.0e9
    hypothesis: plane_strain
```

```
    density: 7850.0
```

Los parámetros aceptados dependen del material; consulte el capítulo *Catálogo de Modelos Constitutivos* para el catálogo completo.

Las leyes de una **interfaz** (tracción frente a desplazamiento relativo: la ley cohesiva de una grieta, la adherencia de una barra) no van en `materials` sino en su propia sección, `interface_materials`, con la misma forma (`id`, `type` y parámetros); los elementos y restricciones que las usan las citan por `id`.

3.4 Secciones de marco: `sections`

Una **sección de marco** es la ley constitutiva de un marco a nivel de sección transversal: recibe la deformación axial y las curvaturas del eje en cada punto de Gauss y devuelve las fuerzas internas de la sección (N, M_z en 2D; N, M_y, M_z en 3D). Hoy existe un tipo, `FiberSection` (ADR 0021): la sección se discretiza en **fibras**, cada una con su propio material 1D y su propia historia, de modo que la fluencia o el daño avanzan fibra a fibra. Es la forma de modelar **plasticidad o daño en un marco**: con `material` + `A`, `I`, los marcos sólo admiten un material elástico lineal.

Una sección se declara una vez en el bloque `sections` y la comparten los marcos que la citan con `section: <id>` (`Frame2DEuler`, `Frame2DEulerCorot` y `Frame3D`; ver el capítulo *Catálogo de Elementos Finitos*). Las fibras se generan con **parches** y **fibras sueltas**:

Tabla 3.1: Primitivas de una FiberSection.

Primitiva	Claves	Fibras que genera
`shape: rectangle`	`material`, `y: [y_min, y_max]`, `z: [z_min, z_max]`, `ny`, `nz`	Rejilla de `ny` × `nz` celdas iguales, una fibra en el centro de cada celda con el área de la celda.
`shape: circle`	`material`, `r_outer`, `r_inner` (default `0.0`, maciza), `center: [y, z]` (default `[0.0, 0.0]`), `n_rad`, `n_circ`	Corona dividida en `n_rad` anillos y `n_circ` sectores; una fibra en el centroide de cada sector, con su área exacta. Ángulos desde `+y` hacia `+z`; el primer sector empieza en 0.
`shape: layer`	`material`, `start: [y, z]`, `end: [y, z]`, `n`, `area`, `displaces` (opcional)	`n` barras de área `area` equiespaciadas de `start` a `end`, extremos incluidos (`n: 1`: una barra en `start`).
fibra suelta (en `fibers`)	`y`, `z`, `area`, `material`, `displaces` (opcional)	Una fibra.

Además, `GJ` (opcional): rigidez torsional elástica GJ de la sección, **obligatoria si la usa un** `Frame3D`. La torsión es elástica y no interactúa con la flexión ni se degrada cuando las fibras fluyen.

Coordenadas y signos. `y`, `z` son coordenadas en los **ejes locales del elemento**, medidas desde el **eje del elemento** (la línea que une sus nodos), no desde el centroide: una sección excéntrica acopla axial y flexión. En 2D, y es la dirección transversal del plano (z sólo da el ancho); en 3D los ejes locales los fija `ref_vector`. $M_z > 0$ tracciona las fibras con $y < 0$ y $M_y > 0$ las fibras con $z > 0$. Materiales de fibra admitidos: cualquiera de `STRAIN_DIM = 1` (`Elastic1D`, `Elastoplastic1D`, `IsotropicDamage1D`…); una misma sección puede mezclarlos (concreto y acero).

Ejemplo 2D mínimo — voladizo de acero elastoplástico, sección rectangular de 0.3 m × 0.5 m en 20 capas, llevado más allá de la primera fluencia ($M(0) = 3.9$ MN·m entre $M_y = 3.125$ MN·m y

$M_p = 4.6875$ MN·m):

```yaml
nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [1.0, 0.0]}
  - {id: 3, coords: [2.0, 0.0]}
  - {id: 4, coords: [3.0, 0.0]}

materials:
  - {id: 1, type: Elastoplastic1D, E: 2.0e11, sigma_y: 2.5e8, H: 2.0e9, density:
      7850.0}

sections:
  - id: 1
    type: FiberSection
    patches:
      - {shape: rectangle, material: 1, y: [-0.25, 0.25], z: [-0.15, 0.15], ny: 20, nz:
          1}

elements:
  - {id: 1, type: Frame2DEuler, nodes: [1, 2], section: 1}
  - {id: 2, type: Frame2DEuler, nodes: [2, 3], section: 1}
  - {id: 3, type: Frame2DEuler, nodes: [3, 4], section: 1, n_ip: 4}

boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}

point_loads_by_node:
  - {node_id: 4, uy: -1.3e6}

solver:
  type: NonlinearSolver
  num_steps: 10
```

Con `nz: 1` el parche es una pila de capas: basta para la flexión en el plano. `n_ip` (puntos de Gauss a lo largo del elemento) vale 3 si no se declara.

Ejemplo 3D con `GJ` — voladizo de un tubo de acero (radios 0.15 m y 0.13 m) con carga transversal en z y un momento torsor en la punta; `ref_vector: [0, 1, 0]` hace coincidir los ejes locales con los globales:

```yaml
nodes:
  - {id: 1, coords: [0.0, 0.0, 0.0]}
  - {id: 2, coords: [1.5, 0.0, 0.0]}
  - {id: 3, coords: [3.0, 0.0, 0.0]}

materials:
  - {id: 1, type: Elastoplastic1D, E: 2.0e11, sigma_y: 2.5e8, H: 2.0e9}

sections:
  - id: 1
    type: FiberSection
    GJ: 2.67e7                  # obligatoria con Frame3D
    patches:
      - {shape: circle, material: 1, r_outer: 0.15, r_inner: 0.13, n_rad: 2, n_circ:
          32}
```

```yaml
elements:
  - {id: 1, type: Frame3D, nodes: [1, 2], section: 1, ref_vector: [0.0, 1.0, 0.0]}
  - {id: 2, type: Frame3D, nodes: [2, 3], section: 1, ref_vector: [0.0, 1.0, 0.0]}

boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0, uz: 0.0, rx: 0.0, ry: 0.0, rz: 0.0}

point_loads_by_node:
  - {node_id: 3, uz: -1.0e5, rx: 1.0e4}

solver:
  type: NonlinearSolver
  num_steps: 10
```

En el empotramiento resultan $M_y = +PL = 3.0 \times 10^5$ N·m (tracción en las fibras con $z > 0$, por encima del momento de primera fluencia del tubo, 2.89×10^5 N·m) y $T = 10^4$ N·m.

Concreto armado: barras que desplazan el concreto. Una sección de concreto armado combina un parche de concreto y capas de barras de acero dentro de él. Las fibras del parche cubren toda la sección, también el sitio que ocupan las barras, así que, si no se indica otra cosa, en la posición de cada barra el área cuenta dos veces: la fuerza axial de la barra es $(\sigma_s + \sigma_c)\, A_b$ en lugar de $\sigma_s A_b$, y lo mismo la rigidez, la masa y el peso propio. Es la práctica de OpenSees y el comportamiento por defecto. El error suele ser pequeño (cuantía de acero del 1 al 3 %), pero es sistemático. Para descontarlo, cada capa de barras o fibra suelta declara con `displaces` el material que desplaza, el del parche que la contiene:

```yaml
materials:
  - {id: 1, type: IsotropicDamage1D, E: 3.0e10, kappa_0: 1.0e-4, alpha: 0.99, density:
      2400.0}   # concreto
  - {id: 2, type: Elastoplastic1D, E: 2.0e11, sigma_y: 4.2e8, H: 2.0e9, density:
      7850.0}        # acero

sections:
  - id: 2
    type: FiberSection
    patches:
      - {shape: rectangle, material: 1, y: [-0.25, 0.25], z: [-0.15, 0.15], ny: 40, nz:
          1}
      - {shape: layer, material: 2, start: [-0.21, -0.11], end: [-0.21, 0.11], n: 3,
          area: 5.1e-4, displaces: 1}
      - {shape: layer, material: 2, start: [0.21, -0.11], end: [0.21, 0.11], n: 2, area
          : 3.1e-4, displaces: 1}
```

En la posición de cada barra con `displaces`, la sección añade una **fibra de compensación** de área $-A_b$ con el material desplazado. Esta fibra se evalúa con la ley de ese material y con su propia historia (daño, fluencia), así que el concreto queda con su área neta $A_c - \sum A_b$ en las fuerzas internas, en la rigidez, en la masa y en el área bruta, también en régimen inelástico. `displaces` es opcional en cada primitiva (sólo `layer` y las fibras sueltas lo admiten) y es un id de `materials`: el del material de alguna fibra declarada de la sección, distinto del de la propia barra. Sin `displaces`, las barras se suman al concreto.

Validación al leer el archivo. Son errores, con el contexto de la entrada (p. ej. `sections[0] (id=7), patches[0].material: referencia a material inexistente (9)`): una clave desconocida en un parche, una `shape` inexistente, un material de fibra inexistente o de dimensión distinta de

1, y una sección sin fibras. También lo son un `displaces` que nombre un material inexistente, de dimensión distinta de 1, igual al de la barra o que no sea el de ninguna fibra de la sección, y un `displaces: true`: no es una bandera sino el id del material desplazado. Un marco que declara a la vez `section` y `material` (o `A`, `I`…) también es un error.

Masa y peso propio. Salen de las fibras: $\rho A = \sum \rho_i A_i$ y los momentos de inercia másicos $\sum \rho_i y_i^2 A_i$, $\sum \rho_i z_i^2 A_i$, con las fibras de compensación incluidas (con `displaces`, $\rho A = \rho_c(A_c - \sum A_b) + \rho_s \sum A_b$). Los marcos Euler-Bernoulli desprecian la inercia rotatoria de flexión, así que su masa sólo usa ρA; los momentos de inercia másicos entran únicamente en la inercia torsional de `Frame3D` ($\sum \rho_i (y_i^2 + z_i^2) A_i$). Cada material de fibra necesita `density` si el análisis usa `gravity` o la matriz de masa.

Advertencia

Límites de la sección por fibras. (1) La curvatura es lineal dentro del elemento: una rótula plástica se reparte sobre el elemento y la carga de colapso **converge por arriba** al refinar (en una viga biempotrada, +35 % con 4 elementos en el vano y +3.5 % con 32). Use varios elementos por miembro cerca de las rótulas. (2) Con fibras en **ablandamiento** (`IsotropicDamage1D`) el daño se localiza en uno o dos puntos de Gauss y la respuesta posterior al pico depende de la malla y de `n_ip`: no hay regularización. (3) Sin deformación por cortante: `Frame2DTimoshenko` no admite sección todavía.

3.5 Módulos de usuario: `user_modules`

El **programa principal** de Solidum contiene los elementos, materiales y solvers de elementos finitos estándar. Las formulaciones no estándar viven aparte, en **módulos de usuario** (`solidum/user/<nombre>/`), a la manera de los elementos, materiales y macros de usuario de FEAP (ADR 0020). Un módulo de usuario puede aportar tipos de elemento, materiales, solvers y secciones de primer nivel propias, pero el programa principal no lo carga por sí solo: el modelo que lo necesita lo declara.

```yaml
user_modules: [discontinuities]
```

Los módulos declarados se cargan al principio de la lectura, antes de validar el resto del archivo, de modo que sus tipos y secciones se validan como los del programa principal. La carga es explícita a propósito: el archivo dice qué formulación necesita, y el mismo modelo no funciona en una sesión y falla en otra según lo que se haya importado antes. Si el YAML usa un tipo o una sección de un módulo sin declararlo, el error lo dice y lista los módulos disponibles; un nombre de módulo inexistente también es un error.

Hoy existen dos módulos de usuario:

- `discontinuities`: el elemento con discontinuidad embebida `CST_Embedded2D` y sus seguimientos de grieta, con la sección `crack_trackers` (capítulo *Catálogo de Elementos Finitos*). Su ley cohesiva es un material de interfaz del programa principal, en `interface_materials`.
- `dissipation_control`: el solver de arco por disipación de energía `DissipationArcLengthSolver` (capítulo *Esquemas de Solución*).

Un modelo puede declarar varios: `user_modules: [discontinuities, dissipation_control]`.

3.6 Condiciones de Frontera

Solidum FEM ofrece tres mecanismos para imponer restricciones cinemáticas (Dirichlet), elegibles según la facilidad operativa:

3.6.1 Por nodo: `boundary_conditions_by_node`

```yaml
boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}
  - {node_id: 4, ux: 0.0, uy: 0.0}
```

3.6.2 Por coordenada: `boundary_conditions_by_coord`

Escanea todos los nodos y aplica la restricción a aquellos cuya coordenada coincide (dentro de `tol`):

```yaml
boundary_conditions_by_coord:
  x_min: {ux: 0.0, uy: 0.0, tol: 1.0e-5}      # empotrar lado izquierdo
  rodillos: {coord: 'y', val: 10.0, uy: 0.0, tol: 1.0e-4}
```

Las claves predefinidas `x_min`, `x_max`, `y_min`, `y_max`, `z_min`, `z_max` usan los extremos detectados automáticamente del dominio; `coord` admite `'x'`, `'y'` o `'z'`. Los selectores funcionan igual con nodos 2D y 3D. Un selector que no casa con **ningún** nodo es un error de modelo y detiene la lectura con `ValueError` (no se ignora en silencio).

3.6.3 Por grupo físico: `boundary_conditions_by_group`

Aplica restricciones a todos los nodos de un *Physical Group* de Gmsh:

```yaml
boundary_conditions_by_group:
  "Borde_Empotrado": {ux: 0.0, uy: 0.0}
```

3.6.4 Restricciones multipunto (MPC): `linear_constraints`

Para apoyos en plano oblicuo, periodicidad de celda unitaria, uniones rígidas master-slave y simetrías no alineadas con los ejes globales, Solidum FEM admite restricciones afines lineales de la forma $u_s = g_s + \sum_i \alpha_{si} u_{m_i}$ (ADR 0004 fase 2). Se declaran en el bloque `linear_constraints`:

```yaml
linear_constraints:
  # Apoyo a 45 grados en el nodo 4: u_y = u_x.
  - slave: {node: 4, dof: uy}
    masters:
      - {node: 4, dof: ux}
    coefficients: [1.0]

  # Periodicidad: u_x del nodo 7 igual al del nodo 3.
  - slave: {node: 7, dof: ux}
    masters:
      - {node: 3, dof: ux}
    coefficients: [1.0]
```

```yaml
# Union rigida: u_x del satelite N9 sigue al maestro N5 con offset r_y=0.5.
- slave: {node: 9, dof: ux}
  masters:
    - {node: 5, dof: ux}
    - {node: 5, dof: rz}
  coefficients: [1.0, -0.5]
  g: 0.0
```

Las restricciones se imponen por eliminación directa (no por penalización), de modo que la imposición es exacta a redondeo y preserva la simetría y la positividad definida del sistema. Las cadenas master-slave se resuelven automáticamente por cierre transitivo: un maestro puede ser a su vez esclavo de otra restricción, o tener un apoyo, y entonces el esclavo hereda su valor prescrito.

Cada entrada admite sólo las claves `slave`, `masters`, `coefficients` y `g`, y cada referencia sólo `node` y `dof`; una clave con errata (`G: 5` por `g: 5`) es un error, no se ignora. Los coeficientes y `g` deben ser números. Al leer el archivo, antes de ensamblar nada y también con malla gmsh, se comprueba además que:

- cada nodo existe y tiene el grado de libertad nombrado (`ux` no existe en un nodo térmico);
- el esclavo no tiene además un apoyo: un grado de libertad no puede ser esclavo de dos restricciones. Si el apoyo es coherente con la restricción (por ejemplo, los dos nodos coincidentes de una interfaz atada tomados por un mismo apoyo por coordenada o por grupo), basta con quitarlo del esclavo, porque la restricción le transmite el valor del maestro;
- el esclavo no figura entre sus propios maestros ni se declara dos veces con restricciones distintas (declarar dos veces la misma es inofensivo);
- las cadenas master-slave no forman ciclos ($a \leftarrow b$, $b \leftarrow a$).

Todos los problemas se informan juntos, cada uno con su entrada (`linear_constraints[i]`) y el par (nodo, dof) afectado. Con la API Python (`Domain.add_linear_constraint`) el ensamblador detecta los mismos errores al construir las restricciones, de uno en uno, y su mensaje también nombra el esclavo y los maestros por (nodo, dof).

En los solvers estáticos no lineales (`NonlinearSolver`, `ArcLengthSolver`, `IndirectDisplacementSolver`) el término independiente `g` se aplica proporcionalmente al factor de carga, como $\lambda\, g$, igual que un desplazamiento prescrito: en un trazado de arco sigue a λ también cuando la carga baja.

Reacciones y fuerzas de restricción. Una restricción multipunto ejerce una fuerza sobre la estructura en los grados de libertad de su esclavo y de sus maestros. Solidum distingue dos papeles:

- **Apoyo**: la restricción impide un movimiento de sólido rígido. Es el caso del apoyo a 45 grados del ejemplo ($u_y = u_x$ en un mismo nodo): su fuerza es una **reacción**, normal al plano de rodadura, y aparece en `R` y en `reactions_by_node` con las componentes `ux` y `uy` del nodo. Una relación con coeficiente distinto de uno entre nodos distintos ($u_s = 2\,u_m$) también cuenta como apoyo.
- **Unión**: una traslación rígida de toda la estructura (o una temperatura uniforme, en el problema térmico) la cumple. Son uniones el atado de mallas, la periodicidad (también con `g` distinto de cero) y la unión rígida del ejemplo. Su fuerza es una **fuerza de restricción**, que transmite fuerza entre partes de la estructura y no la toma del exterior: aparece en `constraint_forces`

y `constraint_forces_by_node` del resultado, con valores iguales y opuestos en los nodos atados, y no en las reacciones.

Un apoyo con valor prescrito reporta su reacción completa, incluida la fuerza que le llega por los nodos atados a él: atar dos nodos coincidentes con $u_s = u_m$ en todos sus grados de libertad da las mismas reacciones que fundirlos en uno. Con esta convención se cumple el equilibrio global $\sum R + \sum F = 0$ por dirección.

3.6.5 Refuerzo embebido: `constraints`

El bloque `constraints` declara **generadores de restricciones** (ADR 0032): objetos que, sobre los elementos ya creados, generan a partir de su geometría los nodos, los elementos y las restricciones multipunto que necesitan. Se aplica después de los elementos y antes de los apoyos. Hoy hay un tipo, `EmbeddedReinforcement`: una barra de refuerzo dentro de los sólidos 2D o 3D, con adherencia perfecta o, con la opción `bond`, con adherencia-deslizamiento (spec `EmbeddedReinforcement`).

```yaml
materials:
  - {id: 1, type: Elastic2D, E: 30.0e9, nu: 0.2, hypothesis: plane_stress}
  - {id: 2, type: Elastoplastic1D, E: 200.0e9, sigma_y: 420.0e6}
constraints:
  - type: EmbeddedReinforcement
    points: [[0.0, 0.05], [4.0, 0.05]]   # polilínea de la barra
    area: 1.6e-3                         # área de la barra
    material: 2                          # material 1D
    element: Truss2D                     # opcional (Truss3D por omisión en 3D)
    hosts: [1, 2, 3]                     # opcional: ids de los anfitriones
    subdivisions: 1                      # opcional: partes por tramo
```

La barra tiene su propia malla. Solidum la corta en los bordes de los elementos que atraviesa y ata cada nodo al desplazamiento interpolado del elemento que lo contiene, $u(\mathbf{X}_b) = \sum_i N_i(\boldsymbol{\xi}_b)\, u_i$. El área de concreto que ocupa la barra no se descuenta: el sobrante de rigidez elástica es $\rho/[(1-\rho)+n\rho]$, con $n = E_s/E_c$ ($+1.8\,\%$ con una cuantía del $2\,\%$), y es cero con el concreto agrietado en tracción.

Adherencia-deslizamiento (`bond`, spec `EmbeddedReinforcement` §12): en lugar de atar los nodos, cada tramo de la barra se une a su anfitrión con una interfaz distribuida gobernada por una ley τ–s de la sección `interface_materials` (`BondSlipModelCode`, `BondSlipMultilinear`). La tracción de adherencia actúa sobre el perímetro $\pi \cdot$d_b y la normal sobre el ancho d_b. Se recomienda la barra de tres nodos con la integración nodal (por omisión):

```yaml
interface_materials:
  - id: 1
    type: BondSlipModelCode
    model_code_2010:                     # parámetros de la Tabla 6.1-1 del Model Code
        2010
      fcm: 30.0e6                        # Pa
      bond: good                         # good | other
      failure: pullout                   # pullout | splitting (con stirrups: true|
          false)
      clear_rib_spacing: 0.0105          # c, m
      diameter: 0.016                    # d_b, m (para k_n = 2·E_c/d_b)
      E_c: 30.0e9                        # Pa
      # k0: 1.8e11                       # Pa/m (180 N/mm^3 por omisión)
constraints:
```

```yaml
  - type: EmbeddedReinforcement
    points: [[0.0, 0.05], [4.0, 0.05]]
    area: 2.01e-4
    material: 2
    element: Truss2D3                    # barra cuadrática (recomendada)
    bond:
      law: 1                             # id en interface_materials
      diameter: 0.016                    # d_b
      # perimeter: ...                   # por omisión π·d_b (paquetes de barras: el
          suyo)
      # integration: nodal               # nodal (por omisión) | gauss
```

La ley también puede darse con sus parámetros explícitos (`tau_max`, `s1`, `s2`, `s3`, `alpha`, `tau_f`, `k0`, `k_n`) o por puntos (`BondSlipMultilinear`: `points: [[s, τ], ...]`, `k_n`, `tau_f` opcional). Sin `bond`, la adherencia es perfecta.

Los nodos de la barra no los toman los selectores por coordenada (`boundary_conditions_by_coord`, `point_loads_by_coord`, `nodal_masses_by_coord`): un apoyo o una carga por coordenada es del sólido. Las fuerzas de esas restricciones salen en `constraint_forces`, no en las reacciones: son las **fuerzas de adherencia** concentradas en los nodos de la barra. Lo que creó cada entrada queda en `model.parser.applied_constraints`, que con `load_yaml` es una lista de `EmbeddedBar`.

3.7 Cargas Puntuales

Misma estructura que las condiciones de frontera, en bloques `point_loads_by_node`, `point_loads_by_coord` y `point_loads_by_group`. Los valores son fuerzas nodales (en las unidades del modelo, típicamente Newtons):

```yaml
point_loads_by_node:
  - {node_id: 3, uy: -15000.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}
```

Las cargas son **estrictas**: un `node_id` inexistente se rechaza en la validación del archivo, y un nombre de DOF que el nodo no tiene (`fy` en vez de `uy`, `rz` sobre una armadura) o un grupo físico inexistente detienen la lectura con `ValueError`. Antes se ignoraban en silencio y el análisis corría sin carga.

En los **solvers transitorios mecánicos** (`NewmarkSolver`, `HHTSolver`, `NewtonNewmarkSolver`, `NewtonHHTSolver`, `CentralDifferenceSolver`) las cargas del archivo —puntuales más peso propio o fuerza de cuerpo— se aplican como **escalón constante en el tiempo** desde $t = 0$ (`F_func = cte`). Una historia $\mathbf{F}(t)$ arbitraria requiere construir el solver desde Python con su propio `F_func` (§10).

3.8 Masas Concentradas en Nudos: `nodal_masses`

La masa que no sale de la densidad de los elementos —en un edificio, la de la losa, los acabados y la fracción de la carga viva de cada nivel— se concentra en los nudos (ADR 0024):

```yaml
nodal_masses:
  - {node_id: 5, ux: 12.0e3, uy: 12.0e3}            # masa [kg]
```

```yaml
  - {node_id: 9, ux: 12.0e3, uy: 12.0e3, rz: 4.0e3} # rz: inercia rotacional [kg·m²]
```

- Cada clave, salvo `node_id`, es un grado de libertad mecánico del nodo: `ux`, `uy`, `uz` llevan una masa; `rx`, `ry`, `rz`, una inercia rotacional (masa·longitud^2). Las rotacionales son opcionales.
- Varias entradas sobre el mismo grado de libertad se suman.
- La masa entra en la diagonal de la matriz de masa, igual con `lumping: consistent` que con `lumped`, así que la usan el análisis modal, el espectral y los dinámicos.
- Es **sólo inercia**: `gravity` no genera su peso. Si su peso actúa, declárelo como carga en `point_loads`.
- Los elementos siguen necesitando `density` en sus materiales (ADR 0008). Un elemento sin masa propia se declara con `density: 0.0`.
- El bloque se valida al leer: valores finitos no negativos, grados de libertad mecánicos que el nodo tenga y nodos existentes. Todos los errores se informan juntos.

Con una malla gmsh, los nodos se seleccionan también por grupo físico o por coordenada, como las cargas puntuales:

```yaml
nodal_masses_by_group:
  - {group_name: Piso_1, ux: 1.5e3, uy: 1.5e3}   # o la forma dict:  Piso_1: {ux: 1.5e3
      , uy: 1.5e3}
nodal_masses_by_coord:
  - {loc: y_max, ux: 800.0}                      # o {coord: y, val: 3.0, ux: 800.0};
      forma dict: y_max: {ux: 800.0}
```

- **Cada** nodo seleccionado recibe la masa indicada, no una parte de ella. Para repartir la masa de un nivel entre sus nodos, divídala entre su número.
- Un grupo inexistente, una coordenada que no selecciona ningún nodo o un grado de libertad que los nodos no tienen se informan junto con los demás errores.

Con masas sólo en algunos nudos, casi todos los grados de libertad quedan sin masa. El análisis modal los condensa de forma exacta (capítulo de dinámica).

3.9 Fuerzas de Cuerpo

Una *fuerza de cuerpo* es una fuerza por unidad de volumen $\mathbf{b}$ que actúa sobre todo el dominio material del elemento, no solo sobre su frontera. Cada elemento del catálogo (armaduras, cables, marcos 2D/3D, sólidos 2D) integra la contribución consistente $\int \mathbf{N}^\top \mathbf{b}\, A\, dx$ y la acumula al vector global de fuerzas. Para vigas Hermite cúbicas el reparto incluye fuerza nodal $qL/2$ y momento $\pm qL^2/12$; para barras axiales es mitad-mitad por nodo.

Casos típicos en mecánica estructural: peso propio (un caso particular con $\mathbf{b} = \rho\,\mathbf{g}$), fuerzas centrífugas en cuerpos rotatorios ($\mathbf{b} = \rho\,\omega^2\,\mathbf{r}$), fuerzas electromagnéticas sobre materiales conductores, expansión térmica modelada como pseudocarga. Solidum expone dos formas declarativas en YAML que cubren todos estos casos.

3.9.1 Forma general: `body_force`

Aplica un vector **b** uniforme (fuerza por unidad de volumen, en ejes globales) a todos los elementos del modelo:

```
body_force: [0.0, -77008.5]          # 2D: bx, by en N/m^3
```

o, para problemas 3D:

```
body_force: [0.0, 0.0, -77008.5]     # 3D: bx, by, bz
```

Es la forma genérica y la única necesaria para fuerzas de cuerpo que no son peso propio (campos electromagnéticos precalculados, centrífuga estática, etc.). También sirve para peso propio cuando todo el modelo es monomaterial y el usuario prefiere precalcular $\rho \cdot g$.

Cada componente puede ser también una expresión de la posición (ver «Valores Variables: Expresiones»), por ejemplo una fuerza centrífuga $\mathbf{b} = \rho\,\omega^2\,\mathbf{r}$:

```
body_force: ["7850*100*x", "7850*100*y"]   # ρ·ω²·r con ω = 10 rad/s
```

La admiten los sólidos, las armaduras y los marcos; los cables, no (reparten la carga con su longitud sin tensión).

3.9.2 Caso particular — peso propio: `gravity` + `density`

El peso propio es el caso especial $\mathbf{b} = \rho\,\mathbf{g}$. Para problemas multimaterial — donde cada elemento debe ver *su propio* ρ — Solidum expone un atajo declarativo: la densidad como propiedad del material, y un vector global de gravedad.

```
materials:
  - {id: 1, type: Elastic1D, E: 210.0e9, density: 7850.0}     # acero
  - {id: 2, type: Elastic1D, E:  30.0e9, density: 2500.0}     # hormigón

gravity: [0.0, -9.81]                # m/s^2, +y hacia arriba
```

Qué hace Solidum. Para cada elemento computa $\mathbf{b}_e = \rho_e \cdot \mathbf{g}$, donde ρ_e es la densidad del material asignado, e integra la contribución consistente con la misma maquinaria de `body_force`. Resultado: estructuras multimaterial dan peso propio físicamente correcto, cada elemento con su propio ρ.

Densidad solo cuando se usa. El campo `density` es **opcional** al declarar un material (ADR 0008): los análisis estáticos puros que no invocan peso propio ni matriz de masa no necesitan declararla. Pero al invocar `gravity` (o, en su día, análisis modal/dinámico), Solidum exige la densidad de cada material involucrado: si algún material no la trae declarada, el cálculo falla con `ValueError` listando los materiales afectados por nombre. Imposible propagar masa cero silenciosa. Si un material legítimamente carece de masa física (penalty, restricción), declararlo explícitamente como `density: 0.0` — caso en el cual `gravity` emite un `WARNING` informativo y su contribución al peso propio es nula.

3.9.3 Detalles transversales

Exclusividad. `body_force` y `gravity` son dos formas declarativas para expresar fuerzas de cuerpo; declarar ambas en el mismo archivo lanza error de validación. La forma general (`body_force`) es la subyacente; `gravity` es atajo para su caso particular más común.

Padding 2D↔3D. Si el modelo mezcla elementos 2D y 3D, declarar el vector con tres componentes; el ensamblador pasa el slice apropiado a cada elemento según la dimensionalidad declarada por su `DOF_NAMES`.

Reservado para análisis dinámico. El atributo `density` introducido aquí es la misma magnitud que consume la matriz de masa $\mathbf{M}_e = \int \rho \mathbf{N}^\top \mathbf{N}\, dV$ en análisis modales y dinámicos. Declararlo prepara el modelo para esas extensiones sin coste adicional (ADR 0008).

3.10 Cargas Distribuidas de Contorno: `tractions` y `tractions_by_group`

Una *tracción* $\mathbf{t}$ es una fuerza por unidad de superficie sobre la frontera del sólido (en 2D, por unidad de longitud de borde y de espesor). Se integra con la geometría real del borde o de la cara, $\int_\Gamma \mathbf{N}^\top \mathbf{t}\, d\Gamma$, también sobre bordes curvos de elementos cuadráticos. La admiten los sólidos 2D y 3D.

Por grupo físico (mallas gmsh): las celdas de contorno del grupo (las líneas de gmsh en 2D) son los bordes exactos que se cargan.

```yaml
tractions_by_group:
  Borde_Carga: {t: [1.0e6, 0.0]}        # tracción en ejes globales [Pa]
  Agujero:     {pressure: 2.0e6}         # presión normal [Pa]
```

Por elemento (cualquier malla, también 3D): el borde (`edge`, 2D) o la cara (`face`, 3D) con la numeración local del elemento.

```yaml
tractions:
  - {element: 12, edge: 1, t: [1.0e6, 0.0]}
  - {element: 40, face: 3, pressure: 2.5e3}
```

Cada entrada lleva **una** de dos formas:

- `t`: el vector de tracción en ejes globales, con 2 ó 3 componentes según la dimensión del elemento.
- `pressure`: la presión p, normal al contorno. **Convención de signo** (Reglas §5): $p > 0$ es **compresión**, la presión empuja hacia el interior del cuerpo, $\mathbf{t} = -p\,\mathbf{n}$ con $\mathbf{n}$ la normal exterior. Una succión se declara con $p < 0$. Es el convenio de Abaqus y ANSYS.

La carga es **muerta**: actúa sobre la geometría de referencia y no sigue la deformación. Un grupo que no sea de contorno (por ejemplo, una superficie de la malla) o un borde compartido por dos elementos se rechazan al leer el archivo.

3.11 Valores Variables: Expresiones

Cualquier valor de `body_force`, `tractions`, `tractions_by_group` y de las cargas térmicas (`body_source`, `boundary_flux`, `boundary_flux_by_group`) puede ser, en vez de un número, una expresión entre comillas de la posición `x`, `y`, `z`; en las cargas de contorno, también de la normal exterior `nx`, `ny`, `nz`:

```yaml
body_force: [0.0, "-78.5e3*(1 + 0.1*y)"]          # campo b(x)
tractions_by_group:
  Agujero: {pressure: "1.0e6*(1 - y/0.2)"}         # presión variable
  Borde:   {t: ["1.0e6*nx", 0.0]}                  # con la normal
```

Se admiten números, `+ - * / **`, paréntesis, la constante `pi` y las funciones `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, `exp`, `log`, `log10`, `sqrt`, `abs`, `min` y `max`. Nada más: la expresión no se ejecuta como código Python, así que un archivo ajeno no puede hacer otra cosa que calcular una carga, y una errata (`w` en lugar de `y`, una función que no existe) se detecta al leer el archivo, con la entrada y lo que no se admite. Una expresión que no se puede evaluar en algún punto (`log` de un negativo) se informa con el punto. Las cargas que varían en el tiempo no se declaran en el YAML (ADR 0023).

3.12 Solver

```yaml
solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5
```

Los solvers disponibles y sus parámetros se documentan en los capítulos *Esquemas de Solución* y *Análisis Dinámico*.

El solver del sistema lineal que cada uno resuelve internamente **lo elige Solidum** según las propiedades de la matriz; no hace falta declararlo. El campo opcional `linear_algebra` sirve para diagnóstico o para pedir el solver iterativo en modelos muy grandes:

```yaml
solver:
  type: LinearSolver
  linear_algebra: iterative    # auto (default), lu, pardiso, cholesky, iterative[:amg|
      jacobi|none]
```

Un valor desconocido, o `iterative:amg` sin `pyamg` instalado, se rechaza al leer el archivo.

3.12.1 Bloque `convergence`

Los solvers no lineales (`NonlinearSolver`, `ArcLengthSolver`) aceptan un bloque `convergence` que configura el criterio de parada de las iteraciones de Newton. La política sigue el patrón estándar de tolerancias mixtas absoluta + relativa, separado por dos criterios físicos: *fuerza* (residuo de equilibrio) y *desplazamiento* (incremento del iterado). La forma exacta de la comparación es:

```
||R_red|| <= atol_force + rtol_force * max(||F_ext||, ||F_int||)
||dU||    <= atol_disp  + rtol_disp  * ||U||
```

`R_red` es el residuo de equilibrio en los grados de libertad libres: las reacciones de los apoyos no cuentan y, con restricciones multipunto, el residuo de cada grado de libertad esclavo se suma a sus maestros multiplicado por su coeficiente ($\mathbf{T}^\top\mathbf{R}$, ADR 0004).

Ambos criterios deben cumplirse **simultáneamente** (semántica AND) para dar el paso por convergido.

Parámetros disponibles (todos opcionales, con defaults razonables):

- `rtol_force` (default `1.0e-5`): tolerancia relativa del residuo de fuerza. Es el parámetro que normalmente ajustarás: bajarlo a `1.0e-7` para análisis de precisión, subirlo a `1.0e-3` para exploración rápida.
- `rtol_disp` (default `1.0e-5`): tolerancia relativa de la corrección de desplazamiento. En problemas casi-rígidos puede afinarse independientemente del de fuerza.
- `atol_force_factor` (default `1.0e-9`): factor adimensional que define el piso absoluto de fuerza. La tolerancia absoluta efectiva se autoderiva como `atol_force_factor` · `escala_inicial`, donde la escala se calcula en el primer ensamblaje. Solo conviene ajustarlo si el análisis tiene tramos donde la carga característica colapsa transitoriamente y aparece falsa no-convergencia.
- `atol_disp_factor` (default `1.0e-9`): análogo para desplazamiento.

Por qué los `atol` se autoderivan. El término absoluto vive en las unidades del problema (newtons para fuerza, metros para desplazamiento). Codificarlo como constante global obligaría a ajustarlo al cambiar de unidades (N/m, kN/mm, MPa/mm…). En su lugar, Solidum calcula la escala característica del problema en el primer ensamblaje y multiplica por el factor adimensional. El resultado: **el mismo análisis planteado en distintos sistemas de unidades converge en el mismo número de iteraciones hasta paridad de bits**, sin tocar las tolerancias. La justificación arquitectural está en el ADR 0007.

Cuándo afinar. Para la inmensa mayoría de análisis los defaults son suficientes. Considera ajustar:

- `rtol_force` y `rtol_disp` más estrictos cuando publiques resultados o compares contra solución analítica (`1.0e-7` a `1.0e-9`).
- `rtol_disp` más laxo que `rtol_force` en problemas casi-rígidos donde el incremento de desplazamiento es naturalmente pequeño y baja muy rápido, dejando que el criterio de fuerza domine.
- Los factores `atol_*` prácticamente nunca; solo si encuentras un análisis específico donde la escala colapsa y los logs muestran iteraciones oscilando cerca del umbral.

Nota sobre el solver algebraico interno. El sistema lineal $\mathbf{K}\,\delta\mathbf{U} = \mathbf{R}$ que aparece dentro de cada iteración se resuelve con un backend algebraico (Cholesky, LU…) seleccionado *automáticamente* según las propiedades de $\mathbf{K}$. Esta elección **no** es decisión de modelado y no requiere configuración. Si por motivos de diagnóstico necesitas forzar un backend específico, consulta el capítulo *Anexos técnicos — Capa algebraica* del Manual de Referencia.

3.13 Etapas: `stages`

Un análisis por etapas encadena análisis que continúan el anterior: la gravedad y después un empuje, una secuencia de cargas, un estático y después una carga dinámica. Cada etapa declara sus cargas (`loads`, con los mismos bloques del nivel raíz: `point_loads…`, `gravity`, `body_force`, `tractions…`, `thermal_loads`) y su `solver`:

```yaml
stages:
  - name: gravedad
    loads:
      gravity: [0.0, -9.81]
    solver: {type: NonlinearSolver, num_steps: 4}
  - name: empuje
    loads:
      point_loads:
        - {node_id: 4, ux: 1.0e5}
    solver:
      type: IndirectDisplacementSolver
      control:
        - {node: 4, dof: ux, coef: 1.0}
      control_target: 0.06
      initial_dl: 0.004
      max_lambda: 1.0e9
```

- **Etapa estática**: parte de los desplazamientos y del estado interno de la anterior; la carga ya aplicada se mantiene constante y la de la etapa crece con λ (ADR 0026).
- **Etapa en el tiempo** (`NewmarkSolver`, `NewtonNewmarkSolver`, HHT, diferencia central, `ThetaMethodSolver`): parte del estado final de la anterior (ADR 0029); sus cargas se aplican como escalón desde su instante inicial, sobre la carga constante de las anteriores. Tras otra etapa en el tiempo, `t_end` es el instante final absoluto.
- Con `stages`, el `solver` y las cargas van en cada etapa: declararlos también en el nivel raíz es un error.
- `run_yaml` devuelve un `StagedResult`: `res["empuje"]` o `res[1]` da el resultado de una etapa, `res.final` el de la última y `res.names` los nombres.
- **Etapa modal, armónica o espectral**: linealiza alrededor del estado de la anterior con su rigidez tangente (P-Δ, agrietamiento; ADR 0031) y no lo cambia: la etapa siguiente continúa la última estática o en el tiempo.
- **Etapa con `LinearSolver`**: perturbación lineal sobre la anterior (casos de carga superponibles sobre el estado P-Δ, ADR 0031); tampoco cambia el estado.
- Pendiente de una decisión física, y por eso rechazada con un error claro: una etapa estática después de una en el tiempo.

3.14 Salida: `output`

Configura qué se exporta y cuándo:

```yaml
output:
  file_name: "resultados_placa"
  log_level: WARNING              # opcional: DEBUG, INFO (por omisión), WARNING o
      ERROR
  pre_process: {export: true}     # exporta el modelo sin deformar (paso 0)
  results:
```

```yaml
    frequency: "all_steps"        # "last_step" o "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State", "Sigma_XX"]
  text_export:                     # opcional: salida estilo FEAP en .txt
    export: true
    nodes: all
    elements: all
```

Nota

Cuando `frequency: "all_steps"` y el solver es no lineal, se genera un archivo `.vtu` por paso convergido más un archivo maestro `.pvd` que ParaView interpreta como animación.

`log_level` fija el nivel del registro durante el análisis (`run_yaml`, `load_yaml(...).solve()` y `examples/ejecutar_yaml.py`): `WARNING` deja sólo avisos y errores. Al terminar, `run_yaml` vuelve al nivel que tenía la aplicación. Un nivel desconocido es un error de validación.

Capítulo 4

Catálogo de Elementos Finitos

El programa principal expone varias familias de elementos: **1D** (estructuras reticulares: armaduras, cables, marcos), **2D** (continuo plano: cuadrilátero y triángulo), **3D** (continuo tridimensional: hexaedros y tetraedros, lineales y cuadráticos) y **térmicos** (conducción de calor, con un grado de libertad escalar por nodo; se documentan en el capítulo «Análisis Térmico»). Todos se referencian desde el YAML por su nombre exacto en `type:`.

Las formulaciones no estándar viven fuera del programa principal, en **módulos de usuario** que el modelo carga de forma explícita (ADR 0020). El elemento con discontinuidad embebida (fractura con salto interior) pertenece al módulo de usuario `discontinuities` y se documenta en la última sección de este capítulo.

4.1 Elementos 1D

4.1.1 Armaduras: `Truss2D` / `Truss2DCorot` / `Truss3D` / `Truss3DCorot` / `Truss2D3` / `Truss3D3`

Barra biarticulada que transmite exclusivamente fuerza axial. Seis variantes según dimensión, régimen geométrico y orden de interpolación:

Parámetros: `A` (área de la sección transversal). Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).

Las de **tres nodos** (`Truss2D3`, `Truss3D3`) son rectas, con los nodos en el orden `[i, j, m]` y el intermedio `m` en el punto medio; interpolan el desplazamiento axial con una cuadrática (dos puntos de Gauss) y dan `N_i`, `N_j` extrapolando el axial lineal del elemento.

Régimen de validez: $|\varepsilon| \lesssim 10^{-2}$. Las variantes `Corot` aceptan rotaciones de cualquier magnitud entre commits; las versiones lineales asumen rotaciones pequeñas.

```yaml
elements:
  - id: 1
    type: Truss2DCorot
    material: 1
    nodes: [1, 2]
    A: 0.0005
```

Tabla 4.1: Familia de armaduras.

Elemento	DOFs/nodo	Dimensión	Régimen geom.	Uso
`Truss2D`	`ux,uy`	2D	lineal	cargas pequeñas, rotaciones pequeñas
`Truss2DCorot`	`ux,uy`	2D	corotacional	grandes desplazamientos y rotaciones planas
`Truss3D`	`ux,uy,uz`	3D	lineal	armaduras espaciales en régimen lineal
`Truss3DCorot`	`ux,uy,uz`	3D	corotacional	grandes rotaciones en el espacio
`Truss2D3`	`ux,uy`	2D	lineal, 3 nodos	refuerzo con adherencia-deslizamiento; axial cuadrático
`Truss3D3`	`ux,uy,uz`	3D	lineal, 3 nodos	ídem en el espacio

4.1.2 Cables: `Cable2DCorot` / `Cable3DCorot`

Elemento 1D que transmite *únicamente* fuerza axial de tracción (no resiste compresión). La unilateralidad la aporta el material (típicamente `CableMaterial1D`, ver capítulo *Catálogo de Modelos Constitutivos*); el elemento implementa la cinemática corotacional y la transferencia de la deformación al material.

- **DOFs/nodo**: `ux, uy` (2D); `ux, uy, uz` (3D).
- **Parámetros**: `A`.
- **Cinemática**: corotacional (longitud y cosenos directores se recalculan en cada evaluación).
- **Régimen de validez**: $|\varepsilon| \lesssim 10^{-2}$ tensado. En estado destensado el elemento aporta rigidez nula al sistema global; otros elementos deben garantizar la estabilidad numérica.

Advertencia

Un cable completamente destensado ($\sigma = 0$) tiene $\mathbf{K}_T = \mathbf{0}$ en sus DOFs. Si todos los caminos de carga del modelo dependen del cable y este se destensa, la matriz tangente global se vuelve singular. Pretensar el cable o garantizar redundancia estructural.

```yaml
materials:
  - id: 1
    type: CableMaterial1D
    E: 150.0e9
    density: 7850.0

elements:
  - id: 1
    type: Cable2DCorot
    material: 1
    nodes: [1, 2]
    A: 5.0e-5
```

4.1.3 Marcos 2D: `Frame2DEuler` / `Frame2DTimoshenko` / `Frame2DEulerCorot`

Elementos viga 2D que transmiten axial, cortante y momento flector. Tres variantes:

- `Frame2DEuler`: vigas esbeltas ($L/h \gtrsim 10$), Euler-Bernoulli, régimen geométricamente lineal. Parámetros: `A`, `I` (con `material`) o `section`, `n_ip`.
- `Frame2DTimoshenko`: vigas peraltadas o cortas ($L/h \lesssim 10$). Incluye deformación por cortante; corrige automáticamente *shear locking* en el límite esbelto. Parámetros: `A`, `I`, `As` (área efectiva de cortante), `nu` (opcional si el material expone `nu`). **Sólo material elástico lineal**; todavía no admite `section`.
- `Frame2DEulerCorot`: variante corotacional de Euler-Bernoulli. Captura grandes desplazamientos y grandes rotaciones rígidas del elemento (rotaciones deformacionales nodales moderadas, $|\bar{\theta}| \lesssim 30°$). Parámetros: `A`, `I` (con `material`) o `section`, `n_ip`; con sección, ésta trabaja en el sistema corrotado del elemento.

DOFs/nodo: `ux, uy, rz`. Convención: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación) en el eje axial; r_z positivo antihorario.

Dos formas de dar la constitutiva (ADR 0021), excluyentes entre sí:

- `material` + **propiedades geométricas** (`A`, `I`): rigidez de forma cerrada. Sólo admite un material **elástico lineal** (`Elastic1D`); un material con plasticidad o daño se rechaza al construir el modelo, con un mensaje que remite a la sección por fibras.
- `section` (id de una sección del bloque `sections`, capítulo *Archivo de Entrada YAML*) + `n_ip` (puntos de Gauss a lo largo del elemento, 3 por defecto, mínimo 2): el elemento integra la sección por fibras en cada punto de Gauss. Cada fibra lleva su material 1D y su historia, de modo que se forman **rótulas plásticas por flexión** y el daño avanza fibra a fibra. Con fibras elásticas y `n_ip` $\geq$ `2` se obtiene exactamente la misma rigidez que con `A` e `I`. Con `section` no se declaran `material`, `A` ni `I`: rigidez, masa y peso propio salen de las fibras.

Advertencia

Cambio desde el 2026-09-29. Hasta esa fecha los marcos aceptaban un material inelástico con `A` e `I`, le pasaban sólo la deformación axial y escalaban toda la matriz local con su módulo tangente: con fluencia axial el momento flector se anulaba y con ablandamiento cambiaba de signo, sin aviso. Un modelo así ahora falla al construirse; su equivalente físico es el mismo marco con `section`. Con la sección por fibras, la carga de colapso converge por arriba al refinar la malla: use varios elementos por miembro cerca de las rótulas.

```yaml
elements:
  - id: 1
    type: Frame2DEulerCorot
    material: 1
    nodes: [1, 2]
    A: 0.01
    I: 8.33e-6

  - id: 2
```

```yaml
    type: Frame2DTimoshenko
    material: 1
    nodes: [2, 3]
    A: 0.01
    I: 8.33e-6
    As: 0.00833

  - id: 3                    # con sección por fibras (bloque sections)
    type: Frame2DEuler
    nodes: [3, 4]
    section: 1
    n_ip: 3
```

4.1.4 Marco 3D: `Frame3D`

Viga 3D Euler-Bernoulli con 6 DOFs/nodo (`ux, uy, uz, rx, ry, rz`). Transmite axial, cortantes en dos planos, flexiones en dos planos y torsión de Saint-Venant pura. Régimen geométricamente lineal (la variante corotacional 3D queda como pendiente).

Parámetros, con las mismas dos formas excluyentes de dar la constitutiva que los marcos 2D:

- Con `material` (sólo elástico lineal): `A` — área de la sección; `Iy`, `Iz` — momentos de inercia respecto a los ejes locales y, z; `J` — constante torsional de Saint-Venant; `nu` — coeficiente de Poisson (opcional; se toma del material si lo expone).
- Con `section` (sección por fibras con `GJ` declarado) y `n_ip` (default 3, mínimo 2): flexión en los dos planos y fuerza axial salen de las fibras, con su interacción; la torsión es elástica, $T = GJ\,\theta'_x$, con el `GJ` de la sección. Sin `nu`.
- `ref_vector` — vector de referencia (opcional, default `[0,0,1]`) que fija la orientación de los ejes locales y, z de la sección, que son también los ejes en que se declaran las fibras. Para barras casi-verticales se sugiere fijarlo explícitamente.

Convención de fuerzas internas (Reglas.md §5): $M_y > 0$ tracciona las fibras con $z > 0$; $M_z > 0$, las fibras con $y < 0$.

```yaml
elements:
  - id: 1
    type: Frame3D
    material: 1
    nodes: [1, 2]
    A: 0.01
    Iy: 8.33e-6
    Iz: 8.33e-6
    J: 1.66e-5
    ref_vector: [0.0, 0.0, 1.0]

  - {id: 2, type: Frame3D, nodes: [2, 3], section: 1, n_ip: 3, ref_vector: [0.0, 0.0,
      1.0]}
```

4.2 Elementos 2D

La importación de mallas desde Gmsh instancia automáticamente los elementos 2D según la topología detectada (cuadriláteros → `Quad4`; triángulos → `Tri3`). También pueden declararse manualmente desde el bloque `elements`.

4.2.1 Cuadrilátero Bilineal: `Quad4`

Elemento isoparamétrico de 4 nodos, integración Gauss configurable.

- **DOFs/nodo**: `ux, uy` (`STRAIN_DIM` = 3, Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$).
- **Nodos**: 4, ordenados en sentido antihorario.
- **Cuadratura**: `"2x2"` por defecto (4 puntos). `"1x1"` disponible pero produce modos espurios (*hourglassing*).
- **Parámetros**: `thickness`, `quadrature` (opcional).
- **Implementación**: kernels críticos compilados con `@njit` (Numba).
- **Limitación**: bloqueo volumétrico con materiales casi-incompresibles ($\nu \to 0.5$); en ese régimen requeriría formulación mixta (no implementada).

4.2.2 Triángulo de Deformación Constante: `Tri3`

Triángulo CST de 3 nodos, un único punto de integración.

- **DOFs/nodo**: `ux, uy` (`STRAIN_DIM` = 3).
- **Cuadratura**: 1 punto (deformación uniforme).
- **Parámetros**: `thickness`.
- **Limitación**: *shear locking* severo, convergencia lenta. **Preferir** `Quad4` salvo en transiciones donde Quad4 no encaja geométricamente.

4.2.3 Cuadrilátero Serendípito de Orden 2: `Quad8`

Cuadrilátero isoparamétrico cuadrático de 8 nodos (4 vértices antihorarios + 4 nodos medios de borde). Funciones de forma serendípitas (sin el término $\xi^2\eta^2$). Reproduce campos cuadráticos exactamente y mitiga el bloqueo por cortante en problemas de flexión.

- **DOFs/nodo**: `ux, uy` (`STRAIN_DIM` = 3).
- **Cuadratura**: Gauss 3×3 (9 puntos) por defecto; 2×2 disponible vía `quadrature` pero con riesgo de modos espurios.
- **Parámetros**: `thickness`, `quadrature` (opcional).
- **Tracción de borde**: reparte 1/6, 4/6, 1/6 entre los nodos del borde (vértice, medio, vértice).

4.2.4 Cuadrilátero Lagrangiano de Orden 2: `Quad9`

Cuadrilátero Lagrangiano cuadrático de 9 nodos (los 8 de `Quad8` más un noveno nodo central interior). Las funciones de forma son producto tensorial Lagrange 1D-1D, espacio polinómico completo Q_2.

- **DOFs/nodo**: `ux, uy` (`STRAIN_DIM` = 3).
- **Cuadratura**: Gauss 3×3.
- **Parámetros**: `thickness`, `quadrature` (opcional).
- **Diferencia con `Quad8`**: el nodo central interior añade el término $\xi^2\eta^2$ y mejora el comportamiento en problemas con campos no separables.

4.2.5 Triángulo Cuadrático Completo P_2: `Tri6`

Triángulo isoparamétrico cuadrático de 6 nodos (3 vértices + 3 medios de borde). Resuelve el *shear locking* severo de `Tri3` y reproduce campos cuadráticos exactamente.

- **DOFs/nodo**: `ux, uy` (`STRAIN_DIM` = 3).
- **Cuadratura**: 3 puntos en los puntos medios (regla `tri_3`).
- **Parámetros**: `thickness`.
- **Tracción de borde**: reparte 1/6, 4/6, 1/6 entre los nodos del borde (vértice, medio, vértice).
- **Cuándo usarlo**: transiciones de malla cuadráticas, geometrías curvadas donde Quad8/Quad9 no encajan.

```
elements:
  - {id: 1, type: Quad8, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6, 7, 8]}
  - {id: 2, type: Quad9, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6, 7, 8,
      9]}
  - {id: 3, type: Tri6,  material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6]}
```

4.3 Elementos 3D

Sólidos tridimensionales isoparamétricos (ADR 0012). Todos comparten DOFs `ux, uy, uz` y `STRAIN_DIM` = 6 sobre la convención Voigt 3D del proyecto, $[\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]$, y exigen un material 3D (`Elastic3D`, `VonMises3D`, `DruckerPrager3D`, `IsotropicDamage3D`).

A diferencia de los 2D, **no llevan `thickness`**: el volumen sale de la geometría. Las cargas de superficie se aplican por **cara numerada con normal saliente** mediante `compute_face_traction(face, t)`, con `t` expresada en ejes globales.

4.3.1 Hexaedro Trilineal: `Hex8`

Hexaedro isoparamétrico de 8 nodos, espejo natural del `Quad4`. Orden de nodos VTK_HEXAHEDRON.

- **Nodos**: 8 · **Cuadratura**: Gauss 2×2×2 (8 puntos) por defecto.
- **Parámetros**: `quadrature` (opcional; `hex_3x3x3` para no lineales severos, `hex_1x1x1` reducida con riesgo de *hourglass*).
- **Caras**: 6, numeradas 0 ($-\zeta$), 1 ($+\zeta$), 2 ($-\eta$), 3 ($+\xi$), 4 ($+\eta$), 5 ($-\xi$).
- **Limitaciones**: bloqueo volumétrico con $\nu \to 0.5$ y *shear locking* con una sola capa de elementos en la sección. Sin mitigación, por política idéntica a la del `Quad4`.

4.3.2 Tetraedro Lineal: `Tet4`

Tetraedro de 4 nodos, CST 3D — espejo del `Tri3`. Deformación uniforme en el elemento.

- **Nodos**: 4 · **Cuadratura**: 1 punto baricéntrico.
- **Caras**: 4 triangulares; la cara i es la opuesta al nodo i.
- **Limitación**: *shear locking* severo, peor que el `Hex8` por la pobreza del espacio lineal, y bloqueo volumétrico aún más acusado. **Preferir** `Hex8` en mallas hexaédricas; reservar `Tet4` para transiciones o geometrías que no admitan hexaedros.

4.3.3 Hexaedro Serendípito de Orden 2: `Hex20`

Hexaedro cuadrático de 20 nodos (8 vértices + 12 medios de arista, sin nodos de cara ni centroide). Análogo 3D del `Quad8`.

- **Nodos**: 20 · **Cuadratura**: Gauss 3×3×3 (27 puntos) por defecto.
- **Caras**: 6, de 8 nodos cada una (cara `Quad8`).
- **Tracción de cara**: reparto serendípito — cada vértice recibe $-A\bar{t}/12$ y cada medio $4A\bar{t}/12$. Los signos negativos en los vértices son el fenómeno serendípito conocido del `Quad8` (Cook-Malkus-Plesha-Witt §6.5), no un error.
- **Ventaja medida**: en la viga esbelta de MacNeal-Harder, una malla 6×1×1 de `Hex20` alcanza el 97 % de la deflexión de Euler-Bernoulli, frente a menos del 55 % de un `Hex8` 12×1×1.
- **Precaución**: con `hex_2x2x2` aparecen 6 modos de *hourglass* por elemento aislado, sin estabilización. Usar la cuadratura por defecto.

4.3.4 Hexaedro Lagrangiano Triquadrático: `Hex27`

Hexaedro Lagrangiano completo de 27 nodos (20 del `Hex20` + 6 centros de cara + 1 centro de cuerpo). Análogo 3D del `Quad9`.

- **Nodos**: 27 · **Cuadratura**: Gauss 3×3×3.
- **Caras**: 6, de 9 nodos cada una (cara `Quad9`).
- **Diferencia con** `Hex20`: reproduce **todos** los polinomios triquadráticos, incluidos los términos $\xi^2\eta^2\zeta^2$ que faltan en el serendípito. En flexión simple con geometría rectilínea ambos dan resultados prácticamente idénticos, con mayor coste para el `Hex27`; la diferencia se paga en geometrías de curvatura severa (Bathe FEP §5.3.2).
- **Precaución**: con `hex_2x2x2` aparecen 27 modos de *hourglass* por elemento aislado — la combinación más problemática del catálogo 3D. Usar 3×3×3 siempre.

4.3.5 Tetraedro Cuadrático: `Tet10`

Tetraedro de 10 nodos (4 vértices + 6 medios de arista). Análogo 3D del `Tri6`.

- **Nodos**: 10 · **Cuadratura**: Stroud `tet_4` (4 puntos) por defecto; `tet_15` (Keast, 15 puntos) para elementos distorsionados.
- **Caras**: 4 triangulares de 6 nodos (cara `Tri6`).
- **Masa**: la matriz consistente usa `tet_15` fijo, independientemente de la cuadratura elegida para K, para integrar exactamente el producto cuadrático×cuadrático en análisis modal y transitorio.
- **Cuándo usarlo**: mallas no estructuradas y geometrías complejas donde un mapeo hexaédrico no es viable. Mitiga drásticamente ambos bloqueos del `Tet4`.
- **Limitación conocida**: sobre **superficies curvas** malladas por descomposición de hexaedros en tetraedros, la representación isoparamétrica se degrada porque parte de los nodos medios quedan sobre aristas rectas. Sobre geometría plana el elemento es exacto. Para geometría curva con tetraedros conviene un mallador nativo que sitúe los nodos medios sobre la superficie.

```
elements:
  - {id: 1, type: Hex8,  material: 1, nodes: [1, 2, 3, 4, 5, 6, 7, 8]}
  - {id: 2, type: Tet4,  material: 1, nodes: [1, 2, 3, 4]}
  - {id: 3, type: Hex20, material: 1, nodes: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
                                              11, 12, 13, 14, 15, 16, 17, 18, 19, 20]}
  - {id: 4, type: Tet10, material: 1, nodes: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]}
```

4.4 Módulo de usuario `discontinuities`: elemento con discontinuidad embebida

Formulación de **fractura computacional** de Retama (2010): el elemento materializa una discontinuidad interna Γ_d cuando se cumple un criterio de activación, y enriquece su cinemática con un salto de desplazamientos u gobernado por un material cohesivo. Los grados de libertad del salto son **elementales**: se condensan dentro del elemento y nunca llegan al ensamblador, de modo que el tamaño del sistema global no cambia.

No es un elemento finito estándar, así que no forma parte del programa principal: vive en el **módulo de usuario** `discontinuities` (`solidum/user/discontinuities/`), junto con sus seguimientos de grieta (ADR 0020); su ley cohesiva es un material de interfaz del programa principal (ADR 0033). `import solidum` no lo carga; el YAML que use `CST_Embedded2D` debe declararlo:

```
user_modules: [discontinuities]
```

Sin esa línea, la lectura se detiene con un error que nombra el tipo o la sección desconocidos y lista los módulos de usuario disponibles.

4.4.1 Triángulo CST con Discontinuidad Interior: `CST_Embedded2D`

CST de 3 nodos con cinemática KOS enriquecida (Retama 2010, Caps. 2, 5, 6 y 7).

- **DOFs/nodo**: `ux, uy` (`STRAIN_DIM` = 3). Los 2 DOFs del salto son elementales, no globales.
- **Cuadratura**: 1 punto (hereda del `Tri3`).
- **Parámetros**: `material` (bulk), `cohesive_material`, `thickness` (opcional, default `1.0`) y `crack_tracker` (opcional).
- **Dos materiales**: a diferencia del resto del catálogo, requiere **un material de bulk** que gobierna el continuo y **uno cohesivo** que gobierna el salto en Γ_d. En YAML son dos campos distintos: `material` se busca en `materials` y `cohesive_material` en `interface_materials`. Sirve cualquier ley de interfaz de superficie con salto de dos componentes: `CohesiveDamageIsotropic` (modo I; la grieta desliza sin transmitir cortante) o `CohesiveDamageMixedMode` (modo mixto; transmite cortante y exige elementos más pequeños en la zona de la grieta: condición de tamaño en cortante, sección «Materiales de interfaz» del capítulo «Catálogo de Modelos Constitutivos»).
- **Estado intacto**: idéntico al `Tri3` hasta que la discontinuidad se activa.
- **Activación**: criterio de Rankine ($\sigma_I > \sigma_{t0}$ del cohesivo) evaluado en el centroide con el estado convergido del paso anterior; es el único criterio y no se declara en el YAML. Es **irreversible**: una vez activada persiste aunque el paso siguiente descargue.
- **Seguimiento de grieta**: qué elementos se agrietan y qué nodo queda solo a un lado lo decide la trayectoria de la grieta, igual para todos los vecinos. Por omisión, cada región conexa de elementos embebidos lleva un seguimiento global (`GlobalCrackTracking`): las grietas nacen donde σ_I supera σ_{t0} lejos de otras y crecen desde sus puntas siguiendo las direcciones principales. Si la trayectoria es conocida (grieta prescrita, como en una banda de

elementos embebidos), se declara `PrescribedCrackPath` con los vértices de la poligonal. Los dos se definen en la sección `crack_trackers` y el elemento los referencia con `crack_tracker`; `GlobalCrackTracking` admite `epsilon`, la regularización adimensional del problema auxiliar que traza las trayectorias (10^{-6} por omisión; spec `GlobalCrackTracking` §2). La activación ocurre al inicio del paso: conviene que, cerca del pico, cada paso de carga aumente el esfuerzo menos de un 4 % de σ_{t0}, o se activan de golpe varios elementos.

- **Longitud efectiva** l_d: $l_d = (A_e/h)\cos\psi$ (Retama 2010, ec. 6.24) se calcula con su signo. El seguimiento elige el nodo solitario de modo que $\cos\psi > 0$; una partición con $l_d \leq 0$ se rechaza con `ValueError` al activar el elemento.
- **Bulk aceptado**: sólo `Elastic2D` en la fase actual — la *discrete approach* presupone bulk elástico (ADR 0010).
- **Post-proceso**: `compute_gauss_state(U)` añade la clave `'discontinuity'` con normal, tangente, centroide, l_d, salto, tracción y daño cuando el elemento está agrietado.
- **Solver para la rama post-pico**: con desplazamiento impuesto, `NonlinearSolver` recorre el ablandamiento hasta la separación completa con penalización cohesiva rígida (tracción uniaxial frente a la solución exacta y ensayo de Van Vliet con grieta prescrita, en `tests/user/discontinuities/`). Las limitaciones registradas de `ArcLengthSolver` y de `DissipationArcLengthSolver` (módulo de usuario `dissipation_control`, al final del capítulo «Esquemas de Solución») con esa penalización son anteriores a la corrección de la tangente cohesiva del 2026-09-23 y no se han vuelto a comprobar.

```
user_modules: [discontinuities]
interface_materials:
  - {id: 1, type: CohesiveDamageIsotropic, sigma_t0: 2.5e6, G_f: 100.0,
     K_e: 1.0e13, softening: linear}
crack_trackers:                    # opcional
  - {id: 1, type: PrescribedCrackPath, points: [[0.0, 0.05], [0.2, 0.05]]}
elements:
  - {id: 1, type: CST_Embedded2D, nodes: [1, 2, 3],
     material: 1, cohesive_material: 1, crack_tracker: 1}
```

La formulación completa (cinemática KOS, condensación, longitud l_d) está en la spec del elemento, en el apéndice «Módulos de usuario» del Manual de Referencia.

Capítulo 5

Catálogo de Modelos Constitutivos

5.1 Elasticidad Lineal

5.1.1 `Elastic1D` — elasticidad lineal axial

Ley de Hooke escalar $\sigma = E \cdot \varepsilon$. Sin variables internas. Compatible con todos los elementos 1D (armaduras y marcos) y como material de fibra de una sección por fibras. Es el único material 1D que los marcos admiten con `material` + `A`, `I` (elástico lineal).

```yaml
materials:
  - id: 1
    type: Elastic1D
    E: 200.0e9
    density: 7850.0
```

5.1.2 `Elastic2D` — elasticidad lineal isótropa 2D

Tensor constitutivo isótropo $\boldsymbol{\sigma} = \mathbf{C} \cdot \boldsymbol{\varepsilon}$ en notación Voigt $[xx, yy, xy]$. Sin variables internas. Compatible con `Quad4`, `Tri3`.

```yaml
materials:
  - id: 1
    type: Elastic2D
    E: 200.0e9
    nu: 0.3
    hypothesis: plane_stress
    density: 7850.0
```

La hipótesis `plane_stress` se usa para placas delgadas sin confinamiento normal; `plane_strain` para secciones de presa, túneles o problemas con extensión infinita en el eje longitudinal.

5.2 Plasticidad

5.2.1 `Elastoplastic1D` — plasticidad J2 axial con endurecimiento isótropo

Plasticidad asociativa con criterio $f = |\sigma_{\text{trial}}| - (\sigma_y + H\alpha) \leq 0$. Algoritmo de *return mapping* clásico 1D; tangente algorítmica consistente $E_t = E\,H/(E + H)$.

- **Parámetros**: `E`, `sigma_y` (fluencia inicial), `H` (módulo de endurecimiento; `H=0` = perfectamente plástico).
- **Variables internas**: ε_p (deformación plástica), α (acumulada equivalente).
- **Compatible con**: armaduras y, en los marcos, **sólo como material de fibra** de una sección por fibras (`section`, bloque `sections` del capítulo *Archivo de Entrada YAML*): así se forman rótulas plásticas por flexión. Un marco con `material` + `A`, `I` lo rechaza al construir.

```yaml
materials:
  - id: 1
    type: Elastoplastic1D
    E: 200.0e9
    sigma_y: 250.0e6
    H: 2.0e9
    density: 7850.0
```

5.2.2 `VonMises2D` — plasticidad J2 plana con endurecimiento isótropo

Plasticidad asociativa con criterio J2 y endurecimiento isótropo lineal. Soporta **dos hipótesis cinemáticas** mutuamente excluyentes, seleccionadas con el campo `hypothesis` al construir el material:

- `plane_strain` ($\varepsilon_{zz} = 0$): return mapping radial cerrado sobre la parte desviadora 3D extendida (Simó-Hughes §3.3). Corrector $\Delta\gamma = f_{\text{trial}}/(2G + \frac{2}{3}H)$ con $N = s_{\text{trial}}/\|s_{\text{trial}}\|$ y actualización $\alpha_{\text{new}} = \alpha + \sqrt{2/3}\,\Delta\gamma$. Default histórico.
- `plane_stress` ($\sigma_{zz} = 0$): *plane stress projected algorithm* (Simó-Hughes §3.4.1). Función de fluencia proyectada $\bar{f} = \frac{1}{2}\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma} - R^2/3$ y Newton local escalar sobre $\Delta\gamma$. Converge en 3–6 iteraciones por evaluación. Actualización físicamente correcta $\alpha_{\text{new}} = \alpha + \Delta\gamma\sqrt{2\boldsymbol{\sigma}^\top \mathbf{P}\,\boldsymbol{\sigma}/3}$ (en plane stress $\Delta\gamma$ tiene unidades [1/esfuerzo], por lo que la heurística $\sqrt{2/3}\,\Delta\gamma$ válida en plane strain rompe la invariancia bajo cambio de unidades). Incompresibilidad plástica cierra $e^p_{zz} = -(e^p_{xx} + e^p_{yy})$.

Parámetros comunes: `E`, `nu`, `sigma_y`, `H` (≥ 0, default 0 = perfectamente plástico), `hypothesis` (default `plane_strain`), `density` (opcional). Variables internas: ε_p tensorial 4 componentes $[xx, yy, zz, xy_{\text{tens}}]$ y α acumulada equivalente adimensional. **Tangente algorítmica consistente cerrada** en ambas hipótesis (con corrección por $\mathrm{d}\alpha/\mathrm{d}\Delta\gamma \neq \sqrt{2/3}$ en plane stress). Compatible con todos los elementos 2D del catálogo.

```yaml
materials:
  # Hipótesis por defecto: plane_strain
  - id: 2
    type: VonMises2D
    E: 210.0e9
    nu: 0.3
    sigma_y: 250.0e6
    H: 2.0e9
    hypothesis: plane_strain
    density: 7850.0
```

```yaml
# Mismo material en plane_stress (placas delgadas sin confinamiento normal)
- id: 3
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
  hypothesis: plane_stress
  density: 7850.0
```

5.2.3 `DruckerPrager2D` — plasticidad friccional cohesivo-friccional 2D

Modelo de Drucker-Prager: cono circular suave de Mohr-Coulomb con cohesión y fricción interna. Criterio de fluencia $f = \sqrt{J_2} + \eta_f\, I_1 - k(\alpha) \le 0$ con $k(\alpha) = k_0 + H\alpha$ (endurecimiento isótropo lineal en cohesión). Plasticidad *no* asociada por defecto: el ángulo de dilatancia ψ es parámetro independiente del ángulo de fricción ϕ.

Algoritmo — dos ramas con detección automática:

- **Return regular** (superficie del cono): $\Delta\gamma = f_{\text{trial}}/(G + 9K\eta_f\eta_g + H)$, dirección desviadora preservada. Tangente algorítmica $\mathbf{C}_{\text{alg}} = K\,\mathbf{v}\otimes\mathbf{v} + 2G(1-\beta)\mathbf{I}_{\text{dev}} + 4G\beta\,\hat{\mathbf{n}}\otimes\hat{\mathbf{n}} - (1/A)\,\mathbf{b}_g\otimes\mathbf{b}_f$ con $\beta = G\Delta\gamma/\sqrt{J_2^{\text{trial}}}$.
- **Return al ápice** (vértice puramente hidrostático): si tras el return regular el estado caería fuera del cono, conmuta a $\Delta\gamma_{\text{apex}} = (I_1^{\text{trial}}\eta_f - k(\alpha))/(9K\eta_f\eta_g + H)$ con $\boldsymbol{\sigma}_{n+1} = (k/3\eta_f)\,\mathbf{I}$ puramente hidrostático. Tangente reducida $KH/(9K\eta_f\eta_g + H)\,\mathbf{v}\otimes\mathbf{v}$.

Calibración con Mohr-Coulomb (campo `variant`):

- `plane_strain_matched` (default): coincide exactamente con MC en plane strain. $\eta_f = \tan\phi/\sqrt{9 + 12\tan^2\phi}$, $k_0 = 3c_0/\sqrt{9 + 12\tan^2\phi}$.
- `outer_cone`: circunscribe MC. $\eta_f = 2\sin\phi/[\sqrt{3}(3 - \sin\phi)]$.
- `inner_cone`: inscribe MC. $\eta_f = 2\sin\phi/[\sqrt{3}(3 + \sin\phi)]$.

Parámetros: `E`, `nu`, `cohesion` (cohesión inicial c_0), `phi_deg` ($0 \le \phi < 90$), `psi_deg` (dilatancia, $0 \le \psi \le \phi$; default $\psi = \phi$ = asociada), `H` (≥ 0, default 0), `variant` (default `plane_strain_matched`), `density` (opcional). Variables internas: ε_p tensorial 4 componentes y α acumulada. `IS_SYMMETRIC = False` declarativo (no asociada ⇒ tangente asimétrica; el despachador algebraico ADR 0003 elige LU).

Advertencia

Restricción cinemática: solo `plane_strain`. La proyección con $\sigma_{zz} = 0$ acoplada a flujo dilatante es notoriamente delicada y queda *out-of-scope* en esta entrega.

```yaml
materials:
  - id: 4
    type: DruckerPrager2D
    E: 30.0e9
    nu: 0.25
    cohesion: 2.0e5       # Pa
    phi_deg: 30.0
    psi_deg: 10.0         # no asociada (psi < phi)
    H: 0.0                # plasticidad perfecta
    variant: plane_strain_matched
    density: 2000.0       # arena densa
```

5.3 Daño Mecánico

5.3.1 `IsotropicDamage1D` — daño isótropo 1D con softening exponencial

Daño escalar $d \in [0, 1)$ con esfuerzo nominal $\sigma = (1 - d)\, E\, \varepsilon$. Evolución por norma de la deformación equivalente $\varepsilon_{eq} = |\varepsilon|$ y umbral κ_0:

$$d = 1 - \frac{\kappa_0}{\kappa} \exp\big(-\alpha(\kappa - \kappa_0)\big) \quad \text{para } \kappa > \kappa_0$$

- **Parámetros**: `E`, `kappa_0` (umbral elástico), `alpha` (velocidad de softening), `density` (opcional).
- **Variables internas**: κ (máxima histórica), d (daño).
- **Tangente**: **algorítmica consistente** en carga activa con daño no saturado: $E_{\tan} = -(1 - d)\, E\, \alpha\, \kappa$ — negativa, refleja la pendiente descendente σ-ε post-pico. En descarga, sin daño activo o al saturar, vuelve a la secante $(1 - d)E$.
- **Compatible con**: armaduras y, en los marcos, sólo como material de fibra de una sección por fibras. En fibras, el ablandamiento se localiza en uno o dos puntos de Gauss y la respuesta posterior al pico depende de la malla (sin regularización).

5.3.2 `IsotropicDamage2D` — daño isótropo 2D

Extensión 2D del modelo anterior. Esfuerzo nominal $\boldsymbol{\sigma} = (1 - d)\, \mathbf{C}_e\, \boldsymbol{\varepsilon}$, con el daño gobernado por una deformación equivalente ε_{eq} que se elige con el parámetro `equivalent_strain`. Las tres medidas disponibles dan $\varepsilon_{eq} = \varepsilon$ en tracción uniaxial de esfuerzo, así que el umbral es siempre $\kappa_0 = f_t/E$:

`equivalent_strain`	**Definición**	**Compresión uniaxial de esfuerzo**
`energy` (por defecto; Simó y Ju 1987)	$\varepsilon_{eq} = \sqrt{\boldsymbol{\varepsilon}^\top \mathbf{C}_e\, \boldsymbol{\varepsilon}/E}$	daña igual que en tracción: $\varepsilon_{eq} = \lvert\varepsilon\rvert$
`mazars` (Mazars 1986)	$\varepsilon_{eq} = \sqrt{\sum_i \langle\varepsilon_i\rangle_+^2}$, sólo extensiones principales	daña por la extensión lateral: $\varepsilon_{eq} = \sqrt{2}\,\nu\,\lvert\varepsilon\rvert$
`modified_von_mises` (de Vree et al. 1995)	función de I_1 y J_2 con $k = f_c/f_t$	$\varepsilon_{eq} = \lvert\varepsilon\rvert/k$: empieza a dañar con $\lvert\varepsilon\rvert = k\,\kappa_0$

Con von Mises modificado, $\varepsilon_{eq} = \frac{k-1}{2k(1-2\nu)}\, I_1 + \frac{1}{2k}\sqrt{\left(\frac{k-1}{1-2\nu}\right)^2 I_1^2 + \frac{12k}{(1+\nu)^2}\, J_2}$. Mazars y von Mises modificado se evalúan sobre el tensor 3D con la componente fuera del plano $\varepsilon_{zz} = c_{zz}(\varepsilon_{xx} + \varepsilon_{yy})$, donde $c_{zz} = -\nu/(1-\nu)$ en tensión plana y $c_{zz} = 0$ en deformación plana; la energética usa la $\mathbf{C}_e$ plana del material, que ya la incluye. Con cualquiera de las tres el 2D es la restricción exacta de `IsotropicDamage3D` en las dos hipótesis.

- **Parámetros**: `E`, `nu`, `kappa_0`, `alpha`, `hypothesis` (`plane_stress` default o `plane_strain`), `equivalent_strain` (`energy` default, `mazars` o `modified_von_mises`), `k` ($f_c/f_t > 0$: obligatorio con `modified_von_mises` y no admitido con las otras dos), `density` (opcional).
- **Tangente algorítmica consistente**: $\mathbf{C}_{\text{alg}} = (1-d)\,\mathbf{C}_e - (1-d)\left(\frac{1}{\kappa} + \alpha\right)(\mathbf{C}_e\boldsymbol{\varepsilon}) \otimes \frac{\partial \varepsilon_{eq}}{\partial \boldsymbol{\varepsilon}}$ en carga activa. Con `energy` es **simétrica** y el material se declara simétrico ante el despachador algebraico (ADR 0003); con `mazars` y `modified_von_mises` no lo es y el despachador elige LU. Recupera convergencia cuadrática del Newton global frente a la secante.
- **Cambio del 2026-09-29**: hasta esa fecha la medida era la norma del tensor $\sqrt{\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}}$, que en tracción uniaxial daba $\varepsilon_{eq} = \varepsilon\sqrt{1+2\nu^2}$ (el daño empezaba en $\varepsilon = \kappa_0/\sqrt{1+2\nu^2}$). Un YAML existente pasa a la medida energética sin tocarlo, y sus resultados cambian porque la calibración es otra: ahora $\kappa_0 = f_t/E$ directamente.
- **Limitación**: un único daño escalar (el modelo de Mazars completo, con daños de tracción y compresión separados, no está implementado; aquí sólo su medida) y sin restitución de rigidez al cerrar fisuras. Sin regularización por longitud característica → *mesh-dependency* en régimen de ablandamiento.

```yaml
materials:
  # Medida energética (por defecto): no hace falta declararla.
  - id: 3
    type: IsotropicDamage2D
    E: 30.0e9
    nu: 0.2
    kappa_0: 1.0e-4       # f_t/E = 3 MPa / 30 GPa
    alpha: 800.0
    hypothesis: plane_stress
    density: 2500.0       # hormigón
  # Mazars: sólo las extensiones principales dañan.
  - id: 4
    type: IsotropicDamage2D
    E: 30.0e9
    nu: 0.2
    kappa_0: 1.0e-4
    alpha: 800.0
    hypothesis: plane_strain
    equivalent_strain: mazars
  # von Mises modificado: exige k = f_c/f_t ≥ 1.
  - id: 5
    type: IsotropicDamage2D
    E: 30.0e9
    nu: 0.2
    kappa_0: 1.0e-4
```

```
    alpha: 800.0
    equivalent_strain: modified_von_mises
    k: 10.0                # f_c = 30 MPa, f_t = 3 MPa
```

5.3.3 `IsotropicDamageCrackBand2D` — daño isótropo con banda de grieta (fisuración distribuida)

Variante de `IsotropicDamage2D` para el agrietamiento del concreto con fisuración distribuida (Bažant y Oh 1983). Los datos son los del concreto: la resistencia a tracción f_t y la energía de fractura G_f. La velocidad de ablandamiento no se da: cada punto de integración la calcula con la **longitud característica** h de su elemento,

$$\alpha(h) = \frac{1}{\dfrac{G_f}{h\,f_t} - \dfrac{\kappa_0}{2}}, \qquad \kappa_0 = \frac{f_t}{E},$$

para disipar G_f/h por unidad de volumen: cuando la deformación se localiza en una fila de elementos, la energía disipada por unidad de área de grieta es G_f con cualquier malla. El elemento siembra h al construirse ($\sqrt{A}$ en `Quad4`, $\sqrt{2A}$ en `Tri3`, $\frac{1}{2}\sqrt{A}$ en `Tri6`, `Quad8` y `Quad9`); con elementos mayores que $h_{\max} = 2EG_f/f_t^2$ (0.67 m en concreto usual) el modelo se rechaza al construirse.

- **Parámetros**: `E`, `nu`, `f_t`, `G_f` (energía por unidad de área, N/m en SI), `hypothesis`, `equivalent_strain` (`rankine` por omisión: $\varepsilon_{eq} = \langle\bar{\sigma}_I\rangle_+/E$ con el esfuerzo efectivo, nunca daña en compresión; también `energy`, `mazars` y `modified_von_mises` con `k`), `damage_max` (tope del daño, $1 - 10^{-6}$ por omisión), `density`.
- **Con qué medida**: con banda de grieta todo daño ablanda con la G_f de tracción. `rankine` es la adecuada para agrietamiento dominado por tracción; las otras dañan también en compresión (la energética a f_t, Mazars a $\approx 3.5f_t$, von Mises modificado a f_c pero con una compresión demasiado frágil).
- **Integración**: implícita. En el ablandamiento el Newton puede detenerse cuando varias zonas se dañan a la vez; la integración IMPL-EX, más robusta, está en preparación.
- **Limitaciones**: daño escalar (al agrietarse el punto pierde rigidez también en compresión), sin cierre de grietas; la banda tiende a seguir las líneas de la malla. En una banda que sus vecinos impiden contraerse lateralmente, la energía disipada es $(1-\nu^2)\,G_f$ por unidad de área.

```
materials:
  # Concreto: E = 30 GPa, f_t = 3 MPa, G_f = 100 N/m; Rankine por omisión.
  - id: 6
    type: IsotropicDamageCrackBand2D
    E: 30.0e9
    nu: 0.2
    f_t: 3.0e6
    G_f: 100.0
    hypothesis: plane_stress
    density: 2400.0
```

5.4 Elasticidad Unilateral (Cable)

5.4.1 `CableMaterial1D` — cable lineal en tracción, nulo en compresión

Material 1D *memoryless* que captura la propiedad esencial del cable: solo transmite esfuerzo cuando está tensado.

$$\sigma(\varepsilon) = E\,\langle\varepsilon\rangle^{+} = \begin{cases} E\,\varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \le 0 \end{cases}$$

con $\langle\cdot\rangle^{+}$ el operador de MacAuley (parte positiva). Tangente discontinua en $\varepsilon = 0$ (asignación al régimen compresivo por convención).

- **Parámetros**: `E` (módulo de Young en tracción).
- **Sin variables internas** (la respuesta depende sólo de ε instantánea).
- **Caveats numéricos**: en transiciones tensado ↔ destensado Newton-Raphson puede oscilar. Mitigación: usar `ArcLengthSolver` o pasos finos con `adaptive`.

```yaml
materials:
  - id: 1
    type: CableMaterial1D
    E: 150.0e9
    density: 7850.0
```

5.5 Concreto

5.5.1 `ManderConcrete1D` — concreto uniaxial con la envolvente de Mander

Material de fibra y de barra para el concreto, simple o confinado. La envolvente de compresión es la de Mander, Priestley y Park (1988) con la ecuación de Popovics; la descarga y la recarga son rectas con la rigidez dañada $(1-d)E$ hacia la deformación plástica de Mander, de modo que el material no puede crear energía. En tracción: lineal hasta f_t y una recta hasta cero, descarga hacia el origen desplazado, recuperación de la rigidez de compresión al cerrarse la grieta y pérdida de resistencia a tracción por la compresión previa (factor de Mander $1 - e_p/\varepsilon_c$). No reproduce los lazos de los ciclos parciales.

- **Parámetros**: `fc`, `eps_c` (resistencia y deformación de pico, en magnitud), `E` (módulo inicial; Mander: $5000\sqrt{f'_c}$ MPa), `eps_cu` (fin de la curva de Popovics); opcionales `eps_sp` (recta hasta cero, desprendimiento del recubrimiento), `ft` (0: sin tracción), `eps_tu` o `G_f` (rama descendente de tracción en deformación o en energía de fractura), `G_fc` (energía de la rama de compresión posterior al pico), `density`.
- **Confinamiento**: `confinement` calcula `fc`, `eps_c`, `eps_cu` y `E` del núcleo a partir del concreto sin confinar y de los estribos (Mander 1988; Chang y Mander 1994 para presiones distintas en x e y). `k_e` se da o se calcula con la geometría (`{b_c, d_c, s_clear, w_clear, rho_cc}` en la rectangular; `{s_clear, d_s, rho_cc, spiral}` en la circular).

- **Regularización**: con `G_f` o `G_fc`, la rama descendente se ajusta a la longitud del punto de integración (la de la barra, o la tributaria del punto de Gauss del marco) y el resultado deja de depender de la malla.
- **Uso**:
- En concreto reforzado, dar la tracción con `eps_tu` (de 5 a 20 veces f_t/E, la rigidez del concreto entre grietas) y regularizar sólo la compresión con `G_fc`.
- `G_f`, para el concreto simple.
- En transitorios, `line_search: true`.
- **Compatible con**: armaduras y fibras de una sección por fibras.

```yaml
materials:
  - id: 1            # núcleo confinado
    type: ManderConcrete1D
    confinement: {fc0: 30.0e6, f_yh: 420.0e6, eps_su: 0.09, k_e: 0.75, rho_x: 0.01,
        rho_y: 0.01}
    ft: 3.0e6
    eps_tu: 1.1e-3
    density: 2400.0
  - id: 2            # recubrimiento
    type: ManderConcrete1D
    fc: 30.0e6
    eps_c: 0.002
    E: 27.4e9
    eps_cu: 0.004
    eps_sp: 0.006
    ft: 3.0e6
    eps_tu: 1.1e-3
    density: 2400.0
```

5.6 Materiales 3D

Los materiales 3D operan sobre la convención **Voigt 6D** del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx},\ \varepsilon_{yy},\ \varepsilon_{zz},\ \gamma_{xy},\ \gamma_{yz},\ \gamma_{xz}]^\top, \qquad \boldsymbol{\sigma} = [\sigma_{xx},\ \sigma_{yy},\ \sigma_{zz},\ \sigma_{xy},\ \sigma_{yz},\ \sigma_{xz}]^\top$$

con $\gamma_{ij} = 2\varepsilon_{ij}$ (deformación angular *engineering*). Sólo son compatibles con elementos 3D (`STRAIN_DIM` $= 6$).

Ninguno admite `hypothesis`: las variantes `plane_stress` / `plane_strain` no tienen sentido en 3D, donde todas las componentes están activas. Declararla es un error.

Caveat de interoperabilidad con ABAQUS: ABAQUS ordena Voigt como $[11, 22, 33, 12, 13, 23]$, es decir con el bloque cortante permutado ($yz \leftrightarrow xz$) respecto al proyecto. Importar tensores de ABAQUS exige intercambiar las componentes 5 y 6 una sola vez en el preprocesador.

5.6.1 `Elastic3D` — elástico lineal isótropo 3D

$\boldsymbol{\sigma} = \mathbf{C}\,\boldsymbol{\varepsilon}$ con $\mathbf{C}$ isótropa 6×6. Tangente constante, sin variables internas.

- **Parámetros**: `E` (> 0), `nu` $\in (-1, 0.5)$, `density` (opcional).

5.6.2 `VonMises3D` — plasticidad J2 3D con endurecimiento isótropo lineal

Extensión 3D de `VonMises2D`. Flujo asociado y *return mapping* radial cerrado sobre la parte desviadora (Simó-Hughes §3.3), con tangente algorítmica consistente.

- **Parámetros**: `E`, `nu`, `sigma_y` (> 0), `H` (≥ 0, default 0), `density` (opcional).
- **Variables internas**: `eps_p` (6 componentes) y `alpha` (deformación plástica equivalente).
- **Relación con el 2D**: bajo la restricción $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ se reduce exactamente a `VonMises2D` en `plane_strain`, verificado a 10 decimales.

5.6.3 `DruckerPrager3D` — plasticidad friccional 3D

Cono circular suave de Mohr-Coulomb, criterio $f = \sqrt{J_2} + \eta_f I_1 - k(\alpha) \leq 0$ con endurecimiento isótropo lineal en la cohesión. Dos ramas cerradas con detección automática: retorno regular a la superficie y retorno al ápice hidrostático.

- **Parámetros**: `E`, `nu`, `cohesion`, `phi_deg` (ángulo de fricción en grados), `psi_deg` (dilatancia en grados, default $= \phi$), `H` (default 0), `variant` (default `outer_cone`), `density` (opcional).
- **Calibraciones**: `outer_cone` (default, circunscribe Mohr-Coulomb) e `inner_cone` (inscribe). La variante `plane_strain_matched` del modelo 2D **no existe en 3D** y el constructor la rechaza con mensaje explícito: es una calibración 2D por construcción, porque Mohr-Coulomb en 3D depende del ángulo de Lode.
- **Plasticidad no asociada**: con $\psi \neq \phi$ la tangente es asimétrica y el despachador algebraico selecciona LU automáticamente.

5.6.4 `IsotropicDamage3D` — daño isótropo 3D con softening exponencial

$\boldsymbol{\sigma} = (1 - d)\,\mathbf{C}_e\,\boldsymbol{\varepsilon}$ con daño escalar d gobernado por la deformación equivalente ε_{eq}, que se elige con `equivalent_strain` entre las mismas tres medidas que `IsotropicDamage2D` (tabla de la sección anterior), ahora sobre el tensor 3D completo:

- `energy` (por defecto): $\varepsilon_{eq} = \sqrt{\boldsymbol{\varepsilon}^\top \mathbf{C}_e\,\boldsymbol{\varepsilon}/E}$;
- `mazars`: $\varepsilon_{eq} = \sqrt{\sum_i \langle \varepsilon_i \rangle_+^2}$ sobre las tres deformaciones principales (sin daño en compresión hidrostática);
- `modified_von_mises`: función de $I_1 = \varepsilon_{xx}+\varepsilon_{yy}+\varepsilon_{zz}$ y de $J_2 = \frac{1}{6}\left[(\varepsilon_{xx} - \varepsilon_{yy})^2 + (\varepsilon_{yy} - \varepsilon_{zz})^2 + (\varepsilon_{zz} - \varepsilon$ $\frac{1}{4}(\gamma_{xy}^2 + \gamma_{yz}^2 + \gamma_{xz}^2)$, con $k = f_c/f_t$.

Las tres dan $\varepsilon_{eq} = \varepsilon$ en tracción uniaxial de esfuerzo ($\kappa_0 = f_t/E$). Historial κ máximo con condiciones de Kuhn-Tucker y ley de softening exponencial compartida con las versiones 1D y 2D. Hasta el 2026-09-29 la medida era la norma del tensor $\sqrt{\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}}$ (retirada; ver `IsotropicDamage2D`).

- **Parámetros**: `E`, `nu`, `kappa_0` (umbral de daño), `alpha` (parámetro de la ley de softening), `equivalent_strain` (`energy` default, `mazars` o `modified_von_mises`), `k` ($f_c/f_t > 0$, sólo y obligatorio con `modified_von_mises`), `density` (opcional).
- **Tangente**: algorítmica consistente en carga activa (simétrica con `energy`, no simétrica en general con `mazars` y `modified_von_mises`), secante en descarga y al saturar.

```
materials:
  - {id: 1, type: Elastic3D,   E: 210.0e9, nu: 0.3, density: 7850.0}
  - {id: 2, type: VonMises3D,  E: 210.0e9, nu: 0.3, sigma_y: 250.0e6, H: 1.0e9}
  # Daño 3D con cada medida de deformación equivalente (hormigón, f_t = 3 MPa):
  - {id: 3, type: IsotropicDamage3D, E: 30.0e9, nu: 0.2, kappa_0: 1.0e-4, alpha: 800.0}
  - {id: 4, type: IsotropicDamage3D, E: 30.0e9, nu: 0.2, kappa_0: 1.0e-4, alpha: 800.0,
     equivalent_strain: mazars}
  - {id: 5, type: IsotropicDamage3D, E: 30.0e9, nu: 0.2, kappa_0: 1.0e-4, alpha: 800.0,
     equivalent_strain: modified_von_mises, k: 10.0}
  # Fisuración distribuida con banda de grieta (ver IsotropicDamageCrackBand2D):
  - {id: 6, type: IsotropicDamageCrackBand3D, E: 30.0e9, nu: 0.2, f_t: 3.0e6, G_f:
      100.0}
```

`IsotropicDamageCrackBand3D` es la versión 3D de `IsotropicDamageCrackBand2D` (mismos parámetros salvo `hypothesis`). La longitud característica la siembran `Hex8` ($\sqrt[3]{V}$), `Tet4` ($\sqrt[3]{6V}$) y los cuadráticos `Hex20`, `Hex27` y `Tet10` ($\frac{1}{2}\sqrt[3]{V}$).

5.7 Materiales de interfaz (tracción frente a desplazamiento relativo)

Familia **paralela** al resto del catálogo: no relacionan $\boldsymbol{\sigma}$ con $\boldsymbol{\varepsilon}$ sino la **tracción t** con el **salto de desplazamientos** u entre los dos lados de una interfaz: una superficie de discontinuidad (grieta, junta) o la línea de una barra embebida en un sólido (adherencia). Las componentes van en el sistema local de la interfaz, normales primero y tangenciales después.

Son una familia del programa principal (ADR 0033): se declaran en el bloque `interface_materials`, no en `materials`, y los citan por `id` los elementos y restricciones que tienen una interfaz. Hasta el 2026-11-10 la ley cohesiva pertenecía al módulo de usuario `discontinuities` y su sección se llamaba `cohesive_materials`.

5.7.1 `CohesiveDamageIsotropic` — daño cohesivo Modo-I

Daño escalar $\omega \in [0, 1]$ sobre el salto normal: $t_n = (1 - \omega) K_e u_n$, con $t_s = 0$ (Modo-I puro). La energía de fractura G_f cierra la curva analíticamente, en variante lineal (apertura crítica $w_c = 2G_f/\sigma_{t0}$) o exponencial (asintótica).

- **Parámetros**: `sigma_t0` (resistencia a tracción, Pa), `G_f` (energía de fractura, N/m), `K_e` (rigidez del salto, Pa/m), `softening` ∈ `linear`, `exponential`.

- **Sobre** `K_e`: es un *penalty* que representa la cohesión antes de agrietar, **no** una propiedad del bulk. No tiene default automático; como guía, $K_e \approx 10\,E_{bulk}/\ell_c$. Valores muy altos endurecen el sistema y dificultan la convergencia; valores bajos introducen flexibilidad espuria antes de la activación.
- **Variables internas**: `kappa` (historial del salto) y `damage` (ω).

```
interface_materials:
  - {id: 1, type: CohesiveDamageIsotropic, sigma_t0: 2.5e6, G_f: 100.0,
     K_e: 1.0e13, softening: linear}
```

5.7.2 `CohesiveDamageMixedMode` — daño cohesivo de modo mixto I–II

Generaliza la ley anterior a la apertura y el deslizamiento de la grieta con una **apertura efectiva** $\delta_{eff} = \sqrt{\langle\delta_n\rangle^2 + \beta^2\delta_s^2}$ (Camacho y Ortiz 1996): $t_n = S\,\delta_n$, $t_s = \beta^2 S\,\delta_s$, con $S = T(\kappa)/\kappa$ y κ la máxima apertura efectiva alcanzada. La grieta inicia cuando $(t_n/\sigma_{t0})^2 + (t_s/\beta\sigma_{t0})^2 = 1$, así que la resistencia a cortante es $\beta\,\sigma_{t0}$, y la energía de fractura es la misma, G_f, en tracción, en cortante y en cualquier combinación proporcional. Con $\beta = 0$ es `CohesiveDamageIsotropic`.

- **Parámetros**: `sigma_t0`, `G_f`, `K_e` (como en la ley de modo I; la rigidez tangencial es $\beta^2 K_e$), `beta` (obligatorio; las calibraciones publicadas para concreto dan τ_c/σ_{t0} entre 1.0 y 2.6) y `softening` ∈ `linear`, `exponential`, `bilinear`.
- **Bilineal del fib Model Code 2010**: $\sigma = \sigma_{t0}(1 - 0.8\,w/w_1)$ hasta $w_1 = G_f/\sigma_{t0}$ y $\sigma = \sigma_{t0}(0.25 - 0.05\,w/w_1)$ hasta $5G_f/\sigma_{t0}$. Otra calibración con `knee_stress_ratio` (esfuerzo en el quiebre entre σ_{t0}, 0.2 por omisión) y `final_opening_factor` (apertura final por σ_{t0}/G_f, 5 por omisión); Petersson (1981): 1/3 y 3.6.
- **Sin fricción**: con la grieta cerrada y comprimida el deslizamiento también daña y el cortante no crece con la compresión.
- **Tamaño de los elementos embebidos**: en cortante la condición de tamaño es $\beta^2|T'|\,h < G$, más estricta que la de tracción; con la bilineal, $h < \ell_{ch}/(1.6\,\beta^2(1+\nu))$ con $\ell_{ch} = E\,G_f/\sigma_{t0}^2$ (unos 8 cm para un concreto usual con $\beta = 1.5$). Si se viola, el Newton local del salto puede no converger.

```
interface_materials:
  - {id: 2, type: CohesiveDamageMixedMode, sigma_t0: 3.0e6, G_f: 100.0,
     K_e: 1.0e15, beta: 1.5, softening: bilinear}
```

5.7.3 `BondSlipModelCode` y `BondSlipMultilinear` — adherencia-deslizamiento barra–concreto

Leyes de interfaz de **línea** para la adherencia de una barra embebida (opción `bond` del refuerzo embebido, capítulo *Archivo de Entrada YAML*). Relacionan el esfuerzo de adherencia τ con el deslizamiento s de la barra respecto del concreto, y la tracción normal con el desplazamiento normal, $t_n = k_n\,\Delta u_n$ (desacoplada).

- **Envolvente** (`BondSlipModelCode`, Eligehausen, Popov y Bertero 1983 / fib Model Code 2010): tramo lineal $k_0\, s$ hasta s_0, $\tau_{max}(s/s_1)^{\alpha}$ hasta s_1, meseta hasta s_2, descenso lineal hasta τ_f en s_3 y residual τ_f. Con `model_code_2010` los parámetros salen de la Tabla 6.1-1 a partir de f_{cm}, la condición de adherencia, el modo de falla (arrancamiento o hendimiento, con o sin estribos) y la separación libre entre corrugas; $k_0 = 180$ N/mm^3 y $k_n = 2E_c/d_b$ por omisión.
- **Envolvente por puntos** (`BondSlipMultilinear`): `points: [[s, τ], ...]`.
- **Descarga y recarga**: rama rígida con k_0 hasta la fricción $\mp\tau_f$, deslizamiento a fricción constante y recarga con k_0 hasta la envolvente (Fig. 5.3b de Eligehausen et al.), **sin degradación**: adecuada para carga monótona y pocos ciclos; en ciclos pequeños sobrestima la energía disipada frente a la ley completa de Eligehausen.
- **Unidades SI**: Pa, m, Pa/m.

```
interface_materials:
  - {id: 1, type: BondSlipModelCode, tau_max: 13.5e6, s1: 1.0e-3, s2: 3.0e-3,
     s3: 10.5e-3, alpha: 0.4, tau_f: 5.0e6, k0: 1.8e11, k_n: 3.0e12}
  - {id: 2, type: BondSlipMultilinear, points: [[1.0e-4, 6.0e6], [1.0e-3, 12.0e6], [4.0
      e-3, 3.0e6]],
     k_n: 3.0e12}
```

Capítulo 6

Esquemas de Solución

Este capítulo cubre los **solvers estáticos** del catálogo: lineal, no lineal (Newton-Raphson), arc-length (Crisfield cilíndrico) y control indirecto de desplazamiento; al final, el arc-length por disipación de energía del módulo de usuario `dissipation_control`. Los solvers de análisis modal, dinámico (Newmark, HHT-α, central differences) y de análisis en frecuencia / espectral (Harmonic, Response Spectrum) se documentan en el siguiente capítulo, "Análisis Dinámico".

6.1 `LinearSolver` — solución directa lineal

Resuelve $\mathbf{K} \cdot \mathbf{U} = \mathbf{F}$ con una sola factorización del sistema (el backend lo elige Solidum: Cholesky, Pardiso o LU según la matriz y lo instalado). Las condiciones de Dirichlet se imponen por eliminación directa (ADR 0004): el sistema entregado al backend algebraico es ya el reducido $\mathbf{K}_{\text{red}} \cdot \mathbf{U}_{\text{libre}} = \mathbf{F}_{\text{red}}$.

- **Cuándo usarlo**: problemas estrictamente lineales (todos los materiales con tangente constante, sin grandes desplazamientos, sin contacto).
- **Parámetros**: `linear_algebra` (default `auto`; ver *Solver* en el capítulo del YAML).
- **Comprobaciones automáticas** (ADR 0019): antes de resolver, que el modelo esté suficientemente apoyado; después, que la solución cumpla el equilibrio y que la matriz no sea singular. Si algo falla, el análisis se detiene con una explicación en vez de devolver resultados absurdos (ver *Diagnóstico de Problemas*). Con `linear_algebra: iterative` la matriz no se factoriza y no se puede comprobar si es singular: Solidum lo avisa en el log, una vez por análisis.
- **No aplica**: cualquier no-linealidad material (daño, plasticidad, cable) o geométrica (corotacional). Producirá resultados inconsistentes.

```yaml
solver:
  type: LinearSolver
```

6.2 `NonlinearSolver` — Newton-Raphson incremental con paso adaptativo

Bucle externo de incrementos del factor de carga $\lambda \in [0, 1]$; bucle interno Newton-Raphson sobre $\mathbf{K}_t \cdot \Delta\mathbf{U} = \mathbf{R} = \lambda\,\mathbf{F}_{\text{ext}} - \mathbf{F}_{\text{int}}$.

Criterio de convergencia dual (se exigen los dos a la vez):

$$\|\mathbf{T}^\top\mathbf{R}\| \le \texttt{atol_force}+\texttt{rtol_force}\cdot\max(\|\mathbf{F}_{\text{ext}}\|, \|\mathbf{F}_{\text{int}}\|), \qquad \|\Delta\mathbf{U}\| \le \texttt{atol_disp}+\texttt{rtol_disp}\cdot\|\mathbf{U}\|$$

donde $\mathbf{T}^\top\mathbf{R}$ es el residuo de equilibrio en los grados de libertad libres: las reacciones de los apoyos no cuentan y, con restricciones multipunto, el residuo de cada esclavo se suma a sus maestros multiplicado por su coeficiente. Las tolerancias se configuran en el bloque `convergence` (capítulo «Archivo de Entrada YAML»).

Desplazamientos prescritos: los valores de Dirichlet no nulos y el término independiente g de las restricciones multipunto se aplican proporcionalmente al factor de carga, como $\lambda\,g$, igual que las cargas.

Adaptatividad (`adaptive: true`):

- Si converge en menos de 5 iteraciones, agranda el siguiente paso ($\times 1.5$).
- Si no converge, biseca el paso ($\div 2$). Falla definitivamente si $\Delta\lambda <$ `min_delta_lambda`.

```yaml
solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5
```

Parámetros: `convergence` (bloque con `rtol_force`, `rtol_disp`, `atol_force_factor`, `atol_disp_factor`; ver ADR 0007), `max_iter`, `num_steps`, `adaptive`, `min_delta_lambda`, `linear_algebra`, `freeze_tangent_after_iter`.

Limitación: no puede atravesar puntos límite (snap-through) ni recorrer ramas con derivada negativa de la curva carga-desplazamiento. Para esos casos → `ArcLengthSolver`.

6.3 `ArcLengthSolver` — longitud de arco cilíndrico (Crisfield)

Traza curvas de equilibrio con puntos límite de carga (snap-through, colapso), controlando simultáneamente desplazamientos y factor de carga λ. **No sigue los retrocesos (snap-back) por localización**, en los que el desplazamiento también disminuye tras el pico: para ellos, `IndirectDisplacementSolver`.

Esquema:

- **Predictor tangente**: $\Delta\mathbf{U}_t = \mathbf{K}_t^{-1} \cdot \mathbf{F}_{\text{ext}}^{\text{ref}}$; $\Delta\lambda = \pm\, dl/\|\Delta\mathbf{U}_t\|$. Signo elegido por proyección con el incremento previo.

- **Corrector iterativo**: en cada iteración resuelve dos sistemas ($\mathbf{K}\cdot\Delta\mathbf{U}_R = \mathbf{R}$ y $\mathbf{K}\cdot\Delta\mathbf{U}_t = \mathbf{F}_{\text{ext}}$) y aplica la restricción cuadrática cilíndrica $\|\delta\mathbf{U}\|^2 = dl^2$.
- **Primer paso**: se declara con `initial_dlambda`, como **fracción de la carga de referencia** (0.1 si no se declara, es decir, el 10 % de la carga). El solver lo convierte en longitud de arco con una resolución elástica, $dl_1 = \Delta\lambda_1 \|\mathbf{K}^{-1}\mathbf{F}_{\text{ext}}^{\text{ref}}\|$, así que no hace falta conocer de antemano los desplazamientos del modelo y el valor significa lo mismo en cualquier sistema de unidades. La alternativa `initial_dl` fija la longitud de arco directamente, en unidades de desplazamiento; sólo tiene sentido si se conoce la escala del problema, y no puede declararse junto con `initial_dlambda`.
- **Métrica del arco**: $\|\delta\mathbf{U}\|$ se mide sobre todos los grados de libertad, también los esclavos de las restricciones multipunto y los prescritos. Un desplazamiento prescrito grande frente a la deformación consume casi toda la longitud de arco y el paso se comporta como un control por carga. Un modelo con nodos atados recorre una secuencia de pasos distinta de la del modelo con los nodos fundidos: todos los puntos son de equilibrio, y con plasticidad o daño los resultados difieren dentro del error de discretización de la trayectoria de carga, como al cambiar el tamaño de paso.
- **Auto-ajuste de `dl`**: se agranda en convergencias rápidas (hasta `dl_max_factor` veces la del primer paso), se reduce en lentas, biseca al fallar.
- **Llegada a `max_lambda`**: todos los pasos siguen la curva. Si uno la cruza, se repite desde el estado anterior hasta llegar exactamente a `max_lambda`. El solver nunca impone la carga máxima sin haber recorrido la curva hasta ella; si hay un colapso antes, el trazado se detiene por `max_steps` y lo avisa.

```yaml
solver:
  type: ArcLengthSolver
  max_iter: 15
  max_lambda: 1.0
  initial_dlambda: 0.05      # primer paso: 5 % de la carga de referencia
  max_steps: 200
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5
```

Cuándo usarlo: problemas con softening pronunciado (daño, post-pandeo, snap-through de cúpulas), o cuando `NonlinearSolver` diverge cerca de un punto límite. Más caro por paso (dos `spsolve` por iteración) pero indispensable en estos casos.

Cómo terminó el trazado (ADR 0025). Tras `solve()`, el atributo `termination` dice por qué se detuvo, sin leer el registro:

`termination`	**Significado**
`"max_lambda"`	Llegó a `max_lambda`.
`"control_target"`	Llegó al valor objetivo de la magnitud vigilada (`control_target`).
`"max_steps"`	Agotó `max_steps` antes de terminar. `solve()` devuelve el último paso convergido sin lanzar, pero **no es el análisis pedido**: `completed` es `False` y `SolveResult.converged` también.
`"diverged"`	Un paso no convergió ni reduciendo la longitud al mínimo. `solve()` lanza una `SolverDivergedError` tipada con la causa; `StepConstraintError` si la restricción del paso no tiene solución (raíces imaginarias).

Parar en un desplazamiento. Con `control` (la magnitud vigilada, la misma sintaxis que en `IndirectDisplacementSolver`) y `control_target`, el arco se detiene cuando la magnitud alcanza ese valor:

```yaml
solver:
  type: ArcLengthSolver
  control:
    - {node: 12, dof: ux, coef: 1.0}   # desplazamiento horizontal del nodo 12
  control_target: 0.15                 # se detiene con 150 mm
  max_lambda: 1.0e9                    # holgado: la parada la decide el objetivo
```

En el arco, `control` sólo mide: la restricción del paso sigue siendo la longitud de arco, así que el trazado pasa puntos límite. El paso que cruza el objetivo se repite con llegada exacta: el último punto tiene exactamente el valor pedido. Si `max_lambda` llega antes, se detiene ahí.

`lambda_final`, `steps_done`, `n_bisections` y `U_final` dicen hasta dónde llegó (último paso convergido), también tras una excepción, y `solve()` los reinicia al empezar. `solidum.run` copia el motivo en `SolveResult.termination`. `NonlinearSolver` deja los mismos atributos (`"max_lambda"` al completar la carga, `"diverged"` antes de lanzar).

Referencia: Crisfield, "A fast incremental/iterative solution procedure that handles snap-through" (*Computers & Structures*, 1981).

6.4 Análisis por etapas: gravedad primero, empuje después

Un análisis de empuje lateral aplica primero la gravedad, que se queda, y después empuja con una carga que crece (ADR 0026). Cada etapa es una llamada a `solidum.run` con el mismo `Assembler` —guarda el estado interno de los elementos— y el resultado de la etapa anterior en `previous`:

```
asm = Assembler(dom)
F_g = asm.assemble_self_weight([0.0, -9.81])      # o cargas puntuales de piso
gravedad = solidum.run(dom, assembler=asm, F_applied=F_g,
                       solver=NonlinearSolver(asm, num_steps=4))
empuje = solidum.run(dom, assembler=asm, F_applied=F_lateral, previous=gravedad,
                     solver=IndirectDisplacementSolver(
                         asm, control=[(12, "ux", 1.0)], initial_dl=2e-3,
                         max_lambda=1e9, control_target=0.09))
```

- La etapa parte de los desplazamientos de la anterior y mantiene **constante** su carga aplicada, también la distribuida. El residuo es $\mathbf{F}_c + \lambda\,\mathbf{F} - \mathbf{F}_{\text{int}}$, con $\lambda = 0$ al empezar; `max_lambda` y

`control_target` se refieren a la etapa (`control_target` es la magnitud controlada total, así que continúa desde donde quedó).

- Un desplazamiento impuesto que no cambia entre etapas se queda; uno que se cambia antes de la etapa crece con λ desde el valor que tenía.
- El resultado separa `F_constant` (lo ya aplicado) y `F_variable` (lo que creció, `load_factor`·F); las reacciones son las de la carga total.
- La etapa comprueba que empieza en equilibrio y lo avisa si no. Empezar en $U = 0$ un modelo que conserva el estado plástico de un análisis anterior también se avisa: para continuarlo se usa `previous`; para un análisis nuevo, se construye el modelo de nuevo.
- Lo admiten `NonlinearSolver`, `ArcLengthSolver` e `IndirectDisplacementSolver`; desde Python también directamente, `solver.solve(F, u0=U_anterior, constant_load=F_anterior)`. Desde el YAML, con el bloque `stages` (capítulo «Archivo de Entrada YAML»).
- Las cargas distribuidas de cada etapa (peso propio, fuerzas de cuerpo) llevan sus cargas equivalentes por elemento (`LoadVector`, ADR 0027), así que las fuerzas internas de extremo descuentan todas las que actúan, también combinadas en una misma etapa (sección «Cargas distribuidas y sus cargas equivalentes» del capítulo «Uso Avanzado: API Programática»).

6.4.1 Casos lineales sobre una etapa (ADR 0031)

Un `LinearSolver` con `previous=` no avanza el estado: es una **perturbación lineal** con la rigidez tangente de la etapa, $\mathbf{K}_t(\mathbf{u}_b)\,\Delta\mathbf{U} = \Delta\mathbf{F}$. Sirve para los casos de carga lineales sobre el estado P-Δ de la gravedad:

```
grav = solidum.run(dom, assembler=asm, F_applied=F_g, solver=NonlinearSolver(asm,
    num_steps=4))
viento = solidum.run(dom, assembler=asm, F_applied=F_v, solver=LinearSolver(asm),
    previous=grav)
sismo  = solidum.run(dom, assembler=asm, F_applied=F_s, solver=LinearSolver(asm),
    previous=grav)
```

- El resultado es el estado de la etapa más el incremento lineal; las reacciones y las fuerzas internas también son lineales, así que los casos se pueden **superponer**.
- No cambia el estado interno de los elementos: lo que sigue continúa `grav`. El resultado lleva `perturbation = True` y no se puede usar como `previous` (es un error).
- Con elementos corrotacionales incluye el efecto P-Δ; con materiales que fluyeron, la rigidez de descarga (elástica); con daño, la dañada.

6.5 `IndirectDisplacementSolver` — control indirecto de desplazamiento

Variante de `ArcLengthSolver` que controla, en vez de la longitud de arco de todos los grados de libertad, el incremento de **una magnitud elegida**: el alargamiento de la zona que se ablanda o la apertura de una grieta (CMOD), como se controla en laboratorio un ensayo de fractura. La carga λ queda

libre y puede bajar, así que sigue los **retrocesos** (snap-back) por localización, que `ArcLengthSolver` no sigue (tampoco el arc-length por disipación del módulo de usuario `dissipation_control`).

$$\mathbf{c}^\top \Delta\mathbf{U} = \Delta l$$

- `control` (obligatorio): la magnitud, como lista de términos `{node, dof, coef}`. El alargamiento entre los nodos 6 y 7 en x es `[{node: 7, dof: ux, coef: 1.0}, {node: 6, dof: ux, coef: -1.0}]`. Los grados de libertad no pueden estar prescritos: un apoyo, o el esclavo de una restricción multipunto cuyos maestros están todos apoyados, tiene un movimiento que no depende del equilibrio. El esclavo de una restricción multipunto con maestros libres sí vale, porque sus maestros fijan su incremento. La magnitud tiene que crecer al cargar; si no, el solver lo explica con un error.
- **Primer paso**: `initial_dlambda`, fracción de la carga de referencia, como en `ArcLengthSolver`.
- **Sin crecimiento del paso por omisión** (`dl_max_factor: 1.0`). Con ablandamiento local, un paso que sobrepasa el pico más que la imperfección que localiza el daño puede converger a otro equilibrio, con más zonas ablandadas de las que el camino físico produciría. Esto le pasa a cualquier solver, también al control por desplazamiento; la defensa es un paso pequeño cerca del pico.
- `control_target` (opcional): valor de la magnitud controlada en el que se detiene el trazado, en sus unidades. El paso que lo rebasaría se recorta a lo que falta, así que el último punto queda **exactamente** en el objetivo, con cualquier tamaño de paso. Es la parada natural de un análisis de empuje lateral o de un ensayo controlado: en la meseta plástica la carga casi no cambia y `max_lambda` no sirve para detenerse. Con un objetivo conviene un `max_lambda` holgado; el trazado se detiene en lo que llegue primero.

```yaml
solver:
  type: IndirectDisplacementSolver
  initial_dl: 2.0e-3          # 2 mm de la magnitud controlada por paso
  max_lambda: 1.0e9           # sin límite de carga
  max_steps: 200
  control_target: 0.09        # se detiene con el desplazamiento de control = 90 mm
  control:
    - {node: 12, dof: ux}
```

```yaml
solver:
  type: IndirectDisplacementSolver
  initial_dlambda: 0.02
  max_lambda: 1.2
  max_steps: 300
  control:
    - {node: 7, dof: ux, coef: 1.0}
    - {node: 6, dof: ux, coef: -1.0}
```

Referencia: R. de Borst, "Computation of post-bifurcation and post-failure behavior of strain-softening solids" (*Computers & Structures*, 1987).

6.6 Módulo de usuario `dissipation_control`: arc-length por disipación de energía

No es un solver estándar según el criterio de admisión del proyecto: ningún código general establecido implementa esta restricción de forma nativa. Por eso no forma parte del programa principal: vive en el **módulo de usuario** `dissipation_control` (`solidum/user/dissipation_control/`, ADR 0020). `import solidum` no lo carga; el YAML que lo use debe declararlo:

```
user_modules: [dissipation_control]
```

Sin esa línea, la lectura se detiene con un error que nombra el solver desconocido y lista los módulos de usuario disponibles. En Python, `solidum.load_user_module("dissipation_control")` o `from solidum.user.dissipation_control import DissipationArcLengthSolver`.

6.6.1 `DissipationArcLengthSolver` — arc-length por disipación de energía

Variante de `ArcLengthSolver` que controla la **disipación incremental de energía** por paso en lugar de la longitud de arco euclídea. Es la herramienta indicada cuando la curva $\mathbf{U}$–λ tiene tramos casi verticales, donde la restricción cilíndrica se vuelve mal condicionada.

Diferencia esencial: la restricción de paso

$$g(\Delta\mathbf{U}, \Delta\lambda) = \tfrac{1}{2}\left(\lambda_n \mathbf{F}\cdot\Delta\mathbf{U} - \Delta\lambda\,\mathbf{F}\cdot\mathbf{U}_n\right) = \tau$$

es **lineal** en $(\Delta\mathbf{U}, \Delta\lambda)$, frente a la cuadrática del cilíndrico. El corrector tiene por tanto una sola raíz: desaparecen la selección de raíz por menor ángulo y la patología de raíces imaginarias.

- **Switching automático**: arranca en modo cilíndrico (la restricción por disipación es idénticamente nula mientras $\lambda_n = \mathbf{U}_n = 0$) y conmuta a disipación en cuanto detecta disipación neta sobre el umbral. La conmutación es automática en ambos sentidos.
- **Parámetros**: todos los de `ArcLengthSolver`, más `initial_tau` (obligatorio) y la familia `tau_grow_factor`, `tau_max_factor`, `tau_shrink_factor`, `tau_grow_iter_threshold`, `tau_shrink_iter_threshold`, `dissipation_threshold`.
- **Cuándo usarlo**: softening de daño continuo (1D/2D) con rama post-pico pronunciada, donde el cilíndrico converge mal.
- **Limitación medida**: tampoco sigue el retroceso de una barra con daño localizado. Su predictor degenera en el régimen de descarga del material y el paso que cruza el pico puede saltar a otro equilibrio (deuda #27). Para retrocesos, `IndirectDisplacementSolver`.
- **Limitación conocida**: **no** resuelve el caso de discontinuidad embebida (`CST_Embedded2D`, módulo de usuario `discontinuities`) con penalty cohesivo rígido. La activación discreta del criterio de Rankine produce un salto en las fuerzas internas que la regla de signo del predictor (producto escalar con el paso anterior) no maneja. Es una limitación documentada del solver, no un error de configuración. El diagnóstico es anterior a la corrección de la tangente cohesiva del 2026-09-23 y no se ha vuelto a comprobar; con desplazamiento impuesto, `NonlinearSolver` sí recorre hoy esa rama. Un modelo que combine los dos módulos los declara juntos: `user_modules: [discontinuities, dissipation_control]`.

- **Puntos abiertos**: la spec del solver registra tres cuestiones físicas pendientes de decisión (el signo del predictor en modo de disipación, el piso de la escala del umbral de conmutación y el alcance en plasticidad). Mientras no se cierren, conviene limitar su uso al daño continuo validado.

```yaml
user_modules: [dissipation_control]
solver:
  type: DissipationArcLengthSolver
  max_lambda: 1.0
  initial_dlambda: 0.05
  initial_tau: 1.0e-4
  max_steps: 300
```

Referencia: Gutiérrez, "Energy release control for numerical simulations of failure in quasi-brittle solids" (*Communications in Numerical Methods in Engineering*, 2004); switching a la Verhoosel et al. (2009).

La formulación completa está en la spec del solver, en el apéndice «Módulos de usuario» del Manual de Referencia.

Capítulo 7

Análisis Dinámico

Solidum FEM expone el subsistema modal/dinámico/espectral **completo** (ADR 0009 cerrado en su totalidad 2026-05-18). Cubre los siete regímenes canónicos de la mecánica computacional clásica:

1. **Modal**: autovalores generalizados $\mathbf{K}\boldsymbol{\phi} = \omega^2 \mathbf{M}\boldsymbol{\phi}$ (`ModalSolver`).
2. **Mass lumping**: matriz de masa diagonal vía HRZ canónico (sólidos y marcos Euler-Bernoulli) o nodal directo (barras, cables y `Frame2DTimoshenko`). Disponible en todos los elementos vía `compute_mass_matrix(lumping="lumped")`.
3. **Transitorio implícito lineal**: Newmark-β (`NewmarkSolver`) y variante HHT-α con disipación numérica (`HHTSolver`).
4. **Transitorio implícito no lineal**: Newton-Newmark (`NewtonNewmarkSolver`) y Newton-HHT (`NewtonHHTSolver`).
5. **Transitorio explícito**: diferencias centradas leapfrog (`CentralDifferenceSolver`).
6. **Frecuencia**: respuesta forzada armónica (`HarmonicSolver`).
7. **Espectral / sísmico**: combinación modal SRSS/CQC contra espectros normativos (`ResponseSpectrumSolver`).

Todos los solvers consumen la masa global que cada elemento devuelve por `compute_mass_matrix(lumping)` con `lumping` ∈ `{"consistent", "lumped"}` y la densidad declarada en el material (`density`; obligatoria para cualquier análisis dinámico, ADR 0008), más las **masas concentradas en nudos** (`nodal_masses` en el YAML, `Domain.add_nodal_mass` en Python, ADR 0024), que se suman a su diagonal con los dos esquemas.

Restricciones multipunto. Los ocho solvers admiten las restricciones lineales `linear_constraints` ($u_s = g_s + \sum_i \alpha_{si}\, u_{m_i}$, ADR 0004) por el mismo operador de eliminación directa que los apoyos: $\mathbf{M}_{\text{red}} = \mathbf{T}^\top \mathbf{M}\, \mathbf{T}$, $\mathbf{C}_{\text{red}} = \mathbf{T}^\top \mathbf{C}\, \mathbf{T}$, $\mathbf{K}_{\text{red}} = \mathbf{T}^\top \mathbf{K}\, \mathbf{T}$, y los resultados (historias, modos, amplitudes armónicas, respuesta espectral) se expanden con $\mathbf{T}$. En el armónico y el espectral el término g no interviene, como el valor de un apoyo. Dos precisiones:

- En los transitorios, de `u0` y `u0_dot` sólo cuentan las componentes libres: los DOFs esclavos se reconstruyen como $\mathbf{T}\,\mathbf{u}_0[\text{libres}] + \mathbf{g}$ y $\mathbf{T}\,\dot{\mathbf{u}}_0[\text{libres}]$. Si el valor dado en un esclavo incumple su

restricción, se descarta con un aviso en el log y la respuesta depende de qué DOF se declaró esclavo; `ModalResult.free_vibration`, que proyecta en norma $\mathbf{M}$, daría otra. Conviene dar condiciones iniciales coherentes con las restricciones.

- `CentralDifferenceSolver` exige $\mathbf{T}^\top\mathbf{M}\,\mathbf{T}$ diagonal: admite un solo maestro por esclavo (atado de nodos, periodicidad, apoyo oblicuo) y rechaza una restricción con varios maestros cuyo esclavo tiene masa (nodo colgante), con un mensaje que nombra los DOFs.

7.1 `ModalSolver` — análisis modal por autovalores generalizados

Resuelve el problema generalizado $\mathbf{K}\boldsymbol{\phi} = \omega^2\mathbf{M}\boldsymbol{\phi}$ con $\mathbf{K}$ evaluada en $\mathbf{u} = 0$ (régimen lineal) y $\mathbf{M}$ ensamblada por cuadratura consistente sobre todos los elementos. La capa algebraica (`EigenSolver`) envuelve `scipy.sparse.linalg.eigsh` con *shift-invert* en σ (ARPACK Lanczos), de modo que basta una sola factorización de $\mathbf{K} - \sigma\mathbf{M}$ para extraer las n frecuencias más bajas.

Parámetros: `n_modes` (obligatorio), `sigma` (default 0.0; `sigma=0` extrae las frecuencias más bajas), `which` (default `"LM"`), `tolerance` (default `1.0e-9`), `lumping` (default `"consistent"`; `"lumped"` para la masa concentrada, ver *Mass lumping* al final del capítulo).

Salida: instancia de `ModalResult` con `frequencies_rad`, `frequencies_hz`, `periods`, `modes` (M-ortonormales). El método `ModalResult.free_vibration(M, u0, u0_dot, t)` reconstruye analíticamente la respuesta libre no amortiguada por superposición modal.

Participación modal: para cada traslación del modelo, `participation_factors["ux"]` da los factores $\Gamma_n = \boldsymbol{\phi}_n^\top\mathbf{M}\,\mathbf{r}$ y `effective_masses["ux"]` las masas efectivas Γ_n^2 (en unidades de masa), con $\mathbf{r}$ la traslación rígida unitaria en esa dirección; igual con `"uy"` y, en 3D, `"uz"`. `condensed_dofs` lista los grados de libertad sin masa condensados, como `(nodo, dof)`. Para un giro del modelo alrededor de un punto (torsión de un edificio) o una dirección oblicua:

```
from solidum.math.modal_response import participation_factors
from solidum.math.rigid_body import rigid_body_vector

r = rigid_body_vector(dom, "rz", about=(x_c, y_c, z_c))   # giro unitario alrededor de
    z
M = asm.assemble_mass_matrix()
gamma = participation_factors(res.modes, M, r)              # masas efectivas: gamma**2
```

Masa efectiva acumulada (el criterio del 90 %): `res.cumulative_effective_mass_ratio("ux")` da $\sum_{n\le k} M_n^*/M_{\text{part}}$ para cada modo, y el registro escribe el porcentaje alcanzado. La referencia es la masa que puede participar, $M_{\text{part}} = \mathbf{b}^\top\mathbf{M}_{LL}^{-1}\mathbf{b}$ con $\mathbf{b} = \mathbf{T}^\top\mathbf{M}\mathbf{r}$ (`res.participating_mass["ux"]`), la suma de las masas efectivas de todos los modos: la razón llega a 1 con todos ellos. Con masa concentrada es la masa de los nudos libres; la de los apoyos no participa en ningún modo y no cuenta (ADR 0009 §8). `res.total_mass["ux"]` es la masa total del modelo, apoyos incluidos, para revisar la entrada.

```
nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [1.0, 0.0]}

materials:
  - {id: 1, type: Elastic1D, E: 200.0e9, density: 7850.0}
```

```yaml
elements:
  - {id: 1, type: Truss2D, material: 1, A: 0.01, nodes: [1, 2]}

boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0}
  - {node_id: 2, uy: 0.0}

solver:
  type: ModalSolver
  n_modes: 3
  sigma: 0.0
```

Nota

La densidad es *obligatoria* para todo material involucrado en análisis modal o transitorio. La omisión lanza `ValueError` (ADR 0008) listando los materiales afectados por nombre antes de iniciar el cálculo.

7.1.1 Modal sobre el estado de una etapa (ADR 0031)

Después de la gravedad o de un empuje, los periodos dependen del estado: la carga axial reduce la rigidez lateral (P-Δ) y el agrietamiento la reduce aún más. `run_modal` con `previous=` usa la **rigidez tangente del estado** de esa etapa:

```
grav = solidum.run(dom, assembler=asm, solver=NonlinearSolver(asm, num_steps=4),
    F_applied=F_g)
modos = solidum.run_modal(dom, assembler=asm, n_modes=6, previous=grav)   # mismo
    assembler
```

- Incluye la rigidez geométrica de los elementos corrotacionales (`Frame2DEulerCorot…`), la rigidez dañada del daño y del concreto agrietado y, en la plasticidad, la de descarga, que es elástica (como la perturbación lineal de Abaqus). Con elementos de cinemática lineal (`Frame2DEuler`) no hay efecto P-Δ.
- No cambia el estado: lo que sigue continúa la etapa anterior.
- Un autovalor negativo indica que el estado es inestable en ese modo (por ejemplo, una carga axial por encima de la de pandeo): se reporta con su signo en `eigenvalues`, con frecuencia y periodo `NaN`, en `unstable_modes`, y con un aviso.
- `run_harmonic` y `run_response_spectrum` aceptan `previous=` con la misma regla; el espectral se rechaza si hay modos inestables.
- Hace falta el `assembler` de la etapa (guarda el estado interno de los elementos).

7.1.2 Revisar las masas: masa total y centro de masa

Antes de un análisis dinámico conviene revisar la masa que entra al modelo (densidades y masas de nudo):

```
from solidum.math.mass_properties import mass_properties

props = mass_properties(asm)                # o mass_properties(asm, lumping="lumped")
props.total_mass                            # {"ux": m, "uy": m[, "uz": m]}
props.center                                # centro de masa (None si la masa depende de
    la dirección)
props.center_of_mass["ux"]                  # línea de acción de la inercia ante una
    aceleración en x
```

- La masa en cada dirección es $m_d = \mathbf{r}_d^\top \mathbf{M}\, \mathbf{r}_d$, con $\mathbf{r}_d$ la traslación rígida unitaria; incluye la masa de los nudos con apoyo.
- `center_of_mass["ux"]` es un punto de la línea de acción de la resultante de las fuerzas de inercia ante una aceleración rígida en x: da su altura y su posición en planta (la excentricidad respecto del centro de rigidez). La coordenada a lo largo de x no la afecta y queda en `NaN`. Sale de los momentos de esas fuerzas con los giros rígidos del modelo, así que es exacta con masa consistente o concentrada, también en barras inclinadas.
- Si la masa es la misma en todas las direcciones, `center` es el centro de masa. Una masa de nudo distinta por dirección (sólo en `ux`, por ejemplo) hace que cada dirección tenga su centro, y `center` es `None`.
- Las masas de nudo se exportan al VTK (`Nodal_Mass`, `Nodal_Inertia`).

7.1.3 Masas de nudo y grados de libertad sin masa (ADR 0024)

En un edificio la masa de cada nivel se concentra en sus nudos y las columnas y vigas pueden no aportar masa propia (`density: 0.0`). Entonces sólo los grados de libertad de los nudos de piso tienen masa: los nudos intermedios y los giros, no. La matriz de masa es semidefinida y el problema tiene tantas frecuencias finitas como grados de libertad con masa.

`ModalSolver` condensa estáticamente los grados de libertad sin masa (diagonal de $\mathbf{M}$ nula). Con m los que tienen masa y 0 los demás, la ecuación de los grados de libertad sin masa no tiene inercia, y entonces

$$\boldsymbol{\phi}_0 = -\mathbf{K}_{00}^{-1}\mathbf{K}_{0m}\,\boldsymbol{\phi}_m, \qquad \left(\mathbf{K}_{mm} - \mathbf{K}_{m0}\mathbf{K}_{00}^{-1}\mathbf{K}_{0m}\right)\boldsymbol{\phi}_m = \omega^2\,\mathbf{M}_{mm}\,\boldsymbol{\phi}_m.$$

La condensación es **exacta**, no una aproximación: da las mismas frecuencias que el problema completo. Los modos se devuelven completos, también en los grados de libertad sin masa, y $\mathbf{M}$-ortonormales. El registro lo anota en una línea.

```
nodal_masses:
  - {node_id: 3, ux: 40.0e3}
  - {node_id: 4, ux: 40.0e3}

solver:
  type: ModalSolver
  n_modes: 2        # no más que grados de libertad con masa
```

`n_modes` no puede pasar del número de grados de libertad con masa. Si se piden más, o ninguno tiene masa, o los grados de libertad sin masa forman un mecanismo (pueden moverse sin rigidez

ni inercia), el análisis se detiene con `MasslessDofsError`. El mensaje dice la causa y nombra los grados de libertad sin masa como `nodo <id> (<dof>)`. `ResponseSpectrumSolver` hereda todo esto: con masas de piso, `total_mass` es la suma de las masas de piso.

En los **transitorios implícitos** (Newmark, HHT y sus variantes de Newton) los grados de libertad sin masa también se admiten. La integración no necesita $\mathbf{M}$ definida; sólo la aceleración inicial queda indeterminada en ellos, y se toma **consistente**: $\ddot{\mathbf{u}}_m = \mathbf{M}_{mm}^{-1}\mathbf{r}_m$ en los que tienen masa y $\ddot{\mathbf{u}}_0 = -\mathbf{K}_{00}^{-1}\mathbf{K}_{0m}\ddot{\mathbf{u}}_m$ en los demás, que siguen de forma cuasiestática a los primeros (ADR 0024, §6 bis). Así las aceleraciones reportadas son correctas en todo el modelo, también con una carga aplicada desde $t = 0$. Si la carga inicial actúa directamente sobre un nudo sin masa, Solidum avisa: ese nudo no tiene inercia y se ajusta en el primer paso. `CentralDifferenceSolver`, por ser explícito, exige masa en todos.

7.2 `NewmarkSolver` — transitorio lineal Newmark-β

Integra en el tiempo $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(t)$ por la familia Newmark-β. Con $(\beta, \gamma) = (1/4, 1/2)$ — *average acceleration*, default — es *incondicionalmente estable*, sin amortiguamiento numérico, con error $\mathcal{O}(\Delta t^2)$.

Esquema operativo:

- Predictor: $\tilde{\mathbf{u}} = \mathbf{u}_n + \Delta t\,\dot{\mathbf{u}}_n + (\Delta t^2/2)(1-2\beta)\ddot{\mathbf{u}}_n$ y análogo para $\tilde{\dot{\mathbf{u}}}$.
- Sistema efectivo en aceleración: $\mathbf{A}_{\text{eff}}\ddot{\mathbf{u}}_{n+1} = \mathbf{F}_{n+1} - \mathbf{C}\tilde{\dot{\mathbf{u}}} - \mathbf{K}\tilde{\mathbf{u}}$ con $\mathbf{A}_{\text{eff}} = \mathbf{M} + \gamma\Delta t\,\mathbf{C} + \beta\Delta t^2\,\mathbf{K}$. Factorización *única* al inicio del análisis y reusada en todos los pasos (`FactorizedSolver`, ADR 0003 fase 2).
- Corrector: $\mathbf{u}_{n+1} = \tilde{\mathbf{u}} + \beta\Delta t^2\,\ddot{\mathbf{u}}_{n+1}$, $\dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}} + \gamma\Delta t\,\ddot{\mathbf{u}}_{n+1}$.

Amortiguamiento Rayleigh $\mathbf{C} = \alpha\mathbf{M} + \beta\mathbf{K}$, declarado de dos formas:

- Directa: `rayleigh: {alpha: 0.1, beta: 1.0e-4}`.
- Calibración modal: `rayleigh: {xi1: 0.02, omega1: 50.0, xi2: 0.02, omega2: 200.0}` — Solidum resuelve los (α, β) que reproducen los amortiguamientos ξ_1, ξ_2 a las frecuencias ω_1, ω_2.

Parámetros: `t_end`, `dt` (obligatorios); `beta` (default 0.25), `gamma` (default 0.5), `rayleigh` (opcional), `u0`, `u0_dot` (condiciones iniciales, default ceros), `F_func` (callback Python $t \to \mathbf{F}(t)$; default vibración libre). Desde YAML, las cargas del archivo (puntuales + peso propio / fuerza de cuerpo) se inyectan automáticamente como escalón constante en el tiempo; para una historia $\mathbf{F}(t)$ arbitraria, pasar `F_func` desde Python.

Salida: `TransientResult` con `t_history`, `u_history`, `udot_history`, `uddot_history` (forma $(n_{\text{dof}}, n_{\text{steps}}+1)$) y los $\alpha_{\text{Rayleigh}}, \beta_{\text{Rayleigh}}$ efectivos. `result.internal_forces_history(domain)` da las fuerzas internas (N, V, M, T) de los elementos estructurales 1D en cada paso: con `record_internal_forces: true` (parámetro común a los cinco solvers transitorios, también desde YAML) se registran **durante** el análisis con el estado interno de cada paso y quedan en `result.element_forces_history`; sin él se reconstruyen a posteriori desde `u_history` con el estado final, lo que sólo es exacto sin historia (materiales elásticos, elementos geométricamente lineales) y avisa por log en caso contrario.

```yaml
solver:
  type: NewmarkSolver
  t_end: 2.0
  dt: 0.005
  beta: 0.25
  gamma: 0.5
  rayleigh:
    xi1: 0.02
    omega1: 50.0
    xi2: 0.02
    omega2: 200.0
```

Limitaciones: lineal ($\mathbf{K}, \mathbf{M}$ constantes; para no-linealidad material o geométrica → `NewtonNewmarkSolver`). Apoyos Dirichlet constantes en el tiempo. Paso Δt fijo (el paso adaptativo es de las variantes no lineales: sección «Paso adaptativo en los transitorios no lineales»). La disipación numérica controlada está en `HHTSolver` (sección «`HHTSolver` y `NewtonHHTSolver`»); generalized-α no está incluido.

7.3 `NewtonNewmarkSolver` — transitorio no lineal Newton-Newmark

Subclase de `NewmarkSolver` (Reglas §4 — variante de componente existente) que añade un bucle Newton-Raphson dentro de cada paso temporal sobre el residuo dinámico:

$$\mathbf{R} = \mathbf{F}_{\text{ext}}(t) - \mathbf{F}_{\text{int}}(\mathbf{u}) - \mathbf{C}\dot{\mathbf{u}} - \mathbf{M}\ddot{\mathbf{u}}, \quad \mathbf{J} = \mathbf{M} + \gamma\Delta t\,\mathbf{C} + \beta\Delta t^2\,\mathbf{K}_t$$

Convergencia dual (ADR 0007): mismas tolerancias `rtol_force` / `rtol_disp` y factores `atol_*_factor` que los solvers no lineales estáticos. Tras converger cada paso se invoca `assembler.commit_all_states()` (trial → committed en todos los elementos); si agota `max_iter`, el paso se repite subdividido desde el último estado convergido (sección «Paso adaptativo en los transitorios no lineales») y, si ni así converge, el análisis se detiene con `SolverDivergedError` y un resultado parcial hasta el último estado convergido.

Amortiguamiento Rayleigh constante en el tiempo, calibrado con la rigidez elástica de referencia $\mathbf{K}_0$ al inicio del análisis. Convención estándar (Abaqus, ANSYS, OpenSees) que evita acoplamiento ad-hoc entre disipación viscosa y plástica.

Newton modificado opcional (`freeze_tangent_after_iter`, ADR 0003 fase 2): factoriza fresco las primeras N iter y reusa la factorización en las siguientes para abaratar pasos largos.

Recuperación del caso lineal: con materiales sin historia o no plastificados, el residuo se anula en 1 iter y el resultado coincide *a paridad de bits* con `NewmarkSolver`. Validado en tests.

Parámetros: heredados de `NewmarkSolver` más `max_iter` (default 20), bloque `convergence` (`rtol_force`, `rtol_disp`, `atol_force_factor`, `atol_disp_factor`; ADR 0007), `freeze_tangent_after_iter` (default `None`), `line_search` (búsqueda lineal en el Newton de cada paso, ADR 0011; default `false`) y `max_subdivisions` (niveles de subdivisión del paso, default 6; `0` la desactiva). El despacho YAML es automático: como `NewtonNewmarkSolver` es subclase de `NewmarkSolver`, `solidum.run_yaml` lo detecta vía `isinstance` y enruta a `run_transient`.

```yaml
materials:
  - {id: 1, type: VonMises2D, E: 200.0e9, nu: 0.3, sigma_y: 250.0e6, H: 2.0e9,
     hypothesis: plane_strain, density: 7850.0}
```

```yaml
solver:
  type: NewtonNewmarkSolver
  t_end: 0.5
  dt: 0.001
  beta: 0.25
  gamma: 0.5
  rayleigh:
    xi1: 0.02
    omega1: 100.0
    xi2: 0.02
    omega2: 500.0
  max_iter: 15
  convergence:
    rtol_force: 1.0e-6
    rtol_disp: 1.0e-6
```

Cuándo usarlo: respuesta dinámica de estructuras con plasticidad transitoria (sísmica con disipación plástica, impacto en hormigón friccional, fatiga de bajo ciclo), vibraciones de marcos elastoplásticos. Materiales con historia disponibles en dinámica: `Elastoplastic1D`, `VonMises2D` (*plane strain* y *plane stress*), `DruckerPrager2D`, `IsotropicDamage1D/2D`.

Limitaciones: apoyos Dirichlet constantes en el tiempo, como `NewmarkSolver`; el paso es el pedido salvo donde el Newton no converge, que se subdivide. No resuelve *snap-back* dinámico con softening severo — combinar con `ArcLengthSolver` si emerge.

7.4 `HHTSolver` y `NewtonHHTSolver` — variantes Hilber-Hughes-Taylor

Variantes de Newmark con **disipación numérica controlada** en altas frecuencias (Hilber, Hughes & Taylor 1977). Útiles cuando modos altos espurios contaminan la respuesta y conviene atenuarlos preservando segundo orden y estabilidad incondicional.

El parámetro $\alpha \in [-1/3, 0]$ controla la disipación; los coeficientes (β, γ) se auto-derivan canónicamente:

$$\beta = \frac{(1-\alpha)^2}{4}, \quad \gamma = \frac{1-2\alpha}{2}$$

con radio espectral $\rho_\infty = (1+\alpha)/(1-\alpha)$. Recuperación exacta a Newmark cuando $\alpha = 0$.

Esquema operativo: idéntico a Newmark pero con factor $(1+\alpha)$ en los términos elásticos y de amortiguamiento del residuo dinámico, y $-\alpha$ multiplicando los del paso anterior. Variantes lineal (`HHTSolver`) y no lineal (`NewtonHHTSolver`) heredan toda la maquinaria de `NewmarkSolver` / `NewtonNewmarkSolver` (Reglas §4 sobre variantes).

```yaml
solver:
  type: HHTSolver           # o NewtonHHTSolver para materiales con historia
  t_end: 2.0
  dt: 0.005
  alpha: -0.05              # default; disipación ligera de altas frecuencias
```

7.5 `CentralDifferenceSolver` — explícito por diferencias centradas (ADR 0009 fase 5)

Integración **explícita** de segundo orden por leapfrog Belytschko-Liu-Moran. Sin sistema lineal en cada paso — la única "inversión" es $\mathbf{M}^{-1}$, trivial con masa lumped diagonal. Apropiado para wave propagation, impacto, dinámica de respuesta rápida.

Esquema operativo (forma predictor-corrector):

$$\mathbf{a}_0 = \mathbf{M}^{-1}(\mathbf{F}(0) - \mathbf{F}_{\text{int}}(\mathbf{u}_0) - \mathbf{C}\mathbf{v}_0); \quad \mathbf{v}_{1/2} = \mathbf{v}_0 + \tfrac{\Delta t}{2}\mathbf{a}_0$$

Paso $n \to n+1$:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t\, \mathbf{v}_{n+1/2}; \quad \mathbf{a}_{n+1} = \mathbf{M}^{-1}(\mathbf{F}_{n+1} - \mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) - \mathbf{C}\mathbf{v}_{n+1/2})$$

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \tfrac{\Delta t}{2}\mathbf{a}_{n+1}$$

Una sola clase cubre lineal y no lineal con parámetro `nonlinear`: con `False` usa $\mathbf{F}_{\text{int}} = \mathbf{K}\mathbf{u}$ con $\mathbf{K}$ constante; con `True` recalcula $\mathbf{F}_{\text{int}}$ cada paso vía `Assembler.assemble_non_linear_system`.

Condición de estabilidad CFL: $\Delta t < 2/\omega_{\text{máx}}$. Para barras: $\Delta t < L_{el}\sqrt{\rho/E}$. El solver detecta la divergencia exponencial a posteriori y, si la condición se viola, lanza `TimeIntegrationInstabilityError` (una `SolverDivergedError`, y por tanto `RuntimeError`) con el historial hasta el último paso estable en `partial_result`.

Requisitos: `lumping="lumped"` obligatorio (`"consistent"` rechazado con `ValueError` — el coste de invertir $\mathbf{M}$ consistente cada paso anula la ventaja del explícito). La masa reducida $\mathbf{T}^\top\mathbf{M}\,\mathbf{T}$ debe ser estrictamente diagonal (umbral relativo a su mayor término diagonal, independiente de las unidades); se rechazan con `ValueError` dos casos: Frame3D con eje oblicuo a los ejes globales (la rotación rompe la diagonalidad estricta — limitación documentada estándar, Cook-Malkus-Plesha §11.4) y una restricción multipunto con varios maestros cuyo esclavo tiene masa (el nodo colgante $u_h = \frac{1}{2}u_a + \frac{1}{2}u_b$ acopla a sus maestros; usar `NewmarkSolver` o `HHTSolver` con `lumping: lumped`).

```yaml
solver:
  type: CentralDifferenceSolver
  t_end: 2.5
  dt: 0.005                  # debe respetar CFL: Δt < L_el·√ρ(/E)
  lumping: lumped            # obligatorio
  nonlinear: false           # true para materiales con historia
  u0: [0.0, 0.0, 1.0, 0.0]
```

7.6 Cómo termina un análisis en el tiempo (ADR 0025)

Todos los integradores (`NewmarkSolver`, `HHTSolver`, sus variantes de Newton, `CentralDifferenceSolver` y el `ThetaMethodSolver` térmico) dejan tras `solve()`:

Atributo	Significado
`termination`	`"t_end"` si llegó al instante final; `"diverged"` si se detuvo
`completed`	`True` si llegó al instante final
`steps_done`, `t_final`, `U_final`	último paso convergido, su instante y su campo
`partial_result`	si se detuvo, el resultado hasta el último paso convergido

Un análisis que se detiene registra una línea `ERROR` y lanza una `SolverDivergedError` tipada: la clasificación del Newton (ADR 0011) en los no lineales, o `TimeIntegrationInstabilityError` si el paso supera el límite de estabilidad (diferencia central, $\theta < 1/2$). El historial calculado no se pierde:

```
try:
    res = solver.solve()
except SolverDivergedError as exc:
    res = exc.partial_result          # hasta el último paso convergido
    print(solver.t_final, res.termination)   # ..., 'diverged'
```

Fuerzas internas con carga distribuida. Con `record_internal_forces=True`, el historial descuenta en cada paso las cargas equivalentes de la carga del instante si `F_func` combina las cargas que devuelven `assemble_self_weight` / `assemble_body_load` (ADR 0027): `F_func=lambda t: F_peso + a(t) * F_viento`.

Grados de libertad sin masa. Los integradores implícitos los admiten con la aceleración inicial consistente (ADR 0024 §6 bis); sólo un mecanismo sin masa (grados de libertad sin masa ni rigidez) da `MasslessDofsError`. La diferencia central exige masa en todos: `MasslessDofsError` los nombra.

7.7 Reacciones y cortante basal (ADR 0030)

Los análisis dinámicos dan las reacciones de los apoyos: la ecuación de movimiento en esas filas,

$$\mathbf{R} = \mathbf{F}_{\text{int}}(\mathbf{u}) + \mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} - \mathbf{F},$$

con la fuerza de los elementos, la inercia de la masa del apoyo (o acoplada a él con masa consistente) y el amortiguamiento. Así cumplen el equilibrio de la estructura entera. La parte de amortiguamiento se da aparte, para quitarla si se quiere la reacción sin Rayleigh (en análisis inelásticos con Rayleigh proporcional a la rigidez inicial puede ser grande).

```
res = solidum.run_transient(dom, solver=NewmarkSolver(asm, t_end=10.0, dt=0.01, F_func=
    F))
V = res.total_reaction("ux")                          # cortante basal en cada instante
V_sin_rayleigh = res.total_reaction("ux", damping=False)
res.reactions_by_node(step=-1)                        # {nodo: {dof: valor}} en el último
    instante
res.R_history, res.R_damping_history, res.reaction_labels   # (n_apoyos, n_t) y ((nodo,
     dof), …)
```

- **Armónico**: `R_complex` (amplitudes complejas, $(\mathbf{K}-\omega^2\mathbf{M}+i\omega\mathbf{C})\hat{\mathbf{u}}-\hat{\mathbf{F}}$) y `R_damping_complex`; `total_reaction("ux")` para cada frecuencia.

- **Espectral**: `R_per_mode` (la reacción de cada modo) y `R_combined` (cada apoyo, combinado). El cortante basal es `total_reaction("ux")`, que suma las reacciones en cada modo **antes** de combinar; sumar las reacciones ya combinadas daría una cota superior, no el cortante basal.

7.8 Paso adaptativo en los transitorios no lineales

En `NewtonNewmarkSolver` y `NewtonHHTSolver`, un paso cuyo Newton no converge no detiene el análisis: se repite desde el último estado convergido en dos mitades, y así sucesivamente hasta `max_subdivisions` niveles (6 por omisión, es decir, hasta $\Delta t/64$). Superado el paso difícil se vuelve al Δt pedido.

- El historial queda en los instantes que se pidieron: los subpasos son internos, y la carga se evalúa en sus instantes.
- Cada subdivisión se avisa en el registro; `solver.n_subdivisions` las cuenta.
- Si ni con el mínimo converge, el análisis se detiene como siempre (`SolverDivergedError`); el resultado parcial termina entonces en el último subpaso convergido, que es el estado de los elementos, así que se puede continuar desde él con otro Δt (`previous=exc.partial_result`).
- `max_subdivisions: 0` lo desactiva.

Newmark y HHT son métodos de un paso: cambiar Δt entre pasos no les quita la estabilidad incondicional.

7.9 Progreso paso a paso

`run_transient(..., step_callback=f)` y `run_thermal_transient(..., step_callback=f)` llaman `f(step, U, t)` tras cada paso convergido, con el número de paso, el campo global (desplazamientos o temperaturas) y el instante: la misma forma que el `step_callback(step, U, λ)` de los solvers estáticos, con el instante en lugar del factor de carga. Sirve para mostrar el avance, vigilar una magnitud o exportar mientras se integra. `run_yaml(path, step_callback=f)` lo pasa a los análisis estáticos y en el tiempo, también en las etapas; en el modal, el armónico y el espectral, que no tienen pasos, avisa de que lo ignora.

7.10 Continuar un análisis (ADR 0029)

Un análisis en el tiempo puede partir del estado final de otro, con el mismo `assembler` (guarda el estado interno de los elementos):

- **Desde un estático** (`SolveResult`): parte en reposo de sus desplazamientos, en $t = 0$, y su carga se mantiene constante, con sus cargas distribuidas; `F_func` añade la que varía. Es el caso de la gravedad aplicada de forma estática y, después, una excitación dinámica. Si el estado no equilibra esa carga (el estático no convergió, o el modelo cambió), Solidum avisa: la diferencia actuaría como una fuerza súbita.

- **Desde un transitorio**: parte de su estado final (desplazamientos, velocidades y aceleraciones) en su instante final. `F_func` se evalúa en tiempo absoluto y `t_end` es el instante final absoluto. Sirve para cambiar el paso de tiempo o para seguir desde el último paso convergido de un análisis que se detuvo (`exc.partial_result`).

Parar y continuar da lo mismo que no parar.

```
from solidum.math.solvers import NewtonNewmarkSolver

asm = solidum.Assembler(dom)
F_g = asm.assemble_self_weight(np.array([0.0, -9.81]))
r0 = solidum.run(dom, assembler=asm, F_applied=F_g)              # gravedad estática
solver = NewtonNewmarkSolver(asm, t_end=10.0, dt=0.01, F_func=F_dinamica)
r1 = solidum.run_transient(dom, assembler=asm, solver=solver, previous=r0)
```

`u0`, `u0_dot` y `previous` se excluyen. Con amortiguamiento de Rayleigh proporcional a la rigidez, $\mathbf{C}$ se forma con la tangente del instante inicial de cada análisis.

7.11 `HarmonicSolver` — respuesta forzada armónica en frecuencia (ADR 0009 fase 6)

Análisis **lineal en estado estacionario**: dada una excitación armónica $\mathbf{F}(t) = \mathrm{Re}\{\hat{\mathbf{F}}\, e^{i\omega t}\}$, calcula la amplitud compleja $\hat{\mathbf{u}}(\omega)$ de la respuesta estacionaria $\mathbf{u}(t) = \mathrm{Re}\{\hat{\mathbf{u}}(\omega)\, e^{i\omega t}\}$ para un barrido de frecuencias.

Ecuación resuelta por frecuencia:

$$(-\omega^2\mathbf{M} + i\omega\mathbf{C} + \mathbf{K})\,\hat{\mathbf{u}}(\omega) = \hat{\mathbf{F}}$$

Aritmética compleja directa con `scipy.sparse.linalg.spsolve`. Factorización LU compleja por frecuencia (no se cachea — la matriz cambia con ω). Coste dominante: $O(n_\omega \cdot \text{coste_factorizacion})$.

Barrido configurable:

- Vector explícito vía `omega` (lista de rad/s).
- Lineal: `omega_min`, `omega_max`, `n_omega`, `scale: linear`.
- Logarítmico: `scale: log` con `np.geomspace`.

Cargas: las `point_loads` del bloque YAML estándar se inyectan automáticamente como `F_amplitude` real con fase cero. Cargas con fase compleja requieren construcción desde código Python (YAML no soporta tipos complejos nativos).

Salida: `HarmonicResult` con `omega`, `u_complex` (shape $(n_{\text{dof}}, n_\omega)$), `F_complex`, y métodos `.amplitude()` ($|\hat{\mathbf{u}}|$) y `.phase()` ($\arg\hat{\mathbf{u}}$). Propiedad derivada `frequencies_hz`.

```
point_loads:
  - {node_id: 2, ux: 1.0}    # amplitud real (fase 0)

solver:
  type: HarmonicSolver
  omega_min: 1.0
```

```
omega_max: 10.0
n_omega: 91
scale: linear              # o "log" para FRFs sobre rangos amplios
rayleigh:
  alpha: 0.5
  beta: 0.0
```

Cuándo usarlo: funciones de transferencia (FRFs), diagnóstico de resonancias antes de un transitorio caro, respuesta forzada por maquinaria rotante o cargas armónicas descompuestas.

Caveats: cerca de resonancia exacta sin amortiguamiento $\mathbf{Z}(\omega_n)$ es singular y el `spsolve` puede fallar — añadir Rayleigh.

7.12 `ResponseSpectrumSolver` — análisis sísmico por combinación modal (ADR 0009 fase 7)

Análisis **sísmico clásico**: el suelo se caracteriza por un espectro de respuesta (en aceleración $S_a(T)$ o desplazamiento $S_d(T)$ vs período), no por un acelerograma temporal. El solver combina las respuestas máximas modales en una envolvente máxima — pierde la fase temporal entre modos por construcción, sólo conserva amplitudes envolventes.

Esquema operativo:

1. Análisis modal interno vía `ModalSolver(n_modes, sigma, lumping)`.
2. Factores de participación $\Gamma_n = \boldsymbol{\phi}_n^T \mathbf{M}\,\mathbf{r}$ con $\mathbf{r}$ el vector de excitación rígida (dirección sísmica).
3. Respuesta máxima por modo: $\mathbf{u}_n^{\max} = \Gamma_n\,\boldsymbol{\phi}_n\,S_d(\omega_n)$.
4. **Combinación SRSS** (Square Root of Sum of Squares, modos bien separados):

$$u_k^{\max} = \sqrt{\sum_n (u_{k,n}^{\max})^2}$$

1. **Combinación CQC** (Complete Quadratic Combination, Der Kiureghian 1980, para modos cercanos):

$$u_k^{\max} = \sqrt{\sum_i \sum_j \rho_{ij}\,u_{k,i}^{\max}\,u_{k,j}^{\max}}, \quad \rho_{ij} = \frac{8\xi^2(1+r)r^{3/2}}{(1-r^2)^2 + 4\xi^2 r(1+r)^2}$$

con $r = \omega_i/\omega_j$. Cuando $\xi \to 0$ o $r \to 0$: $\rho_{ij} \to \delta_{ij}$ y CQC degenera a SRSS.

Interfaz del espectro:

- Callable directo `spectrum(omega) -> S_d`.
- Dict tabulado: `{type: tabulated, kind: Sd|Sa, periods: [...], values: [...]}`. Interpolación lineal en período; clamp fuera de rango.
- Constante: `{type: constant_acceleration, Sa: ...}`.

Dirección de excitación:

- Vector explícito (`np.ndarray` shape $(n_{\text{dof}},)$).
- Atajo por DOF: `{dof_name: ux}` — el solver construye el vector unitario en ese DOF de todos los nodos.

```yaml
solver:
  type: ResponseSpectrumSolver
  n_modes: 5
  direction:
    dof_name: ux
  spectrum:
    type: tabulated
    kind: Sa                  # aceleración espectral; el solver convierte a Sd
    periods: [0.1, 0.5, 1.0, 2.0, 5.0]
    values:  [10.0, 10.0, 5.0, 1.0, 0.25]
  combination: SRSS           # o "CQC" para modos cercanos
  damping: 0.05               # ξ-  sólo usado por CQC
```

Salida: `ResponseSpectrumResult` con `u_combined` (envolvente, positiva), `u_per_mode` (contribución modal con signo), `participation_factors` (Γ_n), `effective_masses` (Γ_n^2), `frequencies_rad`, `combination`, `damping`, `direction`, `total_mass` (masa total del modelo en la dirección, apoyos incluidos) y `participating_mass` (la que puede participar). Método `.cumulative_effective_mass_ratio()` para verificar que la masa modal acumulada alcanza el objetivo normativo (≥ 0.9): se divide entre `participating_mass`, así que vale 1 con todos los modos (ADR 0009 §8).

Cuándo usarlo: diseño sísmico contra espectros normativos (ASCE 7, Eurocódigo 8, NCh 433, NTC-DS México). Diagnóstico de qué modos dominan la respuesta en una dirección de excitación.

Caveats: precisión depende del número de modos incluidos (verifica `cumulative_effective_mass_ratio` ≥ 0.9). Excitación multi-direccional simultánea (CQC3, regla 100/30/30) requiere combinar varios análisis fuera del solver.

7.13 Mass lumping (ADR 0009 fase 2)

Independiente de cualquier solver: todos los elementos del catálogo soportan `compute_mass_matrix(lumping="lumped")`. El esquema se elige por familia:

- **Sólidos isoparamétricos** (Tri3, Quad4, Tri6, Quad8, Quad9): **HRZ canónico** (Hinton-Rock-Zienkiewicz 1976) centralizado en `solidum.math.mass_lumping.lump_hrz`. Preserva masa total y proporcionalidad diagonal; evita masas negativas en elementos de orden superior.
- **Barras y cables de dos nodos** (Truss, Cable): $\rho AL/2$ por grado de libertad traslacional y nodo.
- **Barras de tres nodos** (`Truss2D3`, `Truss3D3`): $\rho AL \cdot (1/6,\ 1/6,\ 2/3)$ en los extremos y el nodo intermedio, por componente (la cuadratura de Lobatto de 3 puntos y HRZ coinciden).
- **Marcos Euler-Bernoulli** (`Frame2DEuler`, `Frame2DEulerCorot`, `Frame3D`): **HRZ** de la masa consistente — $\rho AL/2$ traslacional y $\rho AL^3/78$ en cada giro de flexión por nodo; en `Frame3D`,

además, $\rho J_p L/2$ en el giro de torsión ($J_p = I_y + I_z$). Como la hipótesis de Euler-Bernoulli desprecia la inercia rotatoria de la sección, ni la masa consistente ni la concentrada dependen de I (salvo la torsión de `Frame3D`).

- `Frame2DTimoshenko`: $\rho AL/2$ traslacional y $\rho IL/2$ rotacional por nodo — conserva la inercia rotatoria, porque en Timoshenko el giro es independiente de la pendiente.

En los marcos 2D el bloque traslacional por nodo es $m\,\mathbf{I}_2$, así que la matriz sigue siendo diagonal tras la rotación local→global.

Diagonalidad en globales: garantizada en todos los elementos excepto Frame3D con eje oblicuo (queda **bloque-diagonal** por nodo — limitación documentada estándar; `CentralDifferenceSolver` rechaza este caso explícitamente).

Capítulo 8

Análisis Térmico

Solidum FEM resuelve **conducción de calor** por la ley de Fourier, en régimen estacionario y transitorio, sobre dominios 2D y 3D. El campo incógnita es un escalar T por nodo.

Alcance actual: conducción pura con condiciones de Dirichlet (temperatura impuesta) y Neumann (flujo prescrito). **No** incluye convección (Robin), radiación, conductividad dependiente de la temperatura $k(T)$, cambio de fase ni **acoplamiento termomecánico** — no existe deformación térmica $\alpha\,\Delta T$: el análisis térmico y el mecánico son hoy independientes.

8.1 Formulación

La ecuación de balance de energía, con $\mathbf{q} = -\mathbf{k}\cdot\nabla T$ (Fourier):

$$\rho c\,\dot{T} = \nabla\cdot(\mathbf{k}\,\nabla T) + Q$$

Discretizada por elementos finitos:

$$\mathbf{C}\,\dot{\mathbf{T}} + \mathbf{K}\,\mathbf{T} = \mathbf{F}$$

donde $\mathbf{K} = \int_\Omega \mathbf{B}^\top \mathbf{k}\,\mathbf{B}\,d\Omega$ es la matriz de **conductividad**, $\mathbf{C} = \int_\Omega \rho c\,\mathbf{N}^\top\mathbf{N}\,d\Omega$ la de **capacidad calorífica** y $\mathbf{F}$ recoge la fuente volumétrica y el flujo de frontera.

Nótese que el sistema es de **primer orden** en el tiempo, a diferencia del mecánico $\mathbf{M}\ddot{\mathbf{u}}+\mathbf{C}\dot{\mathbf{u}}+\mathbf{K}\mathbf{u} = \mathbf{F}$. No hay aceleración ni inercia: no existe un análogo térmico de la velocidad con significado físico en este modelo.

Sin notación Voigt. El gradiente ∇T es un vector genuino de 2 ó 3 componentes, no un tensor simétrico comprimido, así que la matriz $\mathbf{B}$ térmica es el gradiente crudo $\partial N/\partial x$ y no lleva los factores $\gamma = 2\varepsilon$ de la notación Voigt mecánica. Los elementos térmicos declaran `FLUX_DIM` (2 ó 3) en lugar de `STRAIN_DIM`.

8.2 Material: `ThermalConduction`

Se declara en un bloque `thermal_materials`, paralelo a `materials`. Los dos bloques tienen espacios de nombres de `type:` independientes, y un elemento resuelve su material contra el bloque donde se declaró el `id`.

Tabla 8.1: Parámetros de `ThermalConduction`.

Parámetro	Unidades	Obligatorio	Descripción
`k`	W/(m·K)	sí	Conductividad. Escalar (isótropo) o tensor completo
`dim`	—	no (2)	Dimensión del problema cuando `k` es escalar
`c`	J/(kg·K)	sólo transitorio	Calor específico
`density`	kg/m^3	sólo transitorio	Densidad

```yaml
thermal_materials:
  - id: 1
    type: ThermalConduction
    k: 45.0            # acero estructural
    c: 460.0           # sólo hace falta en transitorio
    density: 7850.0    # sólo hace falta en transitorio
```

`c` y `density` **son opcionales**: el régimen estacionario no los necesita, porque el término $\mathbf{C}\dot{\mathbf{T}}$ desaparece. Si faltan y se pide un transitorio, el error lo dice explícitamente y sugiere el solver estacionario.

Conductividad anisótropa: `k` admite un tensor completo, sin necesidad de un material distinto. Debe ser simétrico y definido positivo — ambas cosas se validan al construir.

```yaml
thermal_materials:
  - id: 1
    type: ThermalConduction
    k: [[45.0, 0.0], [0.0, 12.0]]   # 3.75× más conductor en x que en y
```

8.3 Elementos: `Quad4Thermal` y `Hex8Thermal`

Un DOF escalar `T` por nodo. Comparten geometría y numeración de nodos con sus gemelos mecánicos `Quad4` y `Hex8`.

Tabla 8.2: Elementos térmicos.

Elemento	FLUX_DIM	Nodos	Cuadratura default	Parámetros
`Quad4Thermal`	2	4	`2x2`	`thickness` (default 1.0), `quadrature`
`Hex8Thermal`	3	8	`hex_2x2x2`	`quadrature`

```yaml
elements:
  - id: 1
    type: Quad4Thermal
    material: 1          # id del bloque thermal_materials
    nodes: [1, 2, 3, 4]
    thickness: 0.3
```

Pasar un material mecánico a un elemento térmico —o al revés— se rechaza al construir, con un mensaje que nombra las dos familias. La dimensión también se comprueba: un material construido con `dim: 2` no entra en un `Hex8Thermal`.

8.4 Condiciones de frontera

8.4.1 Dirichlet — temperatura impuesta

Se declaran como cualquier otra condición de frontera, usando el nombre del DOF:

```yaml
boundary_conditions:
  - {node_id: 1, T: 100.0}
  - {node_id: 2, T: 20.0}
```

El estacionario necesita al menos un Dirichlet; el transitorio, no. En el estacionario, sin ninguno, la matriz de conductividad es singular: un campo de temperatura uniforme no produce gradiente ni flujo, y el nivel absoluto de temperatura queda indeterminado. Es el análogo térmico de un modo de sólido rígido, y el error lo nombra así en vez de reportar un fallo algebraico genérico. Las restricciones multipunto no cuentan como Dirichlet: relacionan temperaturas entre sí y el campo uniforme las cumple. Una restricción lineal sin maestros (`masters: []`, `T_s = g`) sí es un Dirichlet. En el transitorio un modelo sin ninguna temperatura impuesta es un **cuerpo aislado** y está bien planteado: la capacidad calorífica fija el nivel de temperatura a partir de `T_initial`, y la energía almacenada crece exactamente lo que aportan las fuentes (un cuerpo aislado con una fuente uniforme Q se calienta como $T(t) = T_0 + Q\,t/(\rho c)$).

8.4.2 Restricciones multipunto sobre `T`

El bloque `linear_constraints` (ver el capítulo del formato YAML) admite también el DOF `T`: $T_s = g + \sum_i \alpha_i T_{m_i}$. El caso típico es atar dos nodos coincidentes de mallas distintas con $T_s = T_m$ (continuidad de temperatura en una interfaz), la periodicidad, o un salto constante con $g \neq 0$. Atar dos nodos coincidentes da el mismo campo que fundirlos en uno, en el estacionario y en el transitorio.

```yaml
linear_constraints:
  # Continuidad de temperatura entre el nodo 105 y el nodo 5, coincidentes.
  - slave: {node: 105, dof: T}
    masters:
      - {node: 5, dof: T}
    coefficients: [1.0]
```

Con una temperatura impuesta variable en el tiempo (argumento `dirichlet_func` del `ThetaMethodSolver`, sólo desde Python), la función da los valores de los nodos con Dirichlet declarado; los esclavos de las restricciones los reconstruye el solver con la propia restricción, así que un esclavo atado a un nodo con temperatura impuesta la sigue y el salto g se conserva. Un valor no nulo en un nodo sin Dirichlet declarado se rechaza: el conjunto de nodos prescritos lo fija el modelo.

8.4.3 Neumann — flujo prescrito

```yaml
thermal_loads:
  boundary_flux:
```

```
    - {element: 1, edge: 2, q: 500.0}      # 2D: borde del elemento
    - {element: 7, face: 4, q: -200.0}     # 3D: cara del elemento
```

Convención de signo: $\bar{q} > 0$ es flujo **saliente** del dominio, es decir **enfriamiento**. Un flujo entrante (calentamiento) se declara con $\bar{q} < 0$. El signo sale de la forma débil, donde el término de frontera aparece como $-\int_\Gamma w\,\bar{q}\,d\Gamma$.

Un borde o cara sin declarar es adiabático ($\bar{q} = 0$), igual que un borde sin carga es libre de tracción en mecánica. No hay que declarar el flujo nulo.

Con una malla gmsh, el flujo se declara sobre un grupo físico de contorno, cuyas celdas son los bordes exactos que se cargan:

```
thermal_loads:
  boundary_flux_by_group:
    Cara_caliente: {q: -200.0}             # entrante: calentamiento
    Cara_fria: {q: "50*(1 + x)"}           # variable a lo largo del borde
```

`q` y `Q` admiten expresiones de la posición `x`, `y`, `z` (y, en el flujo, de la normal exterior `nx`, `ny`, `nz`); la sintaxis está en el capítulo del archivo YAML, «Valores Variables: Expresiones».

Los índices de borde y cara son los mismos que en los elementos mecánicos: `edge: 0` conecta los nodos locales 0-1, `edge: 1` los 1-2, etc.

8.4.4 Fuente volumétrica

```
thermal_loads:
  body_source:
    - {Q: 1.0e5}                          # W/m^3, a todo el dominio
    - {Q: 5.0e4, elements: [3, 4, 5]}     # sólo a esos elementos
```

`Q` es generación de calor por unidad de volumen (calor de hidratación, efecto Joule, reacción química). Es **carga del elemento y no propiedad del material**: varía en el espacio y con el tiempo, y como propiedad obligaría a duplicar materiales.

También pueden aplicarse fuentes **nodales concentradas** [W] por la vía estándar:

```
point_loads:
  - {node_id: 12, T: 250.0}
```

8.5 Régimen estacionario

K T = F lo resuelve el `LinearSolver` **sin ninguna configuración especial**:

```
solver:
  type: LinearSolver
```

El resultado es un `SolveResult` como el de un análisis mecánico; el campo `U` contiene las temperaturas nodales, y `reactions_by_node` los flujos de calor en los nodos con temperatura impuesta.

8.6 Régimen transitorio: `ThetaMethodSolver`

Integra $\mathbf{C}\dot{\mathbf{T}} + \mathbf{K}\mathbf{T} = \mathbf{F}(t)$ ponderando el estado dentro del paso con el parámetro θ:

$$(\mathbf{C} + \theta\Delta t\,\mathbf{K})\,\mathbf{T}_{n+1} = [\mathbf{C} - (1-\theta)\Delta t\,\mathbf{K}]\,\mathbf{T}_n + \Delta t\,[\theta\mathbf{F}_{n+1} + (1-\theta)\mathbf{F}_n]$$

```yaml
solver:
  type: ThetaMethodSolver
  dt: 60.0
  n_steps: 500
  T_initial: 20.0
```

Tabla 8.3: Parámetros de `ThetaMethodSolver`.

Parámetro	Default	Descripción
`dt`	—	Paso temporal [s]. Obligatorio
`n_steps`	—	Número de pasos. Obligatorio
`T_initial`	—	Condición inicial: escalar (campo uniforme) o vector. Obligatorio
`theta`	`1.0`	Peso del esquema $\in [0, 1]$
`lumping`	`"lumped"`	Forma de la matriz de capacidad
`output_every`	`1`	Almacenar el campo cada N pasos

No hay condición inicial de velocidad: la ecuación es de primer orden y $\mathbf{T}_0$ determina la evolución por completo. Tampoco existe el parámetro `rayleigh`: el amortiguamiento de Rayleigh modela disipación en la ecuación de segundo orden, y aquí la disipación es la propia conducción, ya contenida en $\mathbf{K}$.

8.6.1 Continuar un análisis

`run_thermal_transient(..., previous=r)` parte del estacionario (`r = solidum.run(...)`; su carga se mantiene) o de otro transitorio térmico (sus temperaturas finales en su instante final; `F_func` y `dirichlet_func` en tiempo absoluto). Entonces no se da `T_initial`. Parar y continuar da lo mismo que no parar (ADR 0029).

8.6.2 Elección de θ

Tabla 8.4: Casos particulares del esquema θ.

θ	Nombre	Orden	Estabilidad
`0`	Euler explícito	1	Condicional ($\Delta t \leq 2/\lambda_{\text{máx}}$)
`0.5`	Crank-Nicolson	**2**	Incondicional (A-estable)
`2/3`	Galerkin	1	Incondicional
`1`	Euler implícito	1	Incondicional, **L-estable** (default)

Estabilidad no significa ausencia de oscilaciones, y esta distinción decide el valor por defecto.

Crank-Nicolson ($\theta = 1/2$) es de segundo orden y por tanto el más preciso, pero su factor de amplificación tiende a -1 para los modos de frecuencia alta en lugar de a 0. Ante un **cambio brusco** —un escalón de temperatura impuesto en la frontera, típico en el arranque— produce oscilaciones amortiguadas lentamente, con temperaturas que pueden **salirse del rango de los datos**. Eso viola el *principio del máximo* de la ecuación de difusión, según el cual la solución exacta nunca excede los extremos de los datos iniciales y de frontera: no es un resultado impreciso, es un resultado imposible.

Euler implícito ($\theta = 1$) es sólo de primer orden, pero **L-estable**: amortigua los modos altos por completo en un paso y nunca oscila. Por eso es el default.

En un ensayo con pared a 100 y cuerpo inicialmente a 0, con paso grande, $\theta = 1$ se mantiene en $[0, 100]$ y es monótono, mientras $\theta = 1/2$ alcanza 151.27.

Si su problema es suave y busca la precisión de segundo orden, use `theta: 0.5` conscientemente. El resultado expone el campo `order` (2 con Crank-Nicolson, 1 en el resto) para que el coste en precisión del default robusto sea visible.

8.6.3 Elección de Δt

Con $\theta \geq 0.5$ **ningún paso diverge**, pero eso no significa que cualquier paso sirva. La escala física la da la difusividad $\alpha = k/(\rho c)$ y el tamaño de elemento h:

$$\Delta t_{\text{car}} \sim \frac{h^2}{\alpha}$$

Es el tiempo que tarda el frente térmico en atravesar un elemento. Un Δt mucho mayor es estable pero **se salta el transitorio** que se quiere observar. El solver reporta este valor al arrancar y avisa si el paso elegido lo supera ampliamente. Es información, no restricción.

En el otro extremo, el transitorio no ha **terminado** hasta $t \gg L^2/\alpha$ con L la dimensión global del dominio. Integrar por debajo de esa escala y comparar con el estacionario da diferencias que no son error del esquema, sino un análisis inacabado.

8.6.4 Capacidad `lumped` por defecto

A diferencia del análisis dinámico estructural, donde el default es `consistent`, aquí la matriz de capacidad se agrupa (`lumped`) por defecto. El motivo es el mismo principio del máximo: la capacidad consistente produce oscilaciones espurias ante un frente térmico abrupto. Los dos defaults invertidos —`lumped` en el eje espacial y $\theta = 1$ en el temporal— atacan el mismo fenómeno desde lados distintos.

8.6.5 Resultado

`ThetaMethodSolver` devuelve un `ThermalTransientResult`, no el `TransientResult` del análisis dinámico: no hay velocidad, aceleración ni coeficientes de Rayleigh que reportar.

`extremes()` es la herramienta de diagnóstico del principio del máximo: en un problema sin fuente, cualquier valor fuera del rango de los datos señala oscilación espuria del esquema, no un fenómeno físico.

Con $\theta < 0.5$ y un paso por encima del límite de estabilidad, `solve()` **se detiene** con `TimeIntegrationInstabilityError` (ADR 0025): una línea `ERROR` y la excepción, cuyo

Tabla 8.5: Campos de `ThermalTransientResult`.

Campo / método	Descripción
`t_history`	Instantes almacenados, shape `(n_stored,)`
`T_history`	Campo global, shape `(n_dof, n_stored)`
`T_final`	Campo en el último instante
`temperature_at(dof)`	Historia temporal de un DOF concreto
`extremes()`	`(T_min, T_max)` sobre todos los nodos e instantes
`n_steps`, `theta`, `dt`, `order`	Parámetros efectivos del análisis
`converged`, `termination`	Si llegó al instante final (`"t_end"`)

`partial_result` es el `ThermalTransientResult` hasta el último paso estable. No devuelve un resultado con instantes sin calcular.

8.7 Ejemplo completo: pared plana en régimen transitorio

Pared de acero de 2 m, inicialmente a 20 °C, con una cara llevada súbitamente a 100 °C.

```yaml
nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [1.0, 0.0]}
  - {id: 3, coords: [2.0, 0.0]}
  - {id: 4, coords: [0.0, 0.5]}
  - {id: 5, coords: [1.0, 0.5]}
  - {id: 6, coords: [2.0, 0.5]}

thermal_materials:
  - {id: 1, type: ThermalConduction, k: 45.0, c: 460.0, density: 7850.0}

elements:
  - {id: 1, type: Quad4Thermal, nodes: [1, 2, 5, 4], material: 1, thickness: 0.3}
  - {id: 2, type: Quad4Thermal, nodes: [2, 3, 6, 5], material: 1, thickness: 0.3}

boundary_conditions:
  - {node_id: 1, T: 100.0}
  - {node_id: 4, T: 100.0}
  - {node_id: 3, T: 20.0}
  - {node_id: 6, T: 20.0}

solver:
  type: ThetaMethodSolver
  dt: 2000.0
  n_steps: 200
  T_initial: 20.0
```

Con estos valores el nodo central llega a 59.998, frente al 60 exacto del perfil lineal estacionario. La diferencia **no es error del esquema**: con $\Delta t = 2000$ s y 200 pasos el análisis cubre 4×10^5 s, mientras el tiempo de difusión global de esta pared es $L^2/\alpha \approx 3.2 \times 10^5$ s — el transitorio está terminando pero aún no ha terminado. Es exactamente el efecto descrito en «Elección de Δt»: alargando el análisis, el campo converge al perfil lineal $100 \to 20$ que da el `LinearSolver` sobre el mismo modelo, a precisión de máquina.

Como la condición inicial (20 °C) contradice el Dirichlet impuesto (100 °C) en la cara izquierda, el solver **avisa** de la discontinuidad en $t = 0$ sin abortar: un choque térmico es modelización legítima, y es precisamente el caso que motiva el default $\theta = 1$. El aviso compara sólo los nodos con Dirichlet declarado, no los esclavos de restricciones multipunto.

8.8 Limitaciones actuales

- **Sin convección ni radiación**. Una frontera con convección $q = h(T - T_\infty)$ debe modelarse hoy como flujo prescrito equivalente, lo que exige conocer T de antemano — es decir, no es un sustituto general.
- **Sin acoplamiento termomecánico**: el campo de temperatura no produce deformación. Un análisis termoelástico requiere hoy resolver el térmico y trasladar el resultado a mano.
- **Material lineal**: k no depende de la temperatura, y no hay cambio de fase. Cualquiera de las dos cosas exigiría iteraciones de Newton dentro de cada paso temporal.
- **Paso de tiempo constante**: no hay adaptación automática.
- **Sólo elementos lineales**: `Quad4Thermal` y `Hex8Thermal`. No existen versiones cuadráticas ni triangulares/tetraédricas térmicas.

Capítulo 9

Post-Procesamiento y Visualización

9.1 Formato VTU (VTK XML)

Mediante `meshio`, Solidum FEM exporta los resultados en archivos `.vtu` compatibles con ParaView. La información se clasifica en dos diccionarios estándar:

9.1.1 Datos Nodales (Point Data)

- `Displacements`: campo vectorial (u_x, u_y, u_z) por nodo.
- `Rotations`: giros (r_x, r_y, r_z), sólo si algún nodo tiene grados de libertad de giro (marcos).
- `External_Forces`: distribución nodal del vector de cargas aplicado en el paso actual.
- `Supports` (y `Supports_Rot` con giros): indicador de DOFs restringidos, 1.0 en cada componente con Dirichlet.
- `Temperature`: temperatura nodal, sólo si algún nodo tiene el DOF `T`.
- `Nodal_Mass`, `Nodal_Inertia`: masas de nudo (m_x, m_y, m_z) e inercias rotacionales (J_x, J_y, J_z) declaradas (`nodal_masses`, ADR 0024), sólo si el modelo tiene alguna; `Nodal_Inertia`, si tiene giros. Sirven para revisar la entrada de masas en ParaView.
- `Sigma_XX_nodal` … `Tau_XZ_nodal`, `Von_Mises_nodal`: suavizado nodal de orden 0 de los esfuerzos por celda (media simple sobre las celdas adyacentes a cada nodo). Sólo promedian las celdas de **sólidos mecánicos**: las barras, cables y marcos (celdas `line`) y los elementos térmicos no participan, tampoco cuando un elemento térmico se superpone a un sólido sobre los mismos nodos en un dominio multicampo (u_x, u_y, T). Un modelo sin sólidos mecánicos (sólo térmico o sólo estructural 1D) no exporta estos campos.

Restricciones multipunto y suavizado nodal

Con restricciones multipunto (bloque `linear_constraints` en YAML, `add_linear_constraint` en Python) el archivo recibe la solución completa, con los DOF esclavos ya recuperados: `Displacements`, `Temperature` y todos los campos por celda son los del modelo resuelto también en los nodos esclavos. El suavizado nodal tiene una limitación: dos nodos coincidentes atados por una

restricción ($u_s = u_m$, p. ej. para unir mallas no conformes) siguen siendo dos nodos distintos, y cada uno promedia sólo las celdas de su lado. En la interfaz, `Sigma_*_nodal` del nodo maestro y del esclavo difieren entre sí y del valor que daría la malla con el nodo fundido (que, si cada lado aporta el mismo número de celdas, es la media de las dos copias). Para leer el esfuerzo en la interfaz, use los campos por celda o promedie las dos copias.

9.1.2 Datos de Elemento (Cell Data)

- `Von_Mises`: esfuerzo equivalente J2, calculado como invariante 3D con las seis componentes; en plane strain incluye σ_{zz} (que aporta el material vía `out_of_plane_stress`), en plane stress $\sigma_{zz} = 0$.
- `Internal_State`: variable principal del material según su `PRIMARY_STATE_VAR` (acumulada equivalente α en plasticidad, d en daño, etc.).
- `N_i`, `N_j`, `V_i`, `M_i`… (en 3D, `Vy`, `Vz`, `T`, `My`, `Mz`): **fuerzas internas de extremo** de barras, cables y marcos (`internal_forces`, ADR 0002), en ejes locales del elemento, en el nodo i y el nodo j, con la convención de signos del proyecto; `NaN` en las celdas que no tienen esa componente. Con carga distribuida, `VtkExporter.export(..., element_forces=resultado.element_forces)` descuenta la carga equivalente.
- `Sigma_XX`, `Sigma_YY`, `Sigma_ZZ`, `Tau_XY`, `Tau_YZ`, `Tau_XZ`: las seis componentes del tensor de esfuerzos (en 2D las tres últimas son σ_{zz} del material y ceros). Se evalúan en el campo `U` exportado vía `compute_gauss_state` y se promedian por elemento: el exportador no depende de qué solver produjo `U` ni de que el estado esté comiteado. Cubre sólidos 2D y 3D (`quad`, `triangle`, `quad8`, `quad9`, `triangle6`, `hexahedron`, `tetra`, `hexahedron20`, `hexahedron27`, `tetra10`), elementos térmicos (campo nodal `Temperature` y `Flux` por celda) y los elementos de los módulos de usuario, como el `CST_Embedded2D` del módulo `discontinuities`: el exportador deriva la celda VTK del número de nodos (o de `VTK_CELL_TYPE`, si el elemento lo declara), sin conocer la clase. Las barras y cables de dos nodos y los marcos son celdas `line`; las barras de tres nodos `Truss2D3` y `Truss3D3`, celdas `line3` (segmento cuadrático, con el nodo intermedio).

Las interfaces de adherencia que genera el refuerzo embebido con `bond` (`BondSlipInterface`) no tienen celda propia y no se exportan: unen los nodos de la barra con los de su anfitrión. Su estado (deslizamiento, esfuerzo de adherencia) se consulta con `EmbeddedBar.bond_state` (capítulo «Uso Avanzado: API Programática»).

9.2 Animación con Archivo PVD

Cuando se exporta con `frequency: "all_steps"`, además de un `.vtu` por paso se genera un archivo maestro `.pvd` (XML colección) que ParaView abre como animación temporal. El "tiempo" en el archivo PVD coincide con el factor de carga λ.

9.3 Salida en Texto Plano (Estilo FEAP)

Opcional para verificación manual o post-procesamiento con scripts. Configurable por nodos y elementos seleccionados:

```yaml
output:
  text_export:
    export: true
    nodes: [1, 5, 10]                # o "all"
    elements: all
    nodal_results: ["Displacements", "External_Forces"]
    element_results: ["Von_Mises", "Internal_State"]
```

9.4 Workflow ParaView

1. `File > Open` y seleccionar el archivo `.pvd` (o el grupo de `.vtu`).
2. *Apply* en el panel *Properties*.
3. Reproducir la animación con los controles temporales de la barra superior.
4. Aplicar filtro **Warp By Vector** con `Displacements` para visualizar la deformación física (ajustar `Scale Factor` para amplificar).
5. Cambiar el coloreado de `Solid Color` a la variable de interés (`Von_Mises`, `Internal_State`, `Sigma_XX`, …).

Capítulo 10

Ejemplos Completos

Nota

Este capítulo muestra la sintaxis de modelos completos. Los **problemas resueltos y verificados contra solución analítica**, con la explicación de cada paso, la lectura de resultados y la discusión de los errores, están en el *Manual de Ejemplos* (`manuals/Example_manual.pdf`).

10.1 Marco 2D — Viga en Voladizo

Combinación de un tramo Euler-Bernoulli y un tramo Timoshenko, empotrada en el extremo y cargada en la punta.

```yaml
nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [2.0, 0.0]}
  - {id: 3, coords: [4.0, 0.0]}

materials:
  - id: 1
    type: Elastic1D
    E: 200.0e9
    density: 7850.0

elements:
  - id: 1
    type: Frame2DEuler
    material: 1
    nodes: [1, 2]
    A: 0.01
    I: 8.33e-6

  - id: 2
    type: Frame2DTimoshenko
    material: 1
    nodes: [2, 3]
    A: 0.01
    I: 8.33e-6
    As: 0.00833
```

```yaml
boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}

point_loads_by_node:
  - {node_id: 3, uy: -15000.0}

solver:
  type: LinearSolver

output:
  file_name: "viga_voladizo_marco"
  pre_process: {export: true}
  results:
    frequency: "last_step"
    nodal_results: ["Displacements"]
```

10.2 Placa Continua con Plasticidad J2 (Gmsh)

Placa con perforación central importada desde `.msh`, sometida a carga axial creciente hasta superar el límite elástico del acero. Solver no lineal con paso adaptativo.

```yaml
mesh: "placa_agujero.msh"

mesh_physical_groups:
  "Superficie_Placa":
    material: 1
    thickness: 0.05
    quadrature: "2x2"

materials:
  - id: 1
    type: VonMises2D
    E: 200.0e9
    nu: 0.3
    sigma_y: 250.0e6
    H: 1.5e9
    hypothesis: plane_strain
    density: 7850.0

boundary_conditions_by_group:
  "Borde_Fijo": {ux: 0.0, uy: 0.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}

solver:
  type: NonlinearSolver
  num_steps: 30
  max_iter: 10
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5
```

```yaml
output:
  file_name: "estudio_placa"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State"]
```

Interpretación: durante los primeros pasos (régimen elástico) Newton-Raphson converge en 1–2 iteraciones. Cuando la concentración de esfuerzos en la periferia del orificio supera $\sigma_y = 250$ MPa, comienzan iteraciones del *return mapping*; `Internal_State` (la deformación plástica acumulada α) crece visiblemente alrededor del agujero.

10.3 Cable Pretensado en Régimen Corotacional

Cable 2D con material unilateral, cargado transversalmente en su nodo central. Demuestra el acoplamiento elemento corotacional + material unilateral.

```yaml
nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [5.0, 0.0]}
  - {id: 3, coords: [10.0, 0.0]}

materials:
  - id: 1
    type: CableMaterial1D
    E: 150.0e9
    density: 7850.0

elements:
  - id: 1
    type: Cable2DCorot
    material: 1
    nodes: [1, 2]
    A: 5.0e-5

  - id: 2
    type: Cable2DCorot
    material: 1
    nodes: [2, 3]
    A: 5.0e-5

boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0}
  - {node_id: 3, ux: 0.0, uy: 0.0}

point_loads_by_node:
  - {node_id: 2, uy: -500.0}

solver:
  type: NonlinearSolver
  num_steps: 50
  max_iter: 25
```

```
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

output:
  file_name: "cable_carga_transversal"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
```

Capítulo 11

Uso Avanzado: API Programática

Aunque el flujo principal es *data-driven* (YAML), el motor se puede usar como librería Python para escenarios donde el archivo no es suficiente: optimización paramétrica, generación procedural de mallas, acoplamiento con bibliotecas externas, scripting de batería de tests.

11.1 Patrón Fachada

Los componentes principales se exponen desde la raíz del paquete:

```
import numpy as np
from solidum import Domain, VonMises2D, Quad4, Assembler, NonlinearSolver, VtkExporter

# 1. Dominio y nodos
domain = Domain()
n1 = domain.add_node(1, [0.0, 0.0])
n2 = domain.add_node(2, [1.0, 0.0])
n3 = domain.add_node(3, [1.0, 1.0])
n4 = domain.add_node(4, [0.0, 1.0])

# 2. Material y elemento
acero = VonMises2D(E=200e9, nu=0.3, sigma_y=250e6, H=2e9, hypothesis='plane_strain')
placa = Quad4(element_id=1, nodes=[n1, n2, n3, n4], material=acero, thickness=0.1)
domain.add_element(placa)

# 3. Restricciones (Dirichlet)
n1.fix_dof('ux', 0.0); n1.fix_dof('uy', 0.0)
n4.fix_dof('ux', 0.0); n4.fix_dof('uy', 0.0)

domain.generate_equation_numbers()

# 4. Cargas (Neumann)
F_ext = np.zeros(domain.total_dofs)
F_ext[n2.dofs['ux']] = 150000.0
F_ext[n3.dofs['ux']] = 150000.0

# 5. Resolución
assembler = Assembler(domain)
solver = NonlinearSolver(assembler, num_steps=10, adaptive=True)
U = solver.solve(F_ext)
```

```
# 6. Exportación
VtkExporter(domain).export("resultados.vtu", U=U, F_ext=F_ext)
```

`Assembler(domain)` ensambla **por lotes** (ADR 0014): agrupa los elementos que comparten clase, material y cuadratura en familias y los evalúa en un kernel compilado, con el mismo resultado que el bucle elemento a elemento a precisión de máquina. `Assembler(domain, batch=False)` fuerza el camino por elemento en todo el modelo (útil para comparar o diagnosticar); `parallel=False` usa el kernel serie en vez del paralelo (el resultado es bit a bit el mismo; el número de hilos lo fija Numba con la variable de entorno `NUMBA_NUM_THREADS` o `numba.set_num_threads`); `batch_memory_budget=` acota, en bytes, los temporales por trozo. Los elementos sin kernel por lotes (1D) y los que guardan estado propio fuera de `ElementState` (como el `CST_Embedded2D` del módulo de usuario `discontinuities`) se ensamblan como siempre dentro del mismo `Assembler`. Si se sustituye el material o el espesor de un elemento después del primer ensamblaje, llamar a `assembler.invalidate()`. Un elemento con nodos en orden incorrecto (jacobiano negativo) detiene el ensamblaje con `ValueError` que nombra su `id`.

Para obtener deformaciones y esfuerzos en los puntos de Gauss de todo el modelo, `solidum.math.batch.gauss_states(domain, U)` devuelve `{id: compute_gauss_state(U)}` evaluando cada familia de una vez (las entradas son vistas sobre arreglos) y llamando a `compute_gauss_state` sólo en los elementos sin familia; el exportador VTK hace lo mismo por dentro.

El exportador evalúa los esfuerzos en el `U` que recibe (`compute_gauss_state`), así que funciona igual tras `LinearSolver.solve` directo, tras `solidum.run` o en un paso intermedio del `step_callback`. Si el modelo lleva peso propio o fuerza de cuerpo, usar `solidum.run(...)` en vez de `solver.solve(...)` para que `SolveResult.element_forces` reste la carga nodal equivalente y devuelva fuerzas internas de extremo coherentes con las reacciones.

11.2 Un modelo YAML desde Python: `load_yaml`

`solidum.run_yaml(path)` lee el archivo, arma el modelo y lo resuelve, y devuelve sólo el resultado. Para conservar el modelo y consultarlo antes o después de resolver —los grados de libertad de cada nodo, la matriz de masa, la exportación VTK, el estado de los elementos— se usa `solidum.load_yaml(path)`:

```
import solidum

modelo = solidum.load_yaml("modelo.yaml")   # YamlModel: domain, assembler, parser
res = modelo.solve()                          # lo mismo que run_yaml("modelo.yaml")
tope = modelo.domain.nodes[12]
print(res.U[tope.dofs["ux"]])
M = modelo.assembler.assemble_mass_matrix()
```

`modelo.solve(step_callback=f)` acepta el mismo `step_callback` que `run_yaml`: `f(step, U, λ)` en los estáticos y `f(step, U, t)` en los análisis en el tiempo. `modelo.domain` tiene los grados de libertad ya numerados y `modelo.assembler` es el ensamblador con el que `solve()` resuelve el análisis declarado (estático, modal, transitorio, armónico, espectral, térmico o por etapas). `modelo.parser` da las cargas del archivo (`get_external_forces()`, `get_body_load(assembler)`, `get_thermal_loads()`) y el solver declarado (`get_solver(assembler)`).

11.3 Resultado de un análisis estático: `SolveResult`

`solidum.run(...)` (y `solidum.run_yaml(...)` con un solver estático) devuelve un `SolveResult` inmutable, que queda también en `domain.last_result`:

Campo	Contenido
`U`	Solución global (desplazamientos y giros, o temperaturas), shape `(n_dof,)`.
`F_applied`	Cargas que ve el sistema (nodales más equivalentes de elemento), escaladas por el factor de carga final.
`R`	Reacciones de los **apoyos**, shape `(n_dof,)`: los grados de libertad con valor prescrito y las restricciones multipunto que actúan como apoyo (un rodillo oblicuo). Ceros en el resto.
`reactions_by_node`	`{nodo: {dof: valor}}` de `R`: los nodos con algún valor prescrito y los grados de libertad de las restricciones multipunto de apoyo.
`constraint_forces`	Fuerzas de restricción de las restricciones multipunto de **unión** (atado de mallas, periodicidad, unión rígida), shape `(n_dof,)`; ceros si no hay.
`constraint_forces_by_node`	`{nodo: {dof: valor}}` de `constraint_forces` en los grados de libertad de esas uniones.
`element_forces`	`{elemento: ElementForces}` con las fuerzas internas (N, V, M, T) de los elementos estructurales.
`converged`, `num_steps`	Si el análisis llegó al final y cuántos pasos convergidos dio.
`termination`	Por qué se detuvo un solver incremental (ADR 0025): `"max_lambda"`, `"control_target"` o `"max_steps"`. `None` en el `LinearSolver`.
`load_factor`	Factor de carga final λ de la carga que crece en el análisis o en la etapa.
`perturbation`	`True` en un `LinearSolver` sobre una etapa (ADR 0031): perturbación lineal que no cambió el estado; no se puede usar como `previous`.
`F_constant`, `F_variable`	En un análisis por etapas (`solidum.run(..., previous=…)`, ADR 0026), la carga ya aplicada que se mantuvo constante y la que creció (`F_applied = F_constant + F_variable`). Fuera de una etapa, `F_constant` es `None`.
`equivalent_loads`	Cargas equivalentes de la carga distribuida restadas, por elemento, para dar `element_forces`: las que lleva la carga aplicada (`LoadVector`, ADR 0027); una etapa siguiente las hereda con su carga constante.
`condition_estimate`	Número de condición estimado de la rigidez (escalada por la diagonal) en el análisis lineal: $\log_{10}$ de este valor acota las cifras que la solución puede haber perdido de unas 16; por encima de 10^{12} Solidum avisa (capítulo «Diagnóstico de Problemas»). `None` en los análisis no lineales y con el solver iterativo.

Las reacciones y las fuerzas de restricción cumplen `F_int - F_applied = R + constraint_forces`: el equilibrio global es Σ`R + `Σ`F_applied = 0` por dirección, y las fuerzas de restricción suman cero

porque sólo transmiten fuerza entre partes de la estructura. El criterio que separa una restricción multipunto de apoyo de una de unión se explica en la sección `linear_constraints` del capítulo «Archivo de Entrada YAML». En el problema térmico `R` es el flujo de calor nodal en los nodos con temperatura impuesta.

11.4 Cargas distribuidas y sus cargas equivalentes

`assemble_self_weight` y `assemble_body_load` devuelven un `LoadVector` (ADR 0027): un vector de cargas que se usa como cualquier `np.ndarray`, pero que lleva las cargas nodales equivalentes de su parte distribuida en cada elemento. Las fuerzas internas de extremo descuentan las de la carga aplicada, y la aritmética de las cargas las combina:

```
F_peso = asm.assemble_self_weight([0.0, -9.81])
F_viento = asm.assemble_body_load([2.0e3, 0.0])
F = 1.2 * F_peso + 1.6 * F_viento + F_puntual     # lleva 1.2·f_eq,peso + 1.6·f_eq,
    viento
res = solidum.run(dom, assembler=asm, F_applied=F)
```

Vale igual para dos casos de carga con el mismo ensamblador, para la carga de una etapa y para la función de carga de un transitorio (`F_func=lambda t: F_peso + a(t) * F_viento`, con `record_internal_forces=True`). Las cargas puntuales en un `np.ndarray` simple no aportan cargas equivalentes. Lo que **pierde** la información es convertir la carga con `np.asarray`/`np.array` o sumarla con `+=` sobre un arreglo de ceros: entonces Solidum supone, como antes, que contiene la última carga de cuerpo ensamblada, y avisa si se ensambló más de una.

11.5 Secciones por fibras

El bloque `sections` del YAML tiene su equivalente directo en Python (ADR 0021): las primitivas generan las fibras y la sección se pasa al marco con `section`, en lugar de `material` y las propiedades geométricas.

```
from solidum import Domain, Elastic1D, Elastoplastic1D, Frame2DEuler
from solidum.sections import FiberSection, RectangularPatch, BarLayer

concreto = Elastic1D(E=3.0e10, density=2400.0)
acero = Elastoplastic1D(E=2.0e11, sigma_y=4.2e8, H=2.0e9, density=7850.0)
sec = FiberSection(
    patches=[RectangularPatch(concreto, y=(-0.25, 0.25), z=(-0.15, 0.15), ny=40, nz=1),
             BarLayer(acero, start=(-0.21, -0.11), end=(-0.21, 0.11), n=3, area=5.1e-4,
                      displaces=concreto)],   # opcional: descuenta el concreto que
                          ocupan
    GJ=None,                          # obligatorio sólo si la usa un Frame3D
)

domain = Domain()
n1 = domain.add_node(1, [0.0, 0.0])
n2 = domain.add_node(2, [3.0, 0.0])
domain.add_element(Frame2DEuler(1, [n1, n2], section=sec, n_ip=3))
```

`CircularPatch(material, r_outer=..., r_inner=0.0, center=(0.0, 0.0), n_rad=..., n_circ=...)` y `Fiber(y, z, area, material, displaces=None)` completan las primitivas.

Con `displaces` (el mismo objeto material del parche que contiene la barra), la sección añade al final de sus arreglos una fibra de compensación de área negativa por barra, marcada en `sec.is_compensation` (ver `displaces` en el bloque `sections` del capítulo del YAML). La sección expone `sec.y`, `sec.z`, `sec.area` (arreglos por fibra), `sec.n_fibers`, `sec.materials()` y `sec.mass_properties()`; su respuesta a nivel de sección, `sec.response_2d([eps0, kappa_z], estados)` y `sec.response_3d([eps0, kappa_y, kappa_z], estados)`, devuelve las fuerzas internas, su tangente y los estados nuevos de las fibras (con `sec.initial_states()` como estado virgen), útil para trazar diagramas momento–curvatura sin montar un modelo. Tras resolver, `elem.compute_internal_forces(U)` añade `section_deformations` y `section_forces` en los puntos de Gauss.

11.6 Módulos de usuario

La raíz del paquete sólo reexporta el programa principal. Las formulaciones no estándar viven en módulos de usuario (`solidum/user/<nombre>/`, ADR 0020) que `import solidum` no carga; en Python se cargan importándolos directamente o con `solidum.load_user_module`, el equivalente de `user_modules` en el YAML:

```
import solidum
from solidum import Domain, Elastic2D
from solidum.materials.interface import CohesiveDamageIsotropic   # programa principal
    (ADR 0033)
from solidum.user.discontinuities import CST_Embedded2D

solidum.available_user_modules()                # ['discontinuities', '
    dissipation_control']
solidum.load_user_module("discontinuities")     # equivalente; cargarlo dos veces no
    hace nada

bulk = Elastic2D(E=30e9, nu=0.2, hypothesis="plane_strain")
cohesivo = CohesiveDamageIsotropic(sigma_t0=2.5e6, G_f=100.0, K_e=1.0e13, softening="
    linear")
grieta = CST_Embedded2D(element_id=1, nodes=[n1, n2, n3], material=bulk,
                        cohesive_material=cohesivo, thickness=1.0)
```

`from solidum import CST_Embedded2D` ya no funciona: el elemento y `DiscontinuityState` se importan de `solidum.user.discontinuities`. La ley cohesiva es un material de interfaz del programa principal (`solidum.materials.interface`; clase base `solidum.core.interface_material.InterfaceMaterial`, registro `solidum.registry.InterfaceMaterialRegistry`).

11.7 Masas concentradas en nudos

```
domain.add_nodal_mass(5, ux=12.0e3, uy=12.0e3)   # masa [kg]
domain.add_nodal_mass(5, rz=4.0e3)               # inercia rotacional [kg·m²]
```

Equivale al bloque YAML `nodal_masses` (ADR 0024): un grado de libertad mecánico por argumento, valores finitos no negativos, y varias llamadas sobre el mismo grado de libertad se suman. Las masas quedan en `domain.nodal_masses` (`{(nodo, dof): valor}`) y el `Assembler` las suma a la diagonal de la matriz de masa. Son sólo inercia: no generan peso con la gravedad.

11.8 Refuerzo embebido

`solidum.embed_reinforcement(domain, points, area, material, *, element=None, hosts=None, subdivisions=1, element_kwargs=None, bond=None)` embebe una barra en los sólidos del dominio (ADR 0032, spec `EmbeddedReinforcement`), con adherencia perfecta si `bond` es `None`, y devuelve un `EmbeddedBar`:

```
bar = solidum.embed_reinforcement(domain, [[0.0, 0.05], [4.0, 0.05]], 1.6e-3,
                                  solidum.Elastic1D(E=200e9))
r = solidum.run(domain, F_applied=F)
r.element_forces[bar.element_ids[0]].components["N"]   # fuerza axial del primer tramo
bar.bond_forces(r)    # (n_nodos, ndim): fuerza de adherencia en cada nodo de la barra
```

Adherencia-deslizamiento (spec `EmbeddedReinforcement` §12): `bond=solidum.Bond(law, diameter, perimeter=None, integration="nodal")` (o el diccionario equivalente) une cada tramo de la barra a su anfitrión con una interfaz distribuida gobernada por una ley de interfaz de línea (`BondSlipModelCode`, `BondSlipMultilinear`); `perimeter` es $\pi \cdot$d_b por omisión e `integration` es `"nodal"` (recomendada) o `"gauss"`. Las interfaces generadas quedan en `bar.interface_ids`, y `bar.bond_state(domain, r.U)` devuelve, por tramo, los puntos de integración con el deslizamiento `slip`, el esfuerzo de adherencia `bond_stress`, la apertura y tracción normales y la fuerza de adherencia por unidad de longitud `bond_force`. Con `bond`, `bond_forces` no aplica (no hay restricciones de embebido) y lanza `ValueError`:

```
ley = solidum.BondSlipModelCode.model_code_2010(30e6, clear_rib_spacing=0.0105,
                                                diameter=0.016, E_c=30e9)
bar = solidum.embed_reinforcement(domain, [[0.0, 0.05], [4.0, 0.05]], 2.01e-4,
                                  solidum.Elastic1D(E=200e9), element="Truss2D3",
                                  bond=solidum.Bond(ley, diameter=0.016))
r = solidum.run(domain, F_applied=F)
puntos = bar.bond_state(domain, r.U)          # [[InterfacePoint, ...] por tramo]
s_max = max(abs(p.slip) for tramo in puntos for p in tramo)
```

`EmbeddedBar` guarda los nodos y elementos de la barra, el anfitrión y las coordenadas naturales de cada nodo, y la longitud. La llamada va después de crear los elementos de sólido; si el dominio ya estaba numerado, se renumera. Los generadores se registran en `ConstraintRegistry`; la localización de puntos en elementos (`solidum.mesh.point_location`: `natural_coordinates`, `PointLocator`) sirve a cualquier sólido que declare `REFERENCE_CELL` y sus funciones de forma.

11.9 Errores tipados

Desde un script conviene capturar los errores por su tipo, porque cada uno indica una causa y una salida distintas:

```
from solidum.math.solvers.diagnostics import (
    MechanismError,          # el modelo no está suficientemente apoyado (antes de
        resolver)
    IllPosedSystemError,     # solución no fiable: mecanismo interno o matriz singular
    SolverDivergedError,     # Newton, arco y transitorios: el análisis se detuvo
    StepConstraintError,     # arco: la restricción del paso no tiene solución
    TimeIntegrationInstabilityError,  # diferencia central o θ < 1/2: paso inestable
    MasslessDofsError,       # modal o transitorio: grados de libertad sin masa
```

```
    Termination,              # motivos de parada: solver.termination == Termination.
        MAX_STEPS
)
from solidum.math.linalg import IterativeNotConvergedError   # sólo con linear_algebra:
     iterative

try:
    U = solver.solve(F_ext)
except MechanismError as err:
    print(err.motions)        # p. ej. ['traslación en la dirección x']
```

`MechanismError.motions` lista los movimientos libres ya descritos en palabras. Para el fracaso de un trazado (Newton o arco) se captura `SolverDivergedError`, no `RuntimeError`: `IllPosedSystemError`, el veredicto de la red de seguridad, también es un `RuntimeError`. Tras la excepción, `solver.termination == "diverged"` y `solver.U_final`, `lambda_final` y `steps_done` dicen hasta dónde llegó (ADR 0025). En un análisis en el tiempo, `exc.partial_result` (también `solver.partial_result`) es el resultado hasta el último paso convergido, y `solver.t_final` su instante; un transitorio que llega al final deja `solver.termination == "t_end"` y `result.termination == "t_end"`. `MasslessDofsError` (ADR 0024) lleva la causa en `cause` (en el modal, `too_many_modes`, `no_mass`, `massless_mechanism`, `singular_mass`, `indefinite_mass`; en los transitorios implícitos, las mismas salvo `too_many_modes`; en la diferencia central, `explicit_integration`), los grados de libertad sin masa en `massless_dofs` y cuántos tienen masa en `n_mass`. Los mensajes completos y cómo corregir cada caso están en el capítulo *Diagnóstico de Problemas*.

El solver del sistema lineal se puede fijar también desde Python con el mismo parámetro que en el YAML, p. ej. `LinearSolver(assembler, linear_algebra="iterative")`.

Capítulo 12

Diagnóstico de Problemas

- `YamlValidationError: parámetro 'X' no aceptado por 'Y'.`

El parser detecta un parámetro que el constructor del elemento no admite. La salida lista los parámetros aceptados; revisar el catálogo o la spec del componente.

- `MechanismError: El modelo no está suficientemente apoyado para un análisis estático…`

Los apoyos no impiden que el modelo se mueva como sólido rígido, así que el problema estático no tiene solución única. Solidum lo detecta **antes de resolver** y dice exactamente qué movimiento queda libre, por ejemplo *"traslación en la dirección x"* o *"giro alrededor del eje z que pasa por (0, 0.5)"*. Añadir apoyos (o restricciones lineales) que impidan esos movimientos. Ojo con los rodillos alineados: impedir solo `uy` a lo largo de una recta vertical deja libre el giro alrededor de cualquier punto de esa recta. En un modelo térmico estacionario, el mensaje equivalente es que el campo `T` no tiene ningún valor prescrito. En análisis modal o dinámico un modelo libre sí es válido y no se comprueba.

- `IllPosedSystemError: Resultado descartado…`

El modelo está bien apoyado en conjunto, pero la solución no es fiable: o tiene un **mecanismo interno**, o el solver no consiguió resolver el sistema (*error hacia atrás normado* por encima de 10^{-10}, algo que no debería pasar con un modelo bien planteado). Un mecanismo interno es una parte que puede moverse sin deformar ningún elemento: una rótula o articulación de más, una barra o elemento suelto, o dos partes unidas por un solo nodo que pueden girar entre sí. El mensaje dice qué elementos se mueven y, si lo hacen como un sólido rígido, cómo: *"la parte formada por los elementos 1025–2048 puede moverse sin deformarse: giro alrededor del eje z que pasa por (1, 1)"*. Si el modelo es correcto pero tiene rigideces muy desproporcionadas (zonas rígidas, un material casi incompresible), no es un mecanismo: su modo de menor energía deforma algún elemento y no se rechaza por esto. Más raramente, rigideces muy desproporcionadas entre partes del modelo (penalizaciones, unidades mezcladas). Revisar la conectividad. Antes de esta comprobación, un modelo así devolvía desplazamientos absurdos sin ningún aviso. Si el mensaje dice que *el solver directo no pudo factorizar la matriz*, falta rigidez por completo en algún grado de libertad: un nudo al que sólo llega una barra en línea recta, un grado de libertad que ningún elemento usa, un giro sin rigidez.

- `IllPosedSystemError: Análisis descartado (NonlinearSolver): la matriz tangente inicial (U = 0)…`

Lo mismo en un análisis no lineal (`NonlinearSolver`, `ArcLengthSolver`, `IndirectDisplacementSolver`): la matriz tangente inicial es singular, así que el modelo tiene un mecanismo interno, aunque la carga no lo active, y el mensaje dice qué elementos se mueven. Si ningún elemento es geométricamente no lineal, la rigidez del modelo no puede crecer al deformarse (la no linealidad del material sólo la reduce) y el mecanismo lo es durante todo el análisis: por eso se rechaza antes de iterar. Para ver la forma del mecanismo, hacer un análisis modal del mismo modelo con densidad en los materiales: cada mecanismo aparece como una frecuencia nula y su forma modal muestra qué parte se mueve.

- **Aviso:** *la matriz tangente inicial (U = 0) es singular: el modelo tiene N mecanismo(s) interno(s)…*

El mismo caso con elementos geométricamente no lineales (corrotacionales, cables) o materiales unilaterales. La rigidez geométrica, o la de un cable al tensarse, puede estabilizar el mecanismo al deformarse —una red de cables sin pretensado es el ejemplo típico—, así que el análisis sigue. Si el modelo no es de ese tipo, revisar la conectividad de los elementos que nombra el aviso.

- **Aviso:** *sistema mal condicionado ($\tilde{\kappa} \approx$ …): la solución puede haber perdido hasta N de sus unas 16 cifras significativas…*

El análisis lineal se resolvió, pero el sistema está tan mal condicionado que la solución puede tener pocas cifras correctas. El ordenador trabaja con unas 16 cifras; con un número de condición κ, la solución conserva unas $16 - \log_{10}\kappa$, y en la práctica una o dos más. Solidum lo estima en todo análisis lineal estático y térmico estacionario (queda en `result.condition_estimate`) y avisa por encima de 10^{12}, cuando la solución baja a unas 5 cifras. Causas habituales: rigideces muy desproporcionadas (zonas rígidas de rigidez ficticia enorme, penalizaciones), un coeficiente de Poisson muy cercano a 0.5, o una barra dividida en demasiados elementos: en una viga κ crece con la cuarta potencia del número de elementos, y un voladizo de 10 000 elementos de Euler-Bernoulli da una flecha errónea en un 50 %. Remedios: rigideces de zona rígida unas 10^3–10^4 veces la del resto (no 10^8), una restricción multipunto en vez de una zona rígida, o menos elementos por barra (un elemento de Euler-Bernoulli por tramo ya da los nodos exactos con cargas nodales). Con un material casi incompresible el remedio sería una formulación mixta, que Solidum no implementa todavía: mientras tanto, conviene un ν no más allá de 0.499. No depende de las unidades.

- **Aviso:** *con el solver algebraico iterativo (linear_algebra: iterative) no se comprueba si el modelo tiene un mecanismo interno…*

El solver iterativo no factoriza la matriz y no puede delatar un mecanismo interno que la carga no active: la solución cumpliría el equilibrio, pero con una parte arbitraria del mecanismo. El aviso sale una vez por análisis estático. Para comprobarlo, un análisis modal con densidad o una ejecución con `linear_algebra: auto`.

- `SingularTangentError` **en un análisis no lineal**

La rigidez tangente perdió rango: un punto límite o bifurcación (usar `ArcLengthSolver`), o un mecanismo (ver los puntos anteriores). Si el mensaje añade *"el sistema tangente no se pudo resolver con precisión"*, la causa más probable es un mecanismo.

- **`StepConstraintError` en `ArcLengthSolver` (o en las variantes del arco)**

La restricción del paso no tiene solución aunque se reduzca la longitud del paso: con el arco cilíndrico, las raíces de su ecuación son imaginarias («Raíces imaginarias» en el detalle) porque el arco no corta la trayectoria de equilibrio. Ocurre en un retroceso (snap-back) por localización o tras un punto límite agudo. Usar `IndirectDisplacementSolver`, con una magnitud que crezca de forma monótona (el alargamiento de la zona que se ablanda, la apertura de la grieta), o un primer paso menor. El detalle del mensaje dice también el último paso convergido; el mismo dato queda en `solver.lambda_final` y `solver.U_final`.

- **Aviso:** *trazado detenido en λ=… < max_lambda=… tras agotar max_steps*

El trazado no terminó: el resultado es el del último paso convergido y `solver.termination` vale `"max_steps"` (`SolveResult.converged` es `False`). Aumentar `max_steps` o el tamaño del paso.

- **`TimeIntegrationInstabilityError` en un análisis en el tiempo**

El paso de tiempo supera el límite de estabilidad del esquema: diferencia central por encima de la condición CFL ($\Delta t < 2/\omega_{\text{máx}}$) o θ-método con $\theta < 1/2$. La solución crece sin límite y el análisis se detiene. Reducir `dt` o usar un esquema incondicionalmente estable (Newmark con $\beta = 1/4$, $\gamma = 1/2$; HHT-α; $\theta \geq 1/2$). El historial hasta el último paso estable está en `exc.partial_result`.

- **`MasslessDofsError` en un transitorio**

Hay grados de libertad libres sin masa (sin densidad en los elementos que los comparten ni masa de nudo, o sin inercia rotacional), y el mensaje los nombra («nodo 2 (ux)»). Los integradores implícitos los admiten (aceleración inicial consistente, ADR 0024 §6 bis): el error sólo aparece si además forman un mecanismo (`massless_mechanism`: sin masa y sin rigidez que los fije), y se corrige revisando la conectividad y los apoyos. La diferencia central (`explicit_integration`) exige masa en todos: dar masa a esos grados de libertad (`density` o `add_nodal_mass`) o usar un integrador implícito.

- **Aviso:** *la carga aplicada es un np.ndarray simple, sin sus cargas equivalentes por elemento…*

Las fuerzas internas de los elementos con carga distribuida pueden salir mal: la carga perdió sus cargas equivalentes al convertirla con `np.asarray`/`np.array` o al sumarla con `+=` sobre un arreglo de ceros, y se ensambló más de una carga de cuerpo. Combinar las cargas que devuelven `assemble_self_weight` y `assemble_body_load` con `+` y escalares (`F = F_a + λ·F_b`). Lo mismo avisa el historial de fuerzas internas de un transitorio cuyo `F_func` devuelve un `np.ndarray` simple.

- **`ValueError`: *la longitud característica h = … supera el límite 2·G_f·E/ft²***

Con la energía de fractura `G_f`, la rama de ablandamiento en tracción termina en $\varepsilon_{tu} = 2G_f/(f_t h)$. Si el punto de integración es tan largo que ε_{tu} quedaría por debajo de la deformación de agrietamiento f_t/E, la rama tendría retroceso en el propio material. El elemento lo rechaza al construirse y dice cuál es. Usar elementos más cortos.

- **`ValueError`: *ManderConcrete1D con G_f necesita la longitud característica del punto de integración***

El material regularizado se usa en un elemento que no le entrega la longitud de sus puntos. La entregan las barras y los marcos con sección por fibras. Con otro elemento, dar la rama descendente en deformaciones (`eps_tu`, `eps_cu`, `eps_sp`).

- **Un transitorio con concreto deja de converger en la vibración libre.**

El cierre de una grieta cambia bruscamente la rigidez de la fibra (de la secante agrietada a la de compresión) y el Newton puede rebotar entre las dos ramas. Activar `line_search: true` en el solver, o reducir `dt`.

- **Aviso:** *la etapa empieza fuera de equilibrio (‖F_c − F_int(u0)‖/‖F‖ = …)*

Una etapa (`solidum.run(..., previous=…)` o `solve(F, u0=…, constant_load=…)`) no parte del estado convergido de la anterior: otro `u0`, otra carga constante o un `Assembler` distinto, que no guarda el estado interno de los elementos. Continuar con `previous=resultado` y el mismo `Assembler`. La variante *el modelo conserva el estado interno de un análisis anterior, pero esta corrida empieza en U = 0* avisa de un análisis nuevo sobre un modelo ya cargado: para continuarlo, `previous`; para empezar de cero, construir el modelo de nuevo.

- **`ValueError`: *con 'previous' pase también el 'assembler' de la etapa anterior…***

El `Assembler` guarda el estado interno de los elementos (plasticidad, daño, grietas); uno nuevo partiría del estado virgen. Pasar a cada etapa el mismo `assembler`.

- **`ValueError`: *… no admite etapas de carga (u0, constant_load)…***

Sólo los solvers incrementales estáticos (`NonlinearSolver`, `ArcLengthSolver`, `IndirectDisplacementSolve` continúan una etapa con carga constante. Un `LinearSolver` sobre una etapa es una perturbación lineal (ADR 0031) y los transitorios continúan el estado en el tiempo (ADR 0029).

- **`ValueError`: *'previous' es una perturbación lineal…***

El resultado de un `LinearSolver` sobre una etapa no cambia el estado de los elementos, así que no se puede continuar desde él: la etapa siguiente debe tomar como `previous` la etapa sobre la que se calculó la perturbación.

- **`ValueError`: *una etapa estática no puede continuar un análisis en el tiempo…***

Con `stages`, una etapa estática sólo puede seguir a otra estática: qué carga se mantiene y qué pasa con la velocidad al pasar de un análisis en el tiempo a uno estático son decisiones físicas todavía pendientes. Las etapas modal, armónica y espectral sí pueden seguir a cualquiera (linealizan alrededor de su estado).

- **`YamlValidationError` en `stages`** (*stages: con etapas, el solver y las cargas se declaran en cada etapa…*, *stages[i]: falta el bloque 'solver'*)

Con `stages`, el `solver` y las cargas (`point_loads…`, `gravity`, `body_force`, `tractions…`, `thermal_loads`) van dentro de cada etapa (`loads` y `solver`); declararlos además en el nivel raíz es un error. Cada etapa admite sólo `name`, `loads` y `solver`.

- **Aviso:** *ModalSolver: N autovalor(es) negativo(s)… el estado es inestable en esos modos*

Un modal sobre el estado de una etapa (ADR 0031) usa su rigidez tangente; un autovalor negativo indica que ese estado es inestable en ese modo (por ejemplo, una carga axial por encima de la de pandeo). Los modos quedan en `ModalResult.unstable_modes`, con su autovalor en `eigenvalues` y frecuencia y periodo `NaN`. Un `ResponseSpectrumSolver` sobre ese estado se detiene con `ValueError` (*el estado sobre el que se calcula tiene modos inestables*): revisar la carga de la etapa.

- `ValueError` **de** `EmbeddedReinforcement` (refuerzo embebido, bloque `constraints` o `solidum.embed_reinforcement`)
- *el punto … de la barra no está dentro de ningún elemento anfitrión / la barra sale de los anfitriones cerca de … / el punto medio … de un tramo no está dentro de ningún anfitrión*: la barra debe quedar entera dentro de los sólidos anfitriones (o de los `hosts` indicados); revisar sus `points` o ampliar `hosts`.
- *el dominio no tiene elementos de sólido 2D/3D donde embeber la barra / los elementos … no son sólidos isoparamétricos*: la barra se embebe en sólidos con funciones de forma (Tri3, Quad4, Hex8…); la llamada va después de crear esos elementos.
- *… no es un elemento 1D de N nodos con traslaciones … / … tiene N nodos; la barra admite elementos de línea de dos o tres nodos*: `element` debe ser una barra (`Truss2D`, `Truss3D`, `Truss2D3`, `Truss3D3`…), de la dimensión del sólido.
- *Bond: law debe ser un material de interfaz de línea (GEOMETRY='line')*, *Bond: diameter debe ser > 0*, *bond sin 'law'* o *la ley de adherencia … no admite N componentes*: la ley de `bond` debe ser una ley de interfaz de **línea** (`BondSlipModelCode`, `BondSlipMultilinear`) declarada en `interface_materials`, con `diameter` > 0; una ley de grieta (de superficie) no sirve.
- *bond_forces: con adherencia-deslizamiento no hay restricciones de embebido… / bond_state: la barra tiene adherencia perfecta*: con `bond` las fuerzas de adherencia se leen con `EmbeddedBar.bond_state`; sin `bond`, con `bond_forces`.
- **Aviso:** *CST_Embedded2D: el Newton local del salto no convergió en N iteraciones…*

El salto de un elemento con discontinuidad embebida no se pudo despejar en el paso: el cálculo sigue con el residuo local, que el Newton global ve como desequilibrio. Si se repite, reducir el paso. Con la ley de modo mixto `CohesiveDamageMixedMode`, la causa típica es un elemento demasiado grande para el ablandamiento en cortante: el sistema local sólo tiene solución única si $\beta^2|T'|\,h < G$ (sección «Materiales de interfaz» del capítulo «Catálogo de Modelos Constitutivos»); refinar la malla en la zona de la grieta.

- **Niveles del registro.**

Con el nivel por omisión (`INFO`) cada solver incremental escribe una línea por paso convergido. El detalle por iteración de Newton sale con `solidum.set_log_level("DEBUG")`. Un `WARNING` de bisección («sin convergencia (tangente singular); bisección…») es un evento que el solver resolvió solo; un `ERROR` significa siempre que el análisis se detuvo.

- **Consola de Windows (cp1252) o ASCII.**

Si la consola no tiene un carácter de un mensaje (letras griegas, subíndices, flechas), Solidum lo escribe en texto plano: $\Delta\lambda_1$ sale como `Deltalambda_1` y $\Rightarrow$ como `=>`. No es un error. Con una consola UTF-8 (`PYTHONIOENCODING=utf-8`) los mensajes salen tal cual.

- `MemoryError: Memoria insuficiente para factorizar…`

El modelo es demasiado grande para el solver directo, cuya memoria crece más deprisa que el modelo. Usar `linear_algebra: iterative` en el bloque `solver` (ver el anexo de la capa algebraica).

- **Cables y materiales con ablandamiento.**

En modelos con cables o materiales con softening pronunciado, un elemento que se destensa o se degrada puede dejar un mecanismo: comprobar que existen caminos de carga alternativos.

- `Newton-Raphson no convergió / bisecando incremento.`

El paso de carga es demasiado grande. Aumentar `num_steps`, asegurar `adaptive: true`. Verificar tipeo de los parámetros constitutivos (σ_y, κ_0, etc.). Si el problema tiene snap-through o softening, cambiar a `ArcLengthSolver`.

- `No se importó ningún elemento (Solid2D).`

El parser de Gmsh no encontró superficies bidimensionales. En Gmsh, crear una *Plane Surface* y generar la malla 2D antes de exportar.

- `Cable totalmente destensado --- KT = 0.`

Pretensar el cable o garantizar que la estructura tiene rigidez residual por otros elementos. Considerar también pasos de carga finos para capturar la transición tensado $\rightarrow$ destensado.

Solidum FEM — Manual de Usuario

Sigla del manual: `FF-MU`

Compilado el 11 de noviembre de 2026, 10:12

Commit Git: `bfff90e`

Documento generado automáticamente desde los archivos fuente en `manuals/sources/user/` mediante `python manuals/build_user_manual.py`. No editar directamente el PDF: cualquier corrección debe realizarse sobre el archivo fuente correspondiente y regenerar el manual.